

УНИВЕРЗИТЕТ У БЕОГРАДУ
ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ



Криптографски акцелератор на FPGA платформи
заснован на ECDH, SHA-3 и AES-GCM
алгоритмима

Мастер рад

Студент:
Марко Гавриловић, 2024/3101

Ментор:
др Владимир Петровић, доцент

Београд, 2026.

Sadržaj

Lista slika	7
1 Uvod	10
2 Kriptografske osnove	12
2.1 Osnovni pojmovi i model komunikacije	12
2.2 Sigurnosni zahtevi	12
2.3 Klase kriptografskih algoritama	14
2.3.1 Algoritmi sa simetričnim ključem	14
2.3.1.1 Režimi rada blok šifara	14
2.3.1.2 Autentifikovano šifrovanje	15
2.3.2 Algoritmi sa asimetričnim ključem	15
2.3.3 Kriptografske heš funkcije	16
2.4 Hibridni kriptosistem	17
3 Algoritmi	20
3.1 ECDH – razmena ključeva na eliptičkim krivama	20
3.1.1 Diffie–Hellmanov protokol	20
3.1.2 Eliptičke krive	20
3.1.3 Binarno polje $GF(2^m)$	21
3.1.4 Kriva B-571	22
3.1.5 Inverzija: Itoh–Tsujiijeva metoda	22
3.1.6 Skalarno množenje i problem diskretnog logaritma	24
3.1.7 Montgomeryjeve merdevine	24
3.1.8 ECDH protokol	26
3.2 SHA-3	27
3.2.1 Nastanak standarda	27
3.2.2 Sponge konstrukcija	27
3.2.3 Dopuna poruke i razdvajanje domena	28
3.2.4 Sigurnosna svojstva sponge konstrukcije	29
3.2.5 Keccak- $f[1600]$ permutacija	30
3.2.5.1 Korak θ	31
3.2.5.2 Korak ρ	32
3.2.5.3 Korak π	32
3.2.5.4 Korak χ	33
3.2.5.5 Korak ι	33
3.2.6 Varijante standarda	34
3.2.7 Izvedene funkcije: cSHAKE i KMAC	34
3.3 AES-GCM	37
3.3.1 Nastanak standarda	38
3.3.2 Arhitektura algoritma	38

3.3.3	Tok šifrovanja	39
3.3.4	Transformacije u rundi	41
3.3.4.1	SubBytes	41
3.3.4.2	ShiftRows	41
3.3.4.3	MixColumns	42
3.3.4.4	AddRoundKey	43
3.3.5	Fuzija runde: T-tabele	43
3.3.6	Ekspanzija ključa	45
3.3.7	Dešifrovanje	46
3.3.8	Režimi rada blok šifara	46
3.3.8.1	ECB režim	46
3.3.8.2	CBC režim	47
3.3.8.3	CTR režim	47
3.3.8.4	Od CTR do GCM	48
3.3.9	GCM režim rada	49
3.3.9.1	CTR šifrovanje	49
3.3.9.2	GHASH funkcija	50
3.3.9.3	Množenje u $GF(2^{128})$	51
3.3.9.4	Dešifrovanje i provera oznake	52
3.3.9.5	Paralelizacija GHASH funkcije	53
3.3.10	Sigurnosna svojstva i ograničenja	53
4	Hardverska realizacija	55
4.1	Zajednička arhitektura sistema	55
4.1.1	Ciljna arhitektura i obim rada	55
4.1.2	AXI4-Stream interfejs	55
4.1.3	Konfiguracija i platforma	56
4.2	ECDH akcelerator	57
4.2.1	Organizacija akceleratora	57
4.2.2	Operacije u polju	58
4.2.3	Aritmetička jedinica	60
4.2.4	Iteracija merdevina u hardveru	60
4.2.5	Kontroler merdevina i završna konverzija	62
4.2.6	Osnovno i paralelno jezgro	64
4.2.7	Uklapanje u sistem i provera	64
4.3	SHA-3 akcelerator	65
4.3.1	Arhitektura jezgra	65
4.3.2	Ulazni bafer i dopuna	66
4.3.3	Sponge kontroler i Keccak jezgro	67
4.3.4	Dve realizacije jezgra	68
4.3.5	Izlazni bafer	69
4.3.6	Podrška za cSHAKE i KMAC	69
4.3.7	Uklapanje u sistem i provera	70
4.4	AES-GCM akcelerator	70
4.4.1	Organizacija akceleratora	70
4.4.2	AES jezgro	71
4.4.2.1	Šta jezgro računa	71
4.4.2.2	Dve arhitekture jezgra	71
4.4.2.3	Ekspanzija ključa u hodu	73
4.4.2.4	Upravljanje tokom: FLOW_STYLE	74

4.4.2.5	Realizacija runde: <code>ROUND_STYLE</code>	76
4.4.3	GHASH jedinica	77
4.4.3.1	Množlač u $GF(2^{128})$	77
4.4.3.2	Dve vremenske varijante	78
4.4.4	GCM sloj	78
4.4.4.1	Tok kroz GCM sloj na putu šifrovanja	79
4.4.4.2	Prevođenje redosleda bajtova	80
4.4.4.3	Kapila svežine heš potključa	80
4.4.4.4	Razlike puta dešifrovanja	81
4.4.5	Sistemska omotač	82
4.4.6	Pogodnost za dinamičku rekonfiguraciju	84
4.4.7	Uklapanje u sistem i provera	84
4.5	Integracija jezgara u lanac	84
5	Verifikacija i rezultati	86
5.1	Metodologija verifikacije	86
5.2	ECDH akcelerator	87
5.2.1	Provera po nivoima	87
5.2.2	Zauzeće resursa i radna učestanost	88
5.2.3	Latencija skalarnog množenja	90
5.3	SHA-3 akcelerator	92
5.3.1	Provera po nivoima	92
5.3.2	Zauzeće resursa i radna učestanost	93
5.3.3	Propusnost i model uskog grla	94
5.3.4	Upravljački program i merni put	96
5.3.5	Rad na ploči i poređenje sa softverom	97
5.4	AES-GCM akcelerator	98
5.4.1	Provera po nivoima	98
5.4.2	Zauzeće resursa i radna učestanost	99
5.4.3	Poređenje protočne i višejezgarne arhitekture	100
5.4.4	Propusnost u ustaljenom stanju	101
5.4.5	Poređenje stilova upravljanja tokom	103
5.4.6	Mrežni paketi	104
5.4.7	Ceo put podataka	105
5.5	Sistem u celini	106
6	Potencijalna unapređenja	108
6.1	Od simulisanog lanca do sistema na ploči	108
6.2	Pomeranje granice propusnosti puta podataka	109
6.3	Upravljačka ravan i budućnost razmene ključa	110
7	Zaključak	111
A	Matematičke osnove	113
A.1	Grupa, prsten i polje	113
A.2	Konačna (Galoisova) polja	113
A.3	Polje $GF(2^8)$ u AES-u	114
A.4	Polje $GF(2^{571})$ kod eliptičke krive B-571	114
A.5	Polje $GF(2^{128})$ u GHASH funkciji	114

B	AES tabele	115
B.1	S-boks	115
B.2	Konstante rundi ekspanzije ključa	116
C	Parametri krive B-571	117
D	Keccak konstante	118
D.1	Rotacioni ofseti	118
D.2	Konstante rundi	118
	Reference	120

Lista slika

2.1	Tok hibridnog kriptosistema	18
3.1	Geometrijsko sabiranje tačaka na eliptičkoj krivoj (prikaz nad realnim brojevima)	21
3.2	Itoh–Tsujiijev adicioni lanac za $m - 1 = 570$: crna strelica je udvostručenje, narandžasta uvećanje za jedan	23
3.3	Jedna iteracija Montgomeryjevih merdevina	26
3.4	Tok ECDH protokola	26
3.5	Sponge konstrukcija	28
3.6	Dopuna poruke u SHA-3 familiji	28
3.7	Stanje permutacije Keccak- $f[1600]$	30
3.8	Jedna runda permutacije Keccak- $f[1600]$	30
3.9	Korak θ za ravan $x = 1$ – svaka traka ravni (zeleno) sabira se sa trakom parnosti ravni $x = 0$ (cijan) i rotiranom trakom parnosti ravni $x = 2$ (žuto)	32
3.10	Korak ρ : ciklična rotacija trake	32
3.11	Korak π : preslikavanje svih 25 traka	33
3.12	Korak χ : logika za jednu traku	33
3.13	Korak ι	34
3.14	Uokvirivanje ulaznog niza KMAC funkcije	36
3.15	Matrica stanja AES algoritma	38
3.16	Koraci u AES šifrovanju	40
3.17	Prikaz jedne runde AES šifrovanja	40
3.18	SubBytes transformacija na primeru bajta $\{53\} \mapsto \{ED\}$	41
3.19	ShiftRows transformacija	41
3.20	MixColumns transformacija nad jednom kolonom matrice stanja	42
3.21	AddRoundKey transformacija	43
3.22	Fuzija runde T-tabelama	44
3.23	Ekspanzija ključa za AES-256: izračunavanje kolone W_j , $j \geq 8$, po slučajevima	46
3.24	AES-128 šifrovanje slike u različitim režimima rada	47
3.25	CTR režim rada: blokovi se obrađuju nezavisno i paralelno	48
3.26	GHASH funkcija kao Hornerova šema	50
3.27	Blok šema GCM šifrovanja poruke od dva bloka: CTR grana (žuto-/zeleno) i GHASH grana (cijan)	51
3.28	Karatsubin algoritam	52
3.29	Paralelizovani GHASH za $p = 2$	53
4.1	Tok podataka kroz ECDH akcelerator	58
4.2	Digit-serijski množak polja	59
4.3	Aritmetička jedinica sa deljenim množačem	60

4.4	Ista iteracija merdevina u dva rasporeda resursa	62
4.5	Petlja kontrolera merdevina	63
4.6	Tri stepena SHA-3 akceleratora	65
4.7	Preklapanje rada stepena za SHA3-256 uz <code>DATA_WIDTH = 32</code> , gde 34 ulazne reči i takt predaje bloka (uski tamni segment) stoje prema 24 takta permutacije	65
4.8	Mašina stanja ulaznog bafera SHA-3 akceleratora	67
4.9	Mašina stanja sponge kontrolera	68
4.10	Tok podataka Keccak jezgra za obe arhitekture	69
4.11	Protočna arhitektura AES jezgra	72
4.12	Kružna raspodela u višejezgarnoj arhitekturi	73
4.13	Ključ i nonce pristigli usred paketa (t_2) primenjuju se tek na ciklusu pražnjenja (t_5)	74
4.14	Dva načina upravljanja tokom u protočnoj arhitekturi: (a) jedna za- jednička dozvola, pa se protok pomera u celini, a (b) po jedan brojač zauzeća po stepenu, pa glava protoka nastavlja da prima dok rep stoji	75
4.15	Rekurzivna Karatsubina dekompozicija množača. Nastavak ispod ni- voa 32 prikazan je za jedan čvor, a isti je za svih devet	77
4.16	Moduli GCM sloja na putu šifrovanja	79
4.17	Moduli GCM sloja na putu dešifrovanja	82
4.18	Sistemske omotač na putu šifrovanja	83
4.19	Sistemske omotač na putu dešifrovanja	83
4.20	Ciljna organizacija lanca izvođenja i upotrebe ključa	85
5.1	Zauzeće i radna učestanost ECDH jezgra po vrednostima generika <code>G_D</code> . Uz tačku osnovnog jezgra sa $D = 128$ i paralelnog sa $D = 64$ označeno je i zauzeće u procentima čipa	89
5.2	Radna učestanost po vrednostima generika <code>G_D</code> , sa zauzećem čipa uz dve tačke paralelnog jezgra	90
5.3	Zauzeće i trajanje jednog skalarnog množenja za sve merene konfigu- racije. Označeno je pet tačaka o kojima govori tekst	92
5.4	Zauzeće i radna učestanost svih četrnaest SHA-3 konfiguracija	94
5.5	Propusnost dve arhitekture za četiri merene kombinacije varijante i širine magistrale, pri izmerenoj radnoj učestanosti	95
5.6	Put podataka u merenjima na ploči	96
5.7	Propusnost u zavisnosti od veličine datoteke	98
5.8	Zauzeće i propusnost svih šesnaest AES konfiguracija. Brojevi uz tač- ke višejezgarne LUT varijante su broj jezgara, isprekidane linije su nivoi zauzeća protočne arhitekture	101
5.9	Propusnost višejezgarne arhitekture u zavisnosti od broja jezgara	103
5.10	Lanac u simulaciji sa dve strane. Crvene strelice nose tajni materijal	107

Lista tabela

2.1	Sigurnosni zahtevi, mehanizmi i njihova realizacija u ovom radu . . .	14
2.2	Poređenje klasa kriptografskih algoritama	16
2.3	Sigurnosni nivo pojedinih karika sistema. Nivo ECDH jezgra je polovina bitske dužine reda tačke (odeljak 3.1.6)	19
3.1	Cena skalarnog množenja na krivoj B-571 po Lópezovim i Dahabovim formulama	25
3.2	Sufiksi za razdvajanje domena u SHA-3 familiji	29
3.3	Varijante SHA-3 standarda	34
3.4	Pomoćne funkcije kodiranja iz standarda SP 800-185	35
3.5	Uloge parametara KMAC funkcije: autentifikacija poruka naspram izvođenja ključeva	37
3.6	Varijante AES algoritma	39
3.7	Poređenje CTR i GCM režima rada	48
4.1	Osnovni signali AXI4-Stream interfejsa	56
4.2	Generici ECDH akceleratora	58
4.3	Mikroprogram jedne iteracije merdevina u osnovnom jezgru	61
4.4	Mikroprogram završne konverzije: x i y su koordinate bazne tačke, x_1 , x_2 i y_1 affine koordinate rezultata, a s_i pomoćne vrednosti	64
4.5	Generici SHA-3 akceleratora	66
4.6	Segmenti paketa u AES-GCM akceleratoru	70
4.7	Broj kopija memorijskog bloka po rundi za dve organizacije T-tabela	76
4.8	Broj množenja po nivoima stabla: školska dekompozicija naspram Karatsubine	77
4.9	Uticaj vremenske organizacije GHASH jedinice na granicu propusnosti akceleratora	78
4.10	Redosled bajtova kroz akcelerator, na primeru bloka 00 11 22 ... ff	80
5.1	Nivoi provere po jezgru	87
5.2	Osnovno ECDH jezgro, implementacija van konteksta	88
5.3	Paralelno ECDH jezgro, implementacija van konteksta	88
5.4	Trajanje jednog skalarnog množenja. Kolona $q = \lceil 571/D \rceil$ je broj digita koje množać obradi po jednom množenju u polju. Vreme t dobija se deljenjem broja taktova radnom učestanošću iz tabela 5.2 i 5.3, a poslednja kolona je odnos broja taktova dva jezgra pri istom D . Paralelno jezgro sa $D = 128$ nema vreme jer ne staje na ploču, pa mu se broj taktova zna iz simulacije, a radna učestanost ne postoji	91
5.5	Arhitektura I (jedna runda po taktu), implementacija van konteksta. Kolona usko grlo sledi iz modela u odeljku 5.3.3	93
5.6	Arhitektura II (dve runde po taktu), implementacija van konteksta. Kolona usko grlo sledi iz modela u odeljku 5.3.3	93

5.7	Izmerena vrednost bita po taktu i propusnost izvedena iz nje pri F_{max} iz tabela 5.5 i 5.6	94
5.8	Propusnost pri heširanju datoteke od 100 MiB, varijanta SHA3-512	97
5.9	Protočna arhitektura, implementacija van konteksta	99
5.10	Višezgarna arhitektura, implementacija van konteksta	99
5.11	GHASH jedinica i GCM sloj, implementacija van konteksta	99
5.12	Izmerena propusnost puta šifrovanja u ustaljenom stanju, uz jedno- taktanu GHASH jedinicu. Ulaz je poruka od 512 reči po 128 bitova, jezgro se meri bez sistemskog omotača	102
5.13	Propusnost pri obe vremenske organizacije GHASH jedinice	103
5.14	Propusnost pri mrežnim veličinama paketa, protočna arhitektura	104
5.15	Ceo put podataka nad paketima ESP tunela. Red <i>GCM lanac</i> je GCM sloj sa AES jezgrom, red <i>sa omotačem</i> isti lanac uz razdvajanje, sa- stavljanje i poravnanje oznake. Obe arhitekture su u LUT varijanti, a GHASH jedinica je jednotaktna	105
5.16	Propusnost celog puta podataka u ustaljenom stanju, nad paketima sa korisnim sadržajem od 1504 bajta. Kolona <i>bez omotača</i> je isti sadržaj iz tabele 5.14	106
B.1	AES S-boks: vrsta je gornji, kolona donji polubajt ulaza (heksadeci- malno)	115
B.2	Rcon konstante ekspanzije ključa (prvi bajt, heksadecimalno)	116
D.1	Rotacioni ofseti $r_{x,y}$ koraka ρ	118
D.2	Konstante rundi RC_i permutacije Keccak- $f[1600]$ (heksadecimalno)	119

Poglavlje 1

Uvod

Savremeni svet počiva na razmeni digitalnih informacija. Količina podataka koja se svakodnevno prenosi kroz računarske mreže neprestano raste, a sa njom i potreba da ti podaci budu zaštićeni, kako od neovlašćenog čitanja, tako i od neprimetne izmene. Kriptografija, nekada disciplina rezervisana za vojne i diplomatske potrebe, danas je ugrađena u gotovo svaki aspekt digitalnog života, od onlajn bankarstva i elektronske trgovine, preko virtuelnih privatnih mreža, do komunikacije uređaja u industrijskim postrojenjima i automobilima.

Zaštita komunikacije u praksi nikada ne počiva na jednom kriptografskom algoritmu. Bezbedan komunikacioni kanal zahteva rešavanje više nezavisnih problema. Dve strane najpre moraju da uspostave zajednički tajni ključ preko nezaštićenog medijuma, zatim da iz tog zajedničkog materijala izvedu ključeve sesije, i konačno da svaku poruku šifruju tako da primalac može da proveriti i njenu poverljivost i njen integritet. Moderni protokoli, poput TLS-a (engl. *Transport Layer Security*) ili IPsec-a, zbog toga kombinuju više algoritama različitih klasa u jedinstven hibridni kriptosistem: asimetrične algoritme za razmenu ključeva, heš funkcije za izvođenje ključeva i proveru integriteta, i simetrične šifre za zaštitu samih podataka.

Sa porastom brzina mrežnih interfejsa, softverska realizacija ovih algoritama na procesorima opšte namene postaje usko grlo. Kriptografske operacije su računski intenzivne, a njihovo izvršavanje na procesoru troši resurse koji bi mogli biti iskorišćeni za samu aplikaciju. Prirodno rešenje je izmeštanje kriptografskih operacija u namenski hardver, takozvani kriptografski akcelerator. FPGA (engl. *Field Programmable Gate Array*) platforme su za ovu namenu posebno pogodne, jer nude paralelizam i propusnost blisku namenskim integrisanim kolima, uz zadržavanje fleksibilnosti rekonfiguracije, što je značajno u oblasti u kojoj se standardi i preporuke vremenom menjaju. Kao ciljna platforma u ovom radu izabran je razvojni sistem Kria KR260, sa čipom Zynq UltraScale+ MPSoC. Taj čip na istom silicijumu objedinjuje programabilnu logiku i procesorski sistem, pa akcelerator može da radi samostalno, ali i pod upravom softvera koji se izvršava na istom čipu. Ta mogućnost iskorišćena je za ispitivanje jednog od realizovanih jezgara na ploči, preko namenskog upravljačkog programa.

Cilj ovog rada je projektovanje kriptografskog akceleratora na FPGA platformi koji objedinjuje tri algoritma savremene kriptografije, po jedan iz svake ključne klase:

- ECDH (engl. *Elliptic Curve Diffie–Hellman*) – protokol za uspostavljanje zajedničkog tajnog ključa zasnovan na eliptičkim krivama, realizovan nad binarnim poljem korišćenjem krive B-571;

- SHA-3 (engl. *Secure Hash Algorithm 3*) – standardizovana kriptografska heš funkcija zasnovana na Keccak algoritmu, korišćena kroz KMAC konstrukciju za izvođenje ključeva i proveru integriteta;
- AES-GCM (engl. *Advanced Encryption Standard – Galois/Counter Mode*) – autentifikovana simetrična šifra koja u jednom prolazu obezbeđuje i poverljivost i integritet podataka.

Ova tri algoritma zajedno čine funkcionalnu celinu u kojoj ECDH obezbeđuje zajedničku tajnu, SHA-3 iz nje izvodi ključeve sesije, a AES-GCM tim ključevima štiti korisnički saobraćaj. Za potpun bezbedan sistem neophodna je još i autentifikacija učesnika u razmeni ključeva, koja se tipično ostvaruje algoritmom digitalnog potpisa kao što je ECDSA (engl. *Elliptic Curve Digital Signature Algorithm*). Njegova realizacija prevazilazi obim ovog rada, ali je njegoova uloga u sistemu razmotrena na odgovarajućim mestima.

Sve hardverske arhitekture opisane u radu realizovane su u jeziku za opis hardvera VHDL, korišćenjem alata AMD Vivado. Provera je izvedena na više nivoa, simulacijom nad standardizovanim test vektorima, namenskim testovima graničnih slučajeva i merenjem na samoj ploči. SHA-3 akcelerator pušten je pod operativnim sistemom PetaLinux, preko upravljačkog programa napisanog za tu namenu, a AES-GCM akcelerator proveren je umetnut u postojeći mrežni sistem. Pored hardverskih realizacija, razvijeni su i softverski referentni modeli, koji služe za proveru ispravnosti i kao osnova za poređenje performansi.

Rad je organizovan na sledeći način. U drugom poglavlju dat je pregled osnovnih pojmova kriptografije, od sigurnosnih zahteva i klasa algoritama do načina na koji se oni kombinuju u hibridni kriptosistem kakav je realizovan u ovom radu.

Treće poglavlje teorijski obrađuje tri korišćena algoritma, od matematičkih osnova eliptičkih krivih i ECDH protokola, preko sponge konstrukcije i Keccak permutacije na kojima počiva SHA-3, do AES algoritma u GCM režimu rada sa pripadajućom GHASH funkcijom.

U četvrtom poglavlju opisane su hardverske arhitekture akceleratora za svaki od algoritama i njihova integracija u sistem.

Peto poglavlje prikazuje metodologiju verifikacije i postignute rezultate, zauzeće resursa, radne učestanosti, latenciju skalarnog množenja i propusnost, uz poređenje SHA-3 jezgra sa softverskim realizacijama.

Šesto poglavlje razmatra moguća unapređenja, od zaokruživanja sistema na ploči, preko pomeranja granice propusnosti puta podataka, do razvoja upravljačke ravni i postkvantne zamene razmene ključa, a sedmo donosi zaključak.

Poglavlje 2

Kriptografske osnove

Ovo poglavlje uspostavlja pojmovni okvir na kome počiva ostatak rada: model komunikacije i osnovnu terminologiju, sigurnosne zahteve koje savremeni komunikacioni sistem mora da ispuni, tri klase kriptografskih algoritama i način na koji se one povezuju u hibridni kriptosistem kakav je realizovan u ovom radu. Sistematičan i detaljniji uvod u kriptografiju dat je u [1, 2].

2.1 Osnovni pojmovi i model komunikacije

Kriptografija proučava tehnike zaštite informacija u prisustvu protivnika. Standardni model podrazumeva dve strane koje komuniciraju preko nezaštićenog kanala i protivnika koji taj kanal može da prisluškuje, a u opštem slučaju i da aktivno menja, briše ili ubacuje poruke. Pošiljalac se tradicionalno zove Alisa, primalac Bob, a protivnik Oskar. U literaturi se sreću i druga imena, najčešće Eva za pasivnog prislušivača i Malori za aktivnog napadača. U ovom radu dosledno se koristi Oskar, prema konvenciji iz [1], i to za obe vrste protivnika.

Postupak transformacije izvorne poruke, otvorenog teksta (engl. *plaintext*), u nečitljiv oblik, šifrat (engl. *ciphertext*), naziva se šifrovanje (engl. *encryption*), a obrnuti postupak dešifrovanje (engl. *decryption*). Obe transformacije parametrizovane su ključem. Prema Kerckhoffsovom principu (engl. *Kerckhoffs's principle*), formulisano još 1883. godine [3], sigurnost sistema ne sme počivati na tajnosti samog algoritma, već isključivo na tajnosti ključa. Algoritam se smatra javno poznatim, a jedino što Oskar ne poznaje jeste ključ. Ovaj princip je danas univerzalno prihvaćen [1], pa su svi algoritmi korišćeni u ovom radu javni, međunarodno standardizovani i detaljno proučeni u stručnoj zajednici.

2.2 Sigurnosni zahtevi

Bezbedan komunikacioni sistem mora da obezbedi više međusobno nezavisnih svojstava. Za ovaj rad ključna su sledeća četiri.

Poverljivost (engl. *confidentiality*) garantuje da sadržaj poruke ostaje nedostupan svakome ko ne poseduje odgovarajući ključ. Ovo je istorijski najstariji i najintuitivniji zahtev kriptografije. Oskar, koji prisluškuje kanal, ne može iz šifrata da rekonstruiše otvoreni tekst, niti da o njemu sazna bilo koju delimičnu informaciju.

Integritet (engl. *integrity*) garantuje da poruka nije izmenjena na putu od pošiljaoca do primaoca. Važno je uočiti da poverljivost sama po sebi ne obezbeđuje

integritet. Kod mnogih šifara protivnik može ciljano da izmeni šifrat tako da izazove predvidljivu promenu dešifrovanog teksta, ne znajući pritom ključ. Zbog toga se integritet mora obezbediti posebnim mehanizmima, kao što su kodovi za autentifikaciju poruka (engl. *Message Authentication Code*, MAC). Bitno je da takav kod koristi ključ. Običan sažetak štiti samo od slučajnih izmena, jer ga protivnik može preračunati zajedno sa izmenjenom porukom, dok oznaku izračunatu sa tajnim ključem može proizvesti isključivo onaj ko taj ključ poseduje.

Autentičnost (engl. *authenticity*) garantuje da poruka zaista potiče od strane koja tvrdi da ju je poslala. Korisno je razlikovati dva nivoa, poreklo pojedinačne poruke i identitet učesnika u vezi. *Autentičnost porekla poruke* sledi neposredno iz mehanizma integriteta. Pošto ispravnu oznaku može izračunati samo posednik ključa, MAC istovremeno obezbeđuje i integritet i poreklo, pa se ta dva svojstva u praksi i ne razdvajaju. *Autentičnost učesnika* je, međutim, pitanje ko se uopšte nalazi na drugom kraju veze, i ono se mora rešiti pre nego što zajednički ključ i postoji. Ako Alisa ne može da proveriti da pregovara baš sa Bobom, Oskar može da se postavi između njih i sa svakim od njih uspostavi zaseban ključ, u napadu poznatom kao čovek u sredini (engl. *man-in-the-middle*, MITM). Ovaj drugi nivo ostvaruje se digitalnim potpisima i infrastrukturu javnih ključeva.

Dva nivoa se dopunjuju, a ne zamenjuju. Oznaka koju računa režim autentifikovanog šifrovanja (odjeljak 2.3.1.2) dokazuje samo da poruka potiče od one strane sa kojom je ključ uspostavljen. Ako sama uspostava ključa nije autentifikovana, ta strana može biti i Oskar, pa otuda i redosled u protokolu. Potpis štiti razmenu ključeva, a oznaka štiti saobraćaj kada ključ već postoji.

Svežina poruka je četvrti zahtev. Čak i kada su poverljivost, integritet i autentičnost obezbeđeni, Oskar može da snimi ispravnu šifrovanu poruku i da je kasnije ponovo pošalje, u napadu ponavljanjem (engl. *replay attack*). Odbrana traži da svaka poruka nosi element koji se ne ponavlja, a takva vrednost se naziva nonce (engl. *number used once*): redni broj, vremenska oznaka ili nasumična vrednost dovoljne dužine.

Nonce vrednost se u ovom radu javlja u dve uloge, i jedna ne zamenjuje drugu. Kao mehanizam svežine, treba da omogući primaocu da odbaci poruku koju je već video. Unutar GCM režima, gde nonce ulazi kroz parametar koji standard zove inicijalizacioni vektor (engl. *initialization vector*, *IV*), zahtev da *IV* bude jedinstven obavezuje samo pošiljaoca, i to zato da isti pseudoslučajni niz ne upotrebi dvaput. Primalac pri tome ne pamti koje je vrednosti *IV* ranije primio, pa snimljen paket sa svojim izvornim *IV* i ispravnom oznakom prolazi proveru i drugi put. Zaštita od ponavljanja je zato zaseban mehanizam na nivou protokola, tipično brojač sekvence i prozor prihvatanja kod primaoca, i nije deo GCM režima.

Tabela 2.1 povezuje ova četiri zahteva sa mehanizmima kojima se ostvaruju i sa njihovom realizacijom u sistemu opisanom u ovom radu.

Tabela 2.1: Sigurnosni zahtevi, mehanizmi i njihova realizacija u ovom radu

Zahtev	Mehanizam	Realizacija u ovom radu
poverljivost	simetrično šifrovanje	AES u CTR režimu (deo GCM-a)
integritet i autentičnost poruke	kod za autentifikaciju poruka (MAC)	GHASH i oznaka u GCM režimu
autentičnost učesnika	digitalni potpis i sertifikati	ECDSA, predviđena nadogradnja
svežina poruka	brojač sekvence i prozor prihvatanja	van obima rada

2.3 Klase kriptografskih algoritama

Savremeni kriptografski algoritmi dele se u tri osnovne klase, koje se međusobno razlikuju po načinu upotrebe ključeva i po problemima koje rešavaju.

2.3.1 Algoritmi sa simetričnim ključem

Kod simetričnih algoritama obe strane koriste isti tajni ključ i za šifrovanje i za dešifrovanje. Glavne predstavnike čine blok šifre (engl. *block cipher*), koje podatke obrađuju u blokovima fiksne dužine, i protočne šifre (engl. *stream cipher*), koje generišu pseudoslučajni niz kojim se otvoreni tekst kombinuje bit po bit. Najznačajniji savremeni predstavnik blok šifara je AES, usvojen 2001. godine kao američki federalni standard za obradu informacija (engl. *Federal Information Processing Standard*, FIPS) pod oznakom FIPS 197 [4].

AES nije jedini simetrični algoritam u upotrebi. Trostruka primena starijeg standarda DES (engl. *Data Encryption Standard*), poznata kao 3DES, dugo je bila standard u bankarskim sistemima, ali ju je američki Nacionalni institut za standarde i tehnologiju (engl. *National Institute of Standards and Technology*, NIST) povukao iz upotrebe zbog kratkog bloka od 64 bita i niske brzine [5]. Protočna šifra ChaCha20, u sprezi sa autentifikatorom Poly1305, standardizovana je za internet protokole [6] i koristi se u TLS-u 1.3. Njena je posebnost to što je izrazito brza u softveru, na procesorima bez namenskih instrukcija za AES, što je upravo suprotan slučaj od onog koji ovaj rad razmatra. Za uređaje sa strogo ograničenim resursima NIST je 2025. godine standardizovao porodicu ASCON [7]. AES je ovde izabran jer je i dalje osnovni standard, jer ga podržava GCM režim rada, i jer je njegova struktura izrazito pogodna za hardversku realizaciju.

Simetrični algoritmi su računski efikasni i pogodni za hardversku realizaciju, pa se praktično sav korisnički saobraćaj u savremenim protokolima štiti upravo njima. Njihova suštinska slabost je problem distribucije ključa. Pre početka komunikacije obe strane moraju posedovati isti tajni ključ, a njega je nemoguće bezbedno preneti preko istog nezaštićenog kanala koji tek treba zaštititi.

2.3.1.1 Režimi rada blok šifara

Blok šifra sama po sebi definiše transformaciju samo nad jednim blokom fiksne dužine, kod AES-a 128 bitova. Način na koji se šifra primenjuje na poruke proizvoljne dužine definiše režim rada (engl. *mode of operation*) [8]. Režimi se razlikuju i po sigurnosnim svojstvima i po pogodnosti za hardversku realizaciju. Poređenje tri

karakteristična režima dato je u odeljku 3.3.8. Za klasifikaciju je ključno da nijedan od njih sam po sebi ne obezbeđuje integritet poruke.

2.3.1.2 Autentifikovano šifrovanje

Pošto poverljivost i integritet zahtevaju odvojene mehanizme, a njihovo nezavisno kombinovanje se u praksi pokazalo sklono suptilnim greškama, razvijeni su režimi autentifikovanog šifrovanja sa pridruženim podacima (engl. *Authenticated Encryption with Associated Data*, AEAD). AEAD režim u jednom prolazu šifrue poruku i računa autentifikacionu oznaku (engl. *authentication tag*) nad šifratom i pridruženim podacima koji se prenose nešifrovani, na primer nad zaglavljima mrežnih paketa. Primalac odbacuje svaku poruku čija oznaka nije ispravna, pre nego što dešifrovani sadržaj uopšte postane dostupan aplikaciji. Najrasprostranjeniji AEAD režim danas je GCM (engl. *Galois/Counter Mode*) [9], koji kombinuje CTR režim šifrovanja sa autentifikacijom zasnovanom na množenju u Galoisovom polju. Njegova konstrukcija detaljno je opisana u odeljku 3.3.

GCM nije jedini AEAD režim. Postoje i konstrukcije koje autentifikaciju ostvaruju samom blok šifrom, bez zasebne aritmetičke jedinice. Režim OCB [10] postiže autentifikovano šifrovanje u jednom prolazu koristeći isključivo pozive blok šifre, pa u hardveru ne zahteva množać u Galoisovom polju. Njegovo šire usvajanje dugo je kočilo patentno opterećenje, koje je uklonjeno tek 2021. godine. Režim CCM [11], korišćen u standardu WPA2, takođe se oslanja samo na blok šifru, ali zahteva dva prolaza kroz podatke, što ga čini nepogodnim za obradu u liniji. Detaljnija razmatranja ovih režima mogu se naći u navedenim standardima, uz napomenu da je raniji OCB2 kriptanalitički oboren [12], dok OCB3 i dalje važi za siguran. U ovom radu izabran je GCM jer je standard u protokolima TLS, IPsec i MACsec, radi u jednom prolazu i dopušta paralelnu obradu blokova. Cena tog izbora je namensko GHASH jezgro, čija je realizacija razmotrena u poglavlju 4.

2.3.2 Algoritmi sa asimetričnim ključem

Asimetrična kriptografija, ili kriptografija javnog ključa, koju su konceptualno uveli Diffie i Hellman 1976. godine [13], rešava problem distribucije ključa. Svaki učesnik poseduje par matematički povezanih ključeva: javni, koji se slobodno objavljuje, i privatni, koji se čuva u tajnosti. Sigurnost počiva na postojanju jednosmernih funkcija sa zamkom, operacija koje je lako izračunati u jednom smeru, a praktično nemoguće invertovati bez poznavanja privatnog ključa.

Klasični asimetrični sistemi, poput RSA algoritma, zasnivaju se na teškoći faktORIZACIJE velikih brojeva. *Kriptografija eliptičkih krivih* (engl. *Elliptic Curve Cryptography*, ECC), koju su nezavisno predložili Koblitz i Miller sredinom 1980-ih [14, 15], postiže ekvivalentnu sigurnost sa znatno kraćim ključevima. Sigurnosnom nivou RSA ključa od 3072 bita odgovara eliptička kriva reda veličine 256 bitova. Kraći ključevi znače manje memorije, brže operacije i manju potrošnju, što ECC čini posebno pogodnom za hardversku realizaciju.

Asimetrični algoritmi su za nekoliko redova veličine sporiji od simetričnih, pa se u praksi ne koriste za šifrovanje samih podataka, već za dva specifična zadatka: razmenu ključeva (ECDH protokol, odeljak 3.1) i digitalno potpisivanje (na primer ECDSA algoritam), kojim se obezbeđuje autentičnost.

2.3.3 Kriptografske heš funkcije

Kriptografska heš funkcija preslikava poruku proizvoljne dužine u sažetak (engl. *digest*) fiksne dužine. Za razliku od šifara, heš funkcija ne koristi ključ, njena transformacija je javna i deterministička, i nije invertibilna. Iz sažetka se poruka ne rekonstruiše, jer se pri preslikavanju informacija nepovratno sažima.

Da bi bila kriptografski upotrebljiva, heš funkcija mora ispunjavati tri svojstva:

- *otpornost na pronalaženje originala* (engl. *preimage resistance*, u matematičkoj literaturi i praslika) – za dati sažetak h praktično je nemoguće naći poruku m takvu da je $H(m) = h$;
- *otpornost na pronalaženje drugog originala* (engl. *second preimage resistance*) – za datu poruku m_1 praktično je nemoguće naći drugu poruku $m_2 \neq m_1$ sa istim sažetkom;
- *otpornost na kolizije* (engl. *collision resistance*) – praktično je nemoguće naći bilo koji par različitih poruka sa istim sažetkom.

Poslednje svojstvo je najstrože, i ono određuje potrebnu dužinu sažetka. Zbog takozvanog rođendanskog paradoksa [1], koliziju u funkciji sa sažetkom od n bitova moguće je naći nasumičnim probanjem posle približno

$$2^{n/2} \quad (2.1)$$

pokušaja, a ne 2^n , koliko bi se naivno očekivalo. Sažetak stoga mora biti dvostruko duži od željenog nivoa sigurnosti. Funkcija SHA3-256 pruža 128 bitova otpornosti na kolizije, a SHA3-512 njih 256. Ta srazmera objašnjava izbor varijanti u ovom radu i biće ponovo korišćena pri usklađivanju sigurnosnih nivoa sva tri algoritma.

Heš funkcije su univerzalni gradivni blok savremenih protokola, pa se koriste za proveru integriteta, u konstrukcijama MAC kodova, kod digitalnih potpisa, gde se potpisuje sažetak poruke a ne poruka sama, i za izvođenje ključeva (engl. *key derivation*), odnosno transformaciju zajedničke tajne dobijene razmenom ključeva u kriptografski upotrebljive ključeve sesije. Upravo je poslednja uloga ta u kojoj heš funkcija učestvuje u sistemu opisanom u ovom radu.

Aktuelni standard je SHA-3, zasnovan na Keccak algoritmu [16]. Za razliku od porodice SHA-2, koja počiva na Merkle–Damgardovoj konstrukciji, SHA-3 koristi konstrukciju sundera, pa kompromitacija jedne familije ne ugrožava drugu. Usput nema ni svojstvo produženja poruke (engl. *length extension*). Okolnosti pod kojima je standard nastao razmotrene su u odeljku 3.2.

Tabela 2.2 sumira osnovne razlike između tri opisane klase.

Tabela 2.2: Poređenje klasa kriptografskih algoritama

Klasa	Ključ	Brzina	Uloga	U ovom radu
simetrični	zajednički tajni	visoka	zaštita saobraćaja	AES-GCM
asimetrični	javni i privatni	niska	razmena ključeva i potpis	ECDH
heš funkcije	bez ključa	visoka	integritet i izvođenje ključeva	SHA-3 (KMAC)

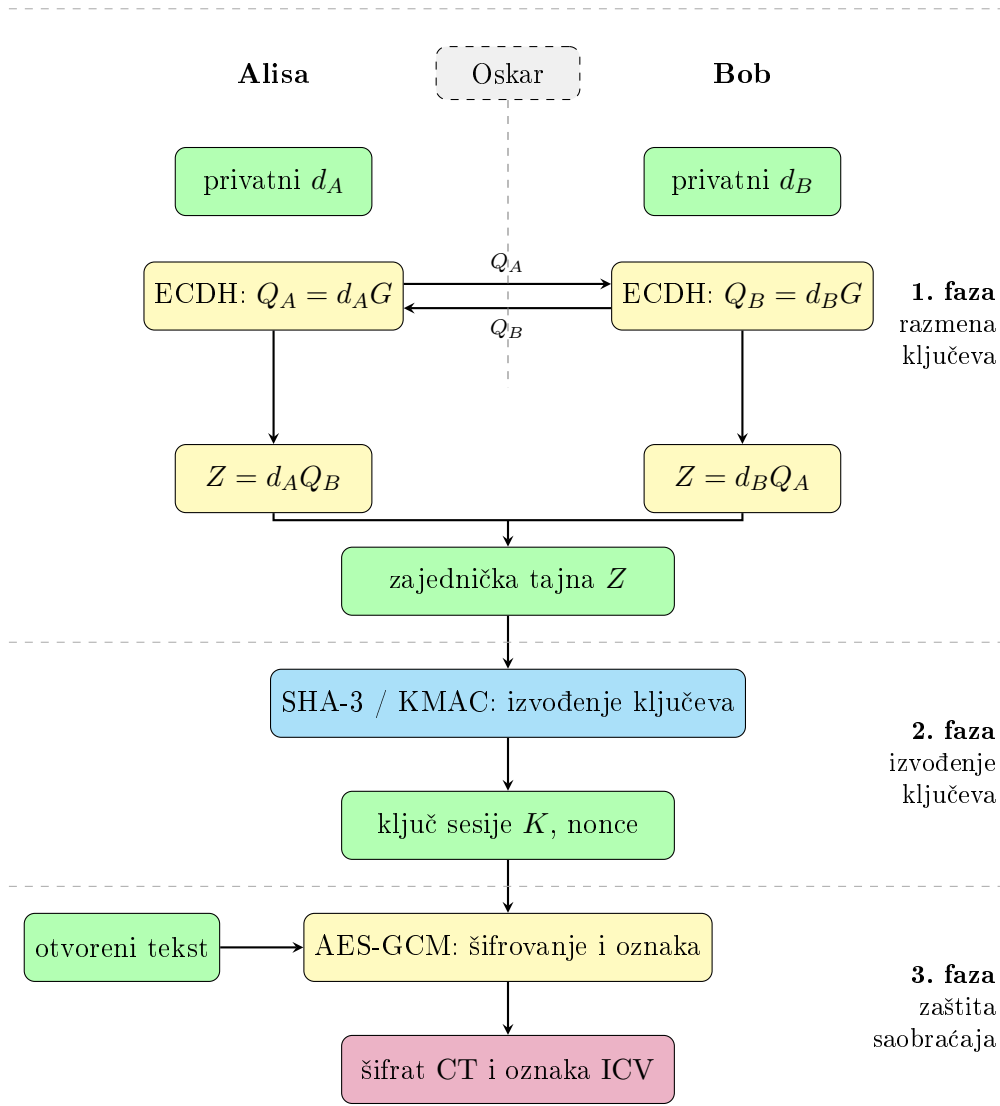
2.4 Hibridni kriptosistem

Nijedna od opisanih klasa algoritama sama po sebi ne rešava sve sigurnosne zahteve iz odeljka 2.2. Simetrične šifre efikasno štite podatke, ali zahtevaju prethodno dogovoren ključ. Asimetrični algoritmi omogućavaju dogovor ključa preko javnog kanala, ali su prespori za zaštitu samog saobraćaja. Heš funkcije obezbeđuju integritet i izvođenje ključeva, ali same ne šifruju. Savremeni protokoli zato kombinuju sve tri klase u hibridni kriptosistem, u kome svaki algoritam obavlja zadatak za koji je najpogodniji.

Akcelerator realizovan u ovom radu prati upravo ovu arhitekturu. Tok uspostavljanja i korišćenja bezbednog kanala odvija se u tri faze:

1. *Razmena ključeva (ECDH)*. Alisa i Bob razmenjuju javne ključeve i, svako korišćenjem svog privatnog ključa, izračunavaju istu zajedničku tajnu. Oskar, koji prisluškuje kanal, vidi oba javna ključa, ali iz njih ne može izračunati zajedničku tajnu.
2. *Izvođenje ključeva (SHA-3)*. Zajednička tajna dobijena ECDH razmenom nije uniformno raspodeljena i ne koristi se neposredno kao ključ. Umesto toga, propušta se kroz funkciju za izvođenje ključeva, u ovom radu KMAC konstrukciju zasnovanu na SHA-3 (odeljak 3.2.7), kojom se izvode ključ sesije i inicijalne vrednosti potrebne AEAD režimu.
3. *Zaštita saobraćaja (AES-GCM)*. Izvedenim ključem sesije korisnički saobraćaj se šifruje u GCM režimu, koji svakom paketu pridružuje autentifikacionu oznaku, čime su poverljivost i integritet obezbeđeni u jednom prolazu.

Opisani tok prikazan je na slici 2.1.



Slika 2.1: Tok hibridnog kriptosistema realizovanog u ovom radu. Boje označavaju vrstu bloka: zelena su podaci, žuta kriptografske operacije, cijan izvođenje ključeva, a ljubičasta zaštićeni izlaz.

Sistem tako obezbeđuje poverljivost saobraćaja, njegov integritet i autentičnost porekla kroz oznaku koju računa GCM. Nonce vrednost pri tome čuva jedinstvenost pseudoslučajnog niza, dok zaštitu od ponovljenih poruka, po tabeli 2.1, mora da obezbedi protokolski sloj iznad, tipično brojačem sekvence i prozorom prihvatanja. Preostaje zahtev autentičnosti učesnika. Bez potpisivanja javnih ključeva prva faza je ranjiva na napad čoveka u sredini, budući da Alisa nema način da utvrdi da javni ključ koji je primila zaista pripada Bobu, a ne Oskar. U praksi se taj problem rešava digitalnim potpisom, tipično algoritmom ECDSA nad istom klasom eliptičkih krivih, u sprezi sa infrastrukturom sertifikata.

Vredi naglasiti da dodavanje ECDSA nije puko proširenje već realizovanog hardvera. Sa ECDH protokolom ono deli samo skalarno množenje na krivoj, a pun obim nadogradnje, od aritmetike po modulu reda grupe do sabiranja dve nezavisne tačke pri proveru potpisa, razmatra odeljak 6.1.

Preostaje da se vidi na kom nivou ceo lanac stoji. Sistem je jak koliko i njegova najslabija karika, pa parametri izabrani u poglavlju 3 nisu birani nezavisno, nego tako da svi budu na istom nivou. Pregled je dat u tabeli 2.3.

Tabela 2.3: Sigurnosni nivo pojedinih karika sistema. Nivo ECDH jezgra je polovina bitske dužine reda tačke (odjeljak 3.1.6)

Uloga u sistemu	Algoritam i parametar	Sigurnosni nivo
razmena ključeva	ECDH nad krivom B-571	≈ 285 bitova
izvođenje ključeva	KMAC256	256 bitova
šifrovanje	AES-256	256 bitova
autentifikacija poruke	oznaka GCM od 128 bitova	128 bitova po pokušaju

Poslednji red odskake i traži objašnjenje, jer dužina oznake ne meri isto što i dužina ključa. Ključ štiti od čitanja saobraćaja, a oznaka od podmetanja izmenjene poruke. Napad na ključ izvodi se *offline*, van veze, pa napadač radi svojim tempom i na sopstvenoj opremi. Falsifikovanje oznake mora se izvoditi *online*, u vezi sa primaocem, jer se oznaka bez ključa ne može proveriti, pa svaki pokušaj košta jedan poslat i odbijen paket. Oznaka od 128 bitova je zato standardan izbor i ne spušta nivo sistema.

Poglavlje 3

Algoritmi

U ovom poglavlju teorijski su obrađena tri algoritma koja čine jezgro realizovanog akceleratora, redosledom kojim učestvuju u hibridnom protokolu iz odeljka 2.4: ECDH protokol za razmenu ključeva, SHA-3 heš funkcija i AES algoritam u GCM režimu rada. Za svaki algoritam dat je matematički okvir, opis same konstrukcije i svojstva bitna za hardversku realizaciju, koja je predmet poglavlja 4. Matematičke osnove konačnih polja, zajedničke za sva tri algoritma, sistematizovane su u dodatku A.

3.1 ECDH – razmena ključeva na eliptičkim krivama

3.1.1 Diffie–Hellmanov protokol

Diffie–Hellmanov protokol [13] omogućava dvema stranama da preko javnog kanala uspostave zajedničku tajnu. Ideja protokola je nezavisna od konkretne matematičke strukture. Potrebna je grupa u kojoj je eksponencijacija (uzastopna primena grupovne operacije) laka, a njena inverzija, problem diskretnog logaritma, praktično nerešiva. U izvornom obliku protokol koristi multiplikativnu grupu ostataka po modulu velikog prostog broja. Vremenom su, međutim, za ovu grupu razvijeni subekspencijalni algoritmi za diskretni logaritam, što zahteva sve veće dužine ključeva. Eliptičke krive nude grupu za koju takvi algoritmi nisu poznati, pa se ekvivalentna sigurnost postiže znatno kraćim ključevima.

3.1.2 Eliptičke krive

Eliptička kriva nad poljem K je skup rešenja jednačine trećeg stepena u dve promenljive, proširen posebnom tačkom u beskonačnosti $\mathcal{O}$. Nad poljima karakteristike različite od 2 i 3 jednačina se svodi na kratki Vajersštrasov (engl. *Weierstrass*) oblik:

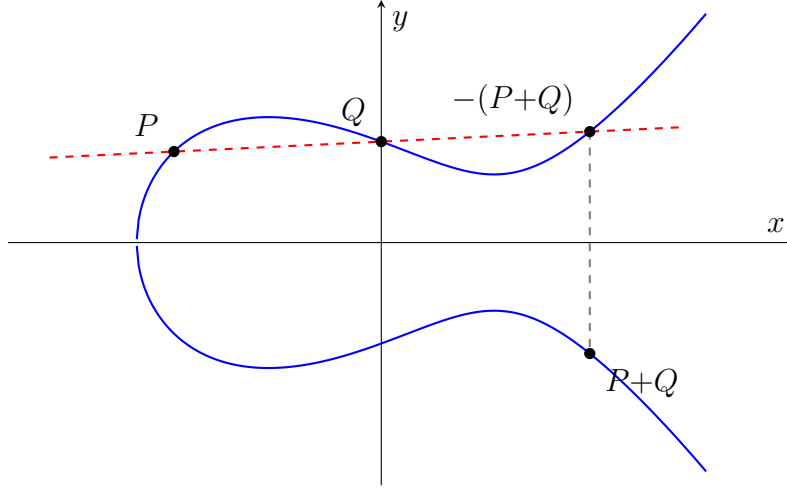
$$y^2 = x^3 + ax + b. \quad (3.1)$$

Nad binarnim poljima $\text{GF}(2^m)$, koja su korišćena u ovom radu, nesupersingularna eliptička kriva ima oblik:

$$E : y^2 + xy = x^3 + ax^2 + b, \quad a, b \in \text{GF}(2^m), b \neq 0. \quad (3.2)$$

Ključno svojstvo eliptičkih krivih je da se nad skupom njihovih tačaka može definisati operacija sabiranja koja taj skup čini Abelovom grupom, sa tačkom $\mathcal{O}$ kao

neutralnim elementom. Geometrijski, zbir tačaka P i Q dobija se tako što se kroz njih povuče prava. Ona seče krivu u još tačno jednoj tački, čijom se refleksijom u odnosu na x -osu dobija $P + Q$, kako je prikazano na slici 3.1. Prikaz je dat nad realnim brojevima, gde je geometrija očigledna. Nad konačnim poljem geometrijska intuicija se gubi, ali važe iste algebarske formule.



Slika 3.1: Geometrijsko sabiranje tačaka na eliptičkoj krivoj (prikaz nad realnim brojevima)

Algebarski, za krivu oblika (3.2) i tačke $P = (x_1, y_1)$ i $Q = (x_2, y_2)$, $P \neq \pm Q$, zbir $P + Q = (x_3, y_3)$ računa se kao:

$$\lambda = \frac{y_1 + y_2}{x_1 + x_2}, \quad x_3 = \lambda^2 + \lambda + x_1 + x_2 + a, \quad y_3 = \lambda(x_1 + x_3) + x_3 + y_1, \quad (3.3)$$

dok se dupliranje tačke $2P = (x_3, y_3)$ računa kao:

$$\lambda = x_1 + \frac{y_1}{x_1}, \quad x_3 = \lambda^2 + \lambda + a, \quad y_3 = x_1^2 + (\lambda + 1)x_3. \quad (3.4)$$

Sve operacije u izrazima (3.3) i (3.4) su operacije u polju $\text{GF}(2^m)$. Sabiranje i oduzimanje se u polju karakteristike 2 poklapaju. Formule pokazuju da se aritmetika tačaka svodi na četiri osnovne operacije u polju: sabiranje, množenje, kvadriranje i inverziju, što neposredno određuje strukturu hardverske aritmetičke jedinice [17].

3.1.3 Binarno polje $\text{GF}(2^m)$

Elementi polja $\text{GF}(2^m)$ mogu se predstaviti kao polinomi stepena manjeg od m sa koeficijentima u $\text{GF}(2) = \{0, 1\}$, odnosno kao binarne reči dužine m bitova u polinomijalnoj bazi. Aritmetika se obavlja po modulu nesvodljivog polinoma $f(x)$ stepena m :

- Sabiranje je sabiranje koeficijenata po modulu 2, dakle bitska operacija isključivo-ili (XOR) u jednom logičkom nivou.
- Množenje je množenje polinoma praćeno redukcijom po modulu $f(x)$. Ovo je dominantna operacija i po broju izvršavanja i po hardverskoj ceni, pa izbor arhitekture množača presudno određuje odnos površine i brzine celog sistema. Taj izbor je pitanje realizacije.

- Kvadriranje je znatno jeftinije od opšteg množenja, jer se kvadriranjem polinoma koeficijenti samo “razmaknu” ($x^i \mapsto x^{2i}$), pa se operacija svodi na umetanje nula i fiksnu redukciju, čistu kombinacionu mrežu.
- Inverzija je najskuplja operacija. Računa se proširenim Euklidovim algoritmom ili lancem kvadriranja i množenja, jer se svodi na stepenovanje unapred poznatim eksponentom (odeljak 3.1.5). U praksi se algoritmi biraju tako da broj inverzija bude sveden na minimum, o čemu je reč u odeljku 3.1.7.

Prednost binarnih polja za hardversku realizaciju je u tome što je aritmetika bez prenosa (engl. *carry-free*), pa nema lanaca prenosa koji ograničavaju radnu učestanost, a sve operacije se prirodno preslikavaju na XOR i AND mreže.

3.1.4 Kriva B-571

U radu je korišćena kriva B-571, čiji su parametri dati NIST preporukom SP 800-186 [18]. Kriva je oblika (3.2) nad poljem $\text{GF}(2^{571})$ sa redukcionim polinomom:

$$f(x) = x^{571} + x^{10} + x^5 + x^2 + 1, \quad (3.5)$$

koeficijentom $a = 1$ i pseudoslučajno generisanim koeficijentom b čija je vrednost data u standardu. Red osnovne tačke G je prost broj n dužine 570 bitova, a kofaktor krive je $h = 2$. B-571 je najveća standardizovana binarna kriva i pruža sigurnosni nivo od približno 285 bitova. Kompletne vrednosti parametara krive (b , koordinate tačke G , red n) date su u dodatku C.

Uz izbor ove krive ide i jedna napomena. Ista preporuka [18] sve krive nad binarnim poljima, pa i B-571, označava kao prevaziđene (engl. *deprecated*) za nove primene i za njih preporučuje krive nad prostim poljima. Preporuka pritom ne navodi nijednu poznatu bezbednosnu slabost ovih krivih. Razlog povlačenja je isključivo slaba rasprostranjenost u praksi, a najbolji poznati napadi na ECDLP nad njima ostaju generički, kao i kod krivih nad prostim poljima. Kriva je u ovom radu ipak izabrana iz hardverskih razloga, obrazloženih u uvodu odeljka 4.2.

3.1.5 Inverzija: Itoh–Tsujiijeva metoda

Od četiri operacije iz odeljka 3.1.3 inverzija je jedina koja se ne računa neposredno, ali se algoritamski svodi na preostale. Multiplikativna grupa polja $\text{GF}(2^m)$ ima $2^m - 1$ elemenata, pa za svako $u \neq 0$ važi $u^{2^m-1} = 1$, odnosno $u^{-1} = u^{2^m-2}$, dakle inverzija je stepenovanje unapred poznatim eksponentom.

Eksponent $2^m - 2$ u binarnom zapisu čini niz od $m - 1$ jedinice praćen nulom. Opšti postupak kvadriraj-i-množi troši po jedno množenje na svaku jedinicu eksponenta sem vodeće, pa bi ovde tražio $m - 2$ množenja, za B-571 čak 569. Upravo pravilnost eksponenta nudi izlaz. Niz jedinica ne mora da se gradi jedinicu po jedinicu, jer se dva već izgrađena niza spajaju jednim množenjem. Taj put sledi Itoh–Tsujiijeva metoda [19], koja traženi eksponent gradi kao adicioni lanac. Metoda je prvobitno data za normalne baze. Na polinomijalnu bazu, koja se koristi u ovom radu, preneli su je Guajardo i Paar [20]. Uvodi se pomoćna veličina

$$\beta_t = u^{2^t-1}, \quad (3.6)$$

čiji eksponent u binarnom zapisu čini t uzastopnih jedinica, i za nju važe dve veze:

$$\beta_{2t} = (\beta_t)^{2^t} \cdot \beta_t, \quad \beta_{t+1} = (\beta_t)^2 \cdot u. \quad (3.7)$$

Prva od niza dužine t pravi niz dužine $2t$ po ceni jednog množenja i t kvadriranja. Druga dodaje jednu jedinicu po ceni jednog množenja i jednog kvadriranja. Dužina niza se time gradi kao broj, udvostručenjem i uvećanjem za jedan, čitajući binarni zapis broja $m - 1$. Konačno je $u^{-1} = (\beta_{m-1})^2$, jer je $2^m - 2 = 2(2^{m-1} - 1)$.

Primer 3.1. Tražimo inverz elementa $u = 00000101$ u polju $\text{GF}(2^8)$, uz redukcionu polinom $x^8 + x^4 + x^3 + x + 1$ koji koristi i AES (dodatak A.3). Inverz je $u^{-1} = u^{2^8-2} = u^{254}$, a $254 = 1111110_2$, dakle niz od sedam jedinica praćen nulom. Kako je $m - 1 = 7 = 111_2$, dužina niza se gradi kao $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7$:

Korak	Račun	Rezultat	EkspONENT binarno	Vrednost
početak	$\beta_1 = u$	u^1	1	05
udvostručenje	$\beta_2 = (\beta_1)^2 \beta_1$	u^3	11	55
uvećanje	$\beta_3 = (\beta_2)^2 u$	u^7	111	13
udvostručenje	$\beta_6 = (\beta_3)^{2^3} \beta_3$	u^{63}	111111	60
uvećanje	$\beta_7 = (\beta_6)^2 u$	u^{127}	1111111	F6
kvadriranje	$u^{-1} = (\beta_7)^2$	u^{254}	11111110	52

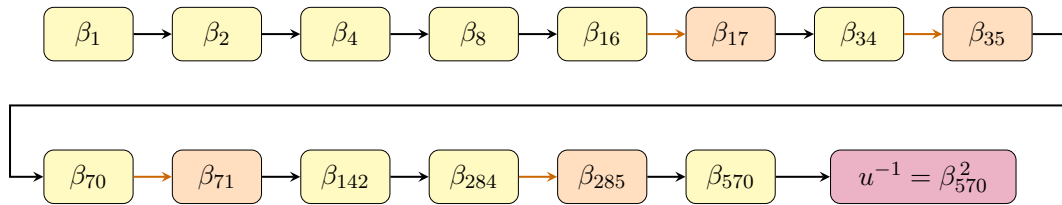
Kvadriranje pomera eksponent za jedno mesto ulevo, a množenje sabira eksponente. Udvostručenje niz od t jedinica pomeri za t mesta i nalepi na samog sebe, a uvećanje ga pomeri za jedno mesto i dopiše jedinicu iz u . Ukupno su četiri množenja, dva za udvostručenja i dva za uvećanja, umesto šest koliko bi tražio opšti postupak, uz $m - 1 = 7$ kvadriranja. Dobijeno je $u^{-1} = 52$, što potvrđuje množenje: $05 \cdot 52 = 01$.

Cena je $m - 1$ kvadriranja, koja su jeftina, i svega

$$\lfloor \log_2(m - 1) \rfloor + \text{HW}(m - 1) - 1 \quad (3.8)$$

množenja, gde je HW broj jedinica u binarnom zapisu. Za B-571 je $m - 1 = 570 = 1000111010_2$, pa je $\lfloor \log_2 570 \rfloor = 9$ i $\text{HW}(570) = 5$, što daje $9 + 5 - 1 = 13$ množenja. Umesto 569, dakle 13.

Lanac za B-571 prikazan je na slici 3.2. Binarni zapis 1000111010_2 čita se sleva nadesno, počev od β_1 , pa svaki bit udvostručuje dosegnuto t , a svaka jedinica ga još uveća za jedan. Devet udvostručenja i četiri uvećanja daju upravo 13 množenja.



Slika 3.2: Itoh–Tsujiijev adicioni lanac za $m - 1 = 570$: crna strelica je udvostručenje, narandžasta uvećanje za jedan

Ova razlika je razlog zbog kog projekтивna reprezentacija iz odeljka 3.1.7, koja u petlji ostavlja jednu jedinu inverziju, na samom kraju, uopšte ima smisla. Da je ta jedna koštala 569 množenja, sama bi nosila šestinu cene celih merdevina. Ovako košta, po broju množenja, koliko i dve iteracije petlje, pa završna konverzija ostaje zanemarljiva.

3.1.6 Skalarno množenje i problem diskretnog logaritma

Osnovna operacija svih protokola nad eliptičkim krivama je skalarno množenje. Za tačku P i ceo broj k računa se

$$kP = \underbrace{P + P + \cdots + P}_{k \text{ puta}}. \quad (3.9)$$

Naivno sabiranje je neupotrebljivo za brojeve k dužine stotina bitova. Umesto toga koristi se metoda dupliraj-i-dodaj (engl. *double-and-add*), analogna binarnoj eksponencijaciji, kojom se kP računa kroz $O(m)$ dupliranja i sabiranja tačaka.

Sigurnost počiva na problemu diskretnog logaritma na eliptičkoj krivoj (engl. *Elliptic Curve Discrete Logarithm Problem*, ECDLP). Za date tačke P i $Q = kP$ praktično je nemoguće odrediti k . Najbolji poznati algoritmi za ECDLP na pravilno odabranim krivama su generički, sa složnošću $O(\sqrt{n})$, pa kriva sa redom od 570 bitova pruža pomenutih ≈ 285 bitova sigurnosti.

3.1.7 Montgomeryjeve merdevine

Osnovna dupliraj-i-dodaj metoda ima svojstvo nepoželjno u praksi. Redosled operacija zavisi od bitova skalar k , a skalar je privatni ključ. Napadač koji posmatra vreme izvršavanja ili potrošnju uređaja može iz tih razlika rekonstruisati bitove ključa. Takvi napadi poznati su kao napadi sporednim kanalima (engl. *side-channel attacks*). *Montgomeryjeve merdevine* (engl. *Montgomery ladder*) [21, 22] tu izloženost bitno smanjuju. Algoritam održava par tačaka koji je u svakom trenutku oblika $(P_1, P_2) = (\kappa P, (\kappa+1)P)$, gde je κ broj sastavljen od do tada pročitanih bitova skalar k . Korak za naredni bit k_i je $\kappa \leftarrow 2\kappa + k_i$. Za $k_i = 0$ novi par je $(2\kappa P, (2\kappa+1)P) = (2P_1, P_1+P_2)$, a za $k_i = 1$ novi par je $((2\kappa+1)P, (2\kappa+2)P) = (P_1+P_2, 2P_2)$. U oba slučaja se, dakle, izvršava tačno jedno sabiranje i tačno jedno dupliranje, bit bira samo raspored rezultata, a razlika $P_2 - P_1 = P$ ostaje nepromenjena:

1. $P_1 \leftarrow P; \quad P_2 \leftarrow 2P$
2. za svaki bit k_i , od $i = t-2$ do $i = 0$ (gde je $k = (k_{t-1}, \dots, k_0)_2$, $k_{t-1} = 1$):
 - ako je $k_i = 1$: $P_1 \leftarrow P_1 + P_2; \quad P_2 \leftarrow 2P_2$
 - ako je $k_i = 0$: $P_2 \leftarrow P_1 + P_2; \quad P_1 \leftarrow 2P_1$
3. rezultat je $P_1 = kP$.

U realizaciji iz poglavlja 4 i korak 1 se izvršava istim mehanizmom iteracije, nad parom (P, P) , kao prva od iteracija, pa ih ima onoliko koliko skalar ima bitova, za najduži standardni ključ 570.

Doprinos Lópeza i Dahaba [22] je dvostruk. Prvo, pošto je razlika para stalno poznata i jednaka P , i sabiranje i dupliranje mogu se izraziti bez y -koordinata. Drugo, deljenje se odlaže prelaskom u projektivni zapis $x = X/Z$. Imenilac putuje kao zaseban registar, pa u petlji nema nijedne inverzije, a jedina se izvršava pri povratku u afini oblik na kraju. Tačka u merdevinama je time samo par (X, Z) .

Uz oznaku x za afinu x -koordinatu ulazne tačke P i koeficijent b krive (3.2), dva koraka iteracije, udvostručenje (engl. *mdouble*) i diferencijalno sabiranje (engl.

madd), glase:

$$\text{udvostručenje } (X, Z) \mapsto X' = X^4 + b Z^4, \quad Z' = X^2 Z^2, \quad (3.10)$$

$$\begin{aligned} \text{sabiranje } (X_1, Z_1), (X_2, Z_2) \mapsto Z_3 &= (X_1 Z_2 + X_2 Z_1)^2, \\ X_3 &= x Z_3 + (X_1 Z_2)(X_2 Z_1). \end{aligned} \quad (3.11)$$

Izraz (3.11) je diferencijalno sabiranje, jer ne važi za proizvoljne dve tačke, nego samo kada je njihova razlika baš P , što merdevine i garantuju. Upravo ta ograničenost je ono što dozvoljava da y -koordinata potpuno ispadne iz računa.

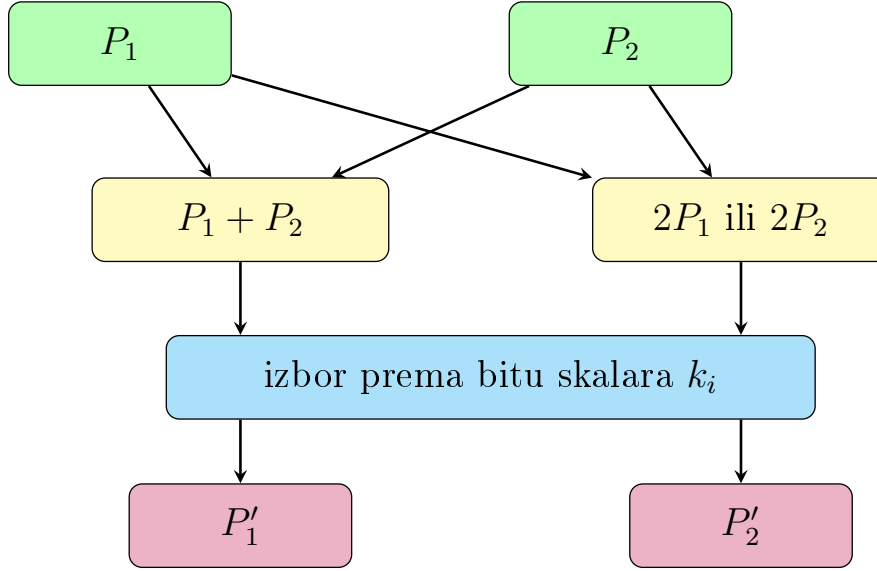
Prebrojavanje operacija u izrazima (3.10) i (3.11) daje cenu jedne iteracije, prikazanu u tabeli 3.1. Pošto se po bitu skalara izvršava tačno jedno udvostručenje i tačno jedno sabiranje, ta cena je ista za svaki bit, što je istovremeno i osnova otpornosti na napade sporednim kanalima i osnova za predviđanje trajanja.

Tabela 3.1: Cena skalarnog množenja na krivoj B-571 po Lópezovim i Dahabovim formulama

Korak	Množenja	Kvadriranja	Sabiranja
udvostručenje (3.10)	2	4	1
diferencijalno sabiranje (3.11)	4	1	2
jedna iteracija	6	5	3
570 iteracija	3420	2850	1710
završna inverzija (Itoh–Tsujii)	13	570	0
završna konverzija (bez inverzije)	11	1	5
ukupno	3444	3421	1715

Tabela pokazuje odakle dolazi cena celog jezgra, sa po jednom iteracijom za svaki bit skalara, kojih je za najduži standardni ključ 570. Skalarno množenje ukupno traži 3444 množenja u polju, i to je jedina operacija u petlji merdevina koja traje više od jednog takta, pa brzina množača nosi glavninu trajanja dok je D mali. Množać koji obrađuje jedan bit operanda po taktu daje 571 takt po množenju i time blizu dva miliona taktova po skalarnom množenju, a množać koji obrađuje 64 bita daje $\lceil 571/64 \rceil = 9$ taktova i oko trideset hiljada taktova samih množenja, uz koje tada očekivano prevagne režija upravljanja. Ove vrednosti su izvedene iz broja operacija i ne obuhvataju taktove upravljačke logike. Izbor širine obrade i arhitektura množača tema su poglavlja 4, a izmerene vrednosti date su u poglavlju 5. Broj iteracija prati bitsku dužinu skalara, pa ukupno trajanje tu dužinu i otkriva. Merenja u ovom radu zato rade sa skalarom pune standardne dužine od 570 bitova, čime je izmereno trajanje ujedno i najduže za važeći ključ.

Struktura jedne iteracije prikazana je na slici 3.3. Sabiranje i dupliranje su nezavisne operacije nad istim parom tačaka, pa se mogu izvršavati paralelno, a bit skalara utiče samo na to koji se od dva rezultata upisuje u koji registar. Bitno je da se taj izbor ne sme realizovati grananjem, nego uslovnom zamenom registara koja se izvršava uvek, sa istim trajanjem bez obzira na vrednost bita. Inače bi se otpornost izgubljena na nivou formula vratila kroz upravljačku logiku.



Slika 3.3: Jedna iteracija Montgomeryjevih merdevina

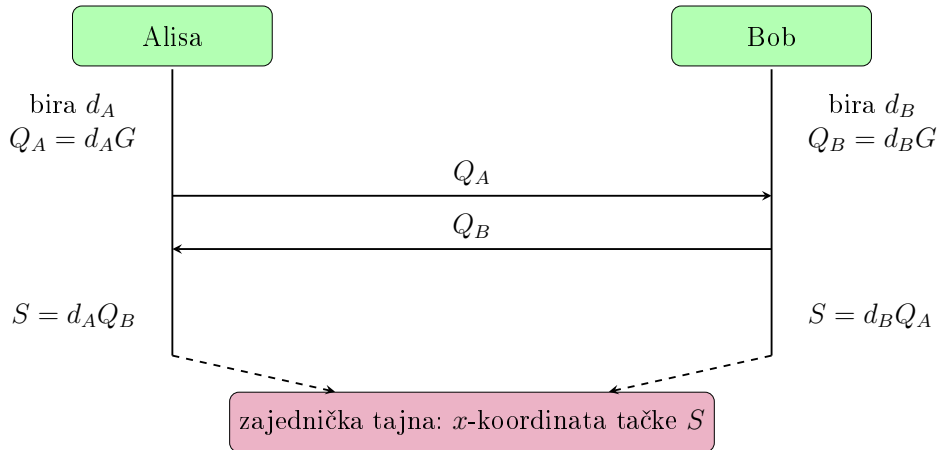
3.1.8 ECDH protokol

Neka su parametri krive (polje, koeficijenti, osnovna tačka G reda n) javno dogovoreni. ECDH protokol [23] odvija se na sledeći način:

1. Alisa bira slučajan privatni ključ $d_A \in [1, n-1]$ i izračunava javni ključ $Q_A = d_A G$. Bob analogno bira d_B i izračunava $Q_B = d_B G$.
2. Alisa i Bob razmenjuju javne ključeve Q_A i Q_B preko javnog kanala.
3. Alisa izračunava $S = d_A Q_B$, a Bob $S = d_B Q_A$. Obe strane dobijaju istu tačku, jer je

$$d_A Q_B = d_A (d_B G) = d_B (d_A G) = d_B Q_A. \quad (3.12)$$

Tok protokola prikazan je na slici 3.4.



Slika 3.4: Tok ECDH protokola

Kao zajednička tajna koristi se x -koordinata tačke S , što je u saglasnosti sa realizacijom skalarnog množenja preko Montgomeryjevih merdevina, kojoj y -koordinata nije ni potrebna. Kada je rezultatu ipak potrebna, y -koordinata se na kraju rekonstruiše izrazom u zatvorenom obliku iz projektivnog stanja merdevina i obe koordinate polazne tačke. Oskar koji posmatra kanal vidi G , Q_A i Q_B , pa se izračunavanje

tajne svodi na ECDLP iz odeljka 3.1.6. Dobijena tajna se, kako je opisano u odeljku 2.4, ne koristi neposredno kao ključ, već se propušta kroz funkciju za izvođenje ključeva zasnovanu na heš funkciji.

Protokol nije potpun bez provere primljenog javnog ključa. Preporuka SP 800-56A [23] zahteva da svaka strana pre skalarnog množenja utvrdi da primljena tačka nije tačka u beskonačnosti, da su joj koordinate ispravno kodirani elementi polja i da zaista zadovoljava jednačinu krive (3.2). Neuspešna provera prekida protokol. Bez toga Oskar može poslati tačku koja leži na nekoj drugoj, slabijoj krivoj i iz odgovora izvlačiti informacije o privatnom ključu, u napadu poznatom kao napad nevažećom krivom (engl. *invalid-curve attack*).

Kod krive B-571 postoji i dodatni razlog za pažnju. Kofaktor krive je $h = 2$, pa grupa tačaka nije prostog reda i sadrži podgrupu malog reda. Poslata tačka može ležati u njoj i time otkriti poslednji bit privatnog ključa. Standardna protivmera je množenje kofaktorom, čime se svaka komponenta malog reda poništava pre upotrebe tajne. Realizacija koja radi samo sa x -koordinatama ovaj problem sama po sebi ne uklanja, jer tačka reda dva ima $x = 0$, pa njeno slanje otkriva parnost privatnog ključa isto kao i sa punim koordinatama. Curenje na jedan bit ograničava sam kofaktor $h = 2$, a uklanja ga validacija, koja uz proveru pripadnosti krivoj odbacuje i tačku sa $x = 0$. U sistemu iz ovog rada te provere pripadaju strani koja jezgro poziva i obavljaju se pre nego što tačka stigne do hardverskog jezgra, o čemu je više reči u odeljku 6.3.

3.2 SHA-3

3.2.1 Nastanak standarda

Nakon što su početkom 2000-ih godina objavljeni praktični napadi kolizijom na MD5 i napadi na SHA-1 znatno ispod rodendanske granice, javila se bojazan da bi slične tehnike mogle ugroziti i tada važeću familiju SHA-2. NIST je 2007. godine raspisao javno takmičenje za novu heš funkciju, po ugledu na proces kojim je izabran AES. Od 64 prijavljena kandidata, 2012. godine pobedio je algoritam Keccak autora Bertonia, Daemena, Peetersa i Van Asschea, standardizovan 2015. godine kao SHA-3 [16, 24]. Presudni razlozi za izbor bili su konstrukcija suštinski različita od SHA-2, čime kompromitacija jedne familije ne ugrožava drugu, i izuzetna pogodnost za hardversku realizaciju. Prva tvrdnja razmotrena je u odeljku 3.2.4, pošto konstrukcija bude uvedena, a pogodnost za hardversku realizaciju u odeljku 3.2.5.

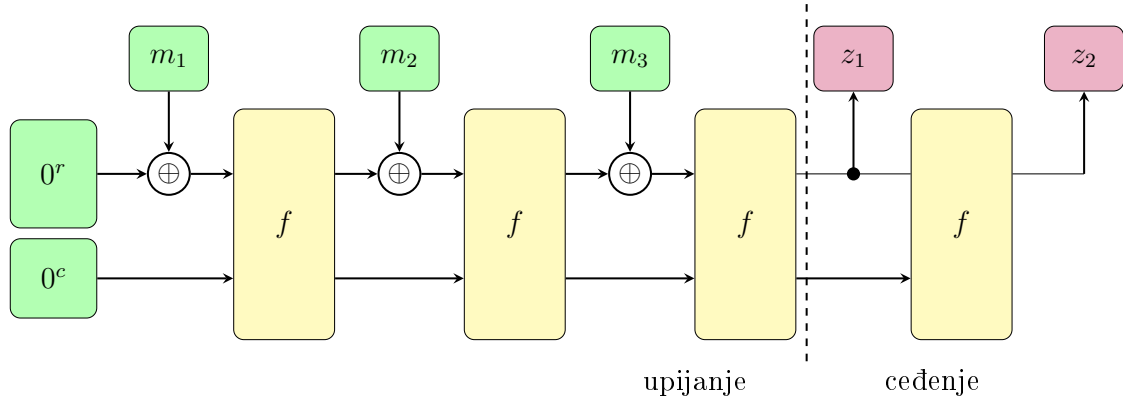
3.2.2 Sponge konstrukcija

SHA-3 je zasnovan na sponge konstrukciji (engl. *sponge*, u prevodu *sunder*), koja od fiksne permutacije f nad stanjem od b bitova gradi funkciju sa ulazom i izlazom proizvoljne dužine. Stanje se deli na dva dela: brzinu (engl. *rate*) od r bitova, kroz koju se razmenjuju podaci, i kapacitet (engl. *capacity*) od $c = b - r$ bitova, koji nikada ne dolazi u neposredan dodir sa ulazom niti izlazom i predstavlja unutrašnju rezervu sigurnosti. Konstrukcija radi u dve faze:

- *Upijanje* (engl. *absorbing*) – poruka se dopuni do umnoška r bitova i podeli na blokove. Svaki blok se sabere sa prvih r bitova stanja operacijom XOR ($\oplus$), nakon čega se na celo stanje primeni permutacija f .

- *Cedenje* (engl. *squeezing*) – izlaz se formira čitanjem prvih r bitova stanja. Ako je potrebno više bitova, stanje se ponovo permutuje i čitanje ponavlja.

Konstrukcija je prikazana na slici 3.5. Blokovi poruke m_i upijaju se u prvih r bitova stanja, permutacija f nakon svakog bloka meša celo stanje, a izlazni blokovi z_i se ce-
de iz istih r bitova. Preostalih c bitova nikada ne napušta unutrašnjost konstrukcije. Vredi uočiti da obe faze koriste isti deo stanja, onih r bitova brzine, i istu permutaciju. Razlikuju se samo po tome da li se kroz taj deo upisuje ili čita. Za hardver to znači da upijanje i cedenje dele isti put podataka, pa se jedno jezgro permutacije koristi u obe faze.



Slika 3.5: Sponge konstrukcija

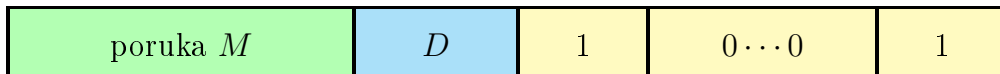
3.2.3 Dopuna poruke i razdvajanje domena

Upijanje obrađuje poruku u blokovima od r bitova, pa poruka proizvoljne dužine mora najpre biti dopunjena do umnoška te dužine. Standard za to koristi šemu $\text{pad}10^*1$, u kojoj se dodaje bit 1, zatim najmanji potreban broj nula i na kraju još jedan bit 1. Oba granična bita su neophodna, ali svaki iz svog razloga. Prva jedinica čini dopunu jednoznačnom, jer bi se bez nje poruke koje se razlikuju samo po završnim nulama posle dopune slile u isti niz i kolizije bi se dobijale trivijalno, bez ikakvog napada na permutaciju. Završna jedinica omogućava da ista permutacija bezbedno radi sa različitim brzinama r , jer poslednji bit bloka kodira gde je granica brzine.

Sama dopuna, međutim, nije dovoljna. Sve funkcije familije dele istu permutaciju i istu konstrukciju, a SHA3-256 i SHAKE256 imaju čak i istu brzinu $r = 1088$. Nad istom porukom one bi zato davale isti izlaz, što je nedopustivo, jer bi sažetak jedne funkcije otkrivao izlaz druge. Standard to sprečava tako što na poruku, pre dopune, dodaje kratak sufiks za razdvajanje domena (engl. *domain separation*) D , različit za svaku porodicu funkcija. Pun oblik dopunjene poruke je stoga

$$M \parallel D \parallel \text{pad}10^*1, \quad (3.13)$$

kako je prikazano na slici 3.6.



Slika 3.6: Dopuna poruke do umnoška r bitova: sufiks za razdvajanje domena D (cijan), pa šema $\text{pad}10^*1$ (žuto). Vrednosti sufiksa date su u tabeli 3.2

Sufiks je jedina razlika u dopuni između funkcija familije, dok je sama šema $\text{pad}10^*1$ za sve ista.

Tabela 3.2: Sufiksi za razdvajanje domena u SHA-3 familiji [16, 25]

Funkcija	Sufiks D	Prvi bajt dopune	Napomena
SHA3-224/256/384/512	01	0x06	heš fiksne dužine
SHAKE128, SHAKE256	1111	0x1F	izlaz proizvoljne dužine
cSHAKE128, cSHAKE256	00	0x04	kada je zadat bar jedan od dodatnih parametara (odjeljak 3.2.7)
KMAC128, KMAC256	00	0x04	gradi se nad cSHAKE (odjeljak 3.2.7)

Na nivou bajtova dopuna se svodi na dve izmene. Sufiks i prva jedinica šeme spajaju se u jedan bajt, čija je vrednost data u tabeli, a poslednji bajt bloka dobija vrednost 0x80, što je završna jedinica šeme. Kada je do kraja bloka ostao tačno jedan bajt, ta dva se poklapaju, pa se njihove vrednosti sabiraju bitskom operacijom OR. Za SHA-3 je tada poslednji bajt $0x06 \vee 0x80 = 0x86$.

3.2.4 Sigurnosna svojstva sponge konstrukcije

Kapacitet je unutrašnja sigurnosna granica sponge konstrukcije. On je jedini deo stanja koji Oskar ne vidi ni na ulazu ni na izlazu, pa se napad na samu konstrukciju svodi na pogađanje tih c bitova. Rodendanska granica iz izraza (2.1) taj posao svodi na $2^{c/2}$ pokušaja.

Kapacitet, međutim, nije jedina granica. Sigurnost zavisi i od posmatranog svojstva i od dužine izlaza d , pa standard [16] navodi dve odvojene granice:

$$\text{kolizije: } \min\left(\frac{d}{2}, \frac{c}{2}\right) \text{ bitova,} \quad \text{original: } \min\left(d, \frac{c}{2}\right) \text{ bitova.} \quad (3.14)$$

Iz granica (3.14) sledi i izbor parametara u standardu. Kod funkcija sa fiksnim izlazom uzima se $c = 2d$, čime kapacitet nikada nije uže grlo. Za SHA3-256 kolizije su ograničene na 2^{128} , a original na 2^{256} , pa granicu postavlja dužina sažetka, ne konstrukcija. Kod funkcija proširivog izlaza (engl. *eXtendable Output Function*, XOF) je obrnuto, jer je d proizvoljno, pa kapacitet postaje gornja granica. SHAKE128 zato ne prelazi 128 bitova ma koliko se izlaza iscedilo. Vrednosti za sve varijante date su u tabeli 3.3.

Druga bitna posledica tiče se strukture. Funkcije MD5, SHA-1 i osnovne članice familije SHA-2, poput SHA-256 i SHA-512, pripadaju Merkle–Damgardovoj (engl. *Merkle–Damgård*) klasi konstrukcija, kod kojih je izlaz funkcije ujedno i celo unutrašnje stanje nakon poslednjeg bloka. Posledica je napad produžavanjem poruke (engl. *length-extension attack*). Znajući $H(M)$ i dužinu poruke M , Oskar može, ne znajući samu poruku, da nastavi izračunavanje i dobije ispravan $H(M \parallel \text{pad} \parallel M')$ za proizvoljan nastavak M' . Zbog toga se SHA-2 ne sme koristiti kao MAC u naivnom obliku $H(K \parallel M)$, već isključivo kroz HMAC konstrukciju, koja problem rešava po cenu dva ulančana poziva heš funkcije [1, 2].

Sunder ovu slabost nema po samoj strukturi. Izlaz čini samo r bitova stanja, dok c bitova kapaciteta nikada ne napušta konstrukciju, pa Oskar iz sažetka ne može rekonstruisati stanje niti nastaviti upijanje. Otuda i praktična prednost bitna za

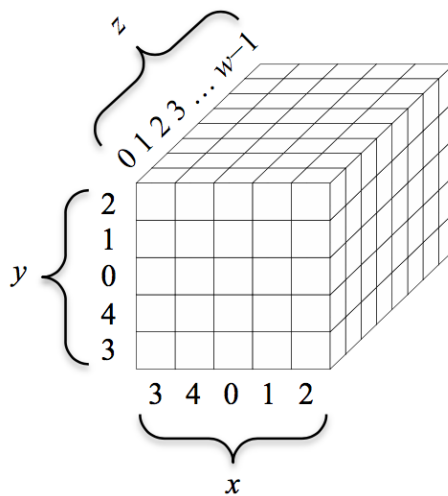
ovaj rad. SHA-3 omogućava konstrukcije sa ključem u jednom pozivu heš funkcije, poput KMAC funkcije iz odeljka 3.2.7, bez HMAC nadogradnje sa dva ulančana poziva, kod koje spoljašnji poziv mora da sačeka unutrašnji sažetak. Za hardver to znači jednostavniju obradu i kraću latenciju po poruci.

3.2.5 Keccak- $f[1600]$ permutacija

Za sve SHA-3 varijante koristi se permutacija Keccak- $f[1600]$ nad stanjem od $b = 1600$ bitova. Stanje nije niz bitova nego trodimenzionalna struktura $5 \times 5 \times 64$, prikazana na slici 3.7. Delovi te strukture imaju svoja imena, koja se koriste u opisu koraka:

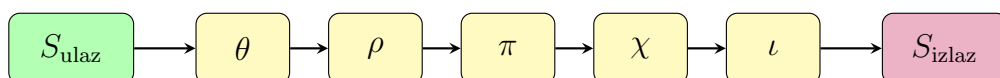
- *traka* (engl. *lane*) – 64 bita duž z -ose, za fiksne x i y , označava se $A_{x,y}$;
- *kolona* (engl. *column*) – 5 bitova duž y -ose, za fiksne x i z ;
- *vrsta* (engl. *row*) – 5 bitova duž x -ose, za fiksne y i z .
- *ravan* (engl. *sheet*) – svih pet traka sa istim x .

Ova organizacija je izuzetno pogodna i za hardver, gde se traka prirodno preslikava na 64-bitni registar, i za softver, gde odgovara jednoj mašinskoj reči. Svih pet koraka runde deluje po jednoj od navedenih osa, pa je vredi imati na umu tokom celog opisa.



Slika 3.7: Stanje permutacije Keccak- $f[1600]$. Preuzeto iz [24]

Permutacija se sastoji od 24 runde, a svaka runda primenjuje redom pet koraka, θ , ρ , π , χ i ι , kako je prikazano na slici 3.8. Svi indeksi u nastavku računaju se po modulu 5, a $\text{rot}(\cdot, k)$ označava cikličko pomeranje trake za k bitova. U nastavku je svaki korak opisan pojedinačno.



Slika 3.8: Jedna runda permutacije Keccak- $f[1600]$

3.2.5.1 Korak θ

Korak θ je jedini koji meša bitove duž y -ose i time obezbeđuje difuziju između kolona. Za svako x najpre se izračuna traka parnosti C_x , kao XOR svih pet traka sa tim x , pa se od parnosti dve susedne ravni formira korekcija D_x i dodaje svim trakama ravni x :

$$\begin{aligned} C_x &= \bigoplus_{y=0}^4 A_{x,y}, & D_x &= C_{x-1} \oplus \text{rot}(C_{x+1}, 1), \\ A_{x,y} &\leftarrow A_{x,y} \oplus D_x. \end{aligned} \quad (3.15)$$

Pošto je XOR bitska operacija, C_x nije jedan bit nego cela traka od 64 bita. Njen bit na dubini z jednak je parnosti onih pet bitova koji na toj dubini čine kolonu (x, z) . Dimenzija z se pri tome ne meša, računa se 64 nezavisne parnosti, po jedna za svaku dubinu. Jedina operacija u ovom koraku koja uopšte pomera bitove duž z -ose jeste rotacija za jedno mesto u izrazu za D_x . Sve ostalo se odvija bit po bit, na istoj dubini.

Ako se izraz (3.15) raspiše na nivou pojedinačnog bita, dobija se

$$\begin{aligned} A_{x,y}[z] &\leftarrow A_{x,y}[z] \\ &\oplus \underbrace{\bigoplus_{y'=0}^4 A_{x-1,y'}[z]}_{\text{parnost kolone } (x-1, z)} \\ &\oplus \underbrace{\bigoplus_{y'=0}^4 A_{x+1,y'}[z-1]}_{\text{parnost kolone } (x+1, z-1)}, \end{aligned} \quad (3.16)$$

gde $A_{x,y}[z]$ označava bit na dubini z trake $A_{x,y}$. Svaki izlazni bit zavisi, dakle, od jedanaest ulaznih, od sebe samog i od dve pune kolone. Isto važi i u suprotnom smeru, jer jedan ulazni bit ulazi u jedanaest izlaznih. Otuda brza difuzija, jer promena jednog jedinog bita stanja već posle nekoliko rundi zahvata celo stanje. Delovanje koraka po ravnima prikazano je na slici 3.9.

U hardveru θ nosi najdužu kombinacionu putanju od pet koraka, ali je i ona plitka, jer je parnost XOR sa pet ulaza, koji staje u jedan šestoulazni LUT, pa je ceo korak dubok svega tri nivoa logike, parnosti kolona, korekcije D_x i završnog XOR-a. Nema ni množenja ni tabela u memoriji.

	$x=0$	$x=1$	$x=2$	$x=3$	$x=4$
$y=0$	$A_{0,0}$	$A_{1,0}$	$A_{2,0}$	$A_{3,0}$	$A_{4,0}$
$y=1$	$A_{0,1}$	$A_{1,1}$	$A_{2,1}$	$A_{3,1}$	$A_{4,1}$
$y=2$	$A_{0,2}$	$A_{1,2}$	$A_{2,2}$	$A_{3,2}$	$A_{4,2}$
$y=3$	$A_{0,3}$	$A_{1,3}$	$A_{2,3}$	$A_{3,3}$	$A_{4,3}$
$y=4$	$A_{0,4}$	$A_{1,4}$	$A_{2,4}$	$A_{3,4}$	$A_{4,4}$

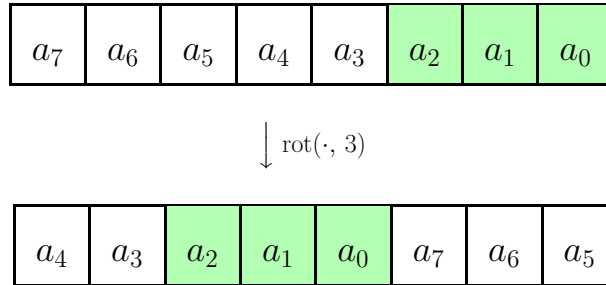
Slika 3.9: Korak θ za ravan $x = 1$ – svaka traka ravni (zeleno) sabira se sa trakom parnosti ravni $x = 0$ (cijan) i rotiranom trakom parnosti ravni $x = 2$ (žuto)

3.2.5.2 Korak ρ

Korak ρ ciklično rotira svaku traku za njen fiksni ofset:

$$A_{x,y} \leftarrow \text{rot}(A_{x,y}, r_{x,y}), \quad (3.17)$$

gde su $r_{x,y}$ konstante definisane standardom, date u dodatku D. Rotacija premešta bitove unutar trake (duž z -ose stanja), čime difuzija iz koraka θ , koja deluje po kolonama, dobija i treću dimenziju. Delovanje na jednu traku prikazano je na slici 3.10. U hardveru su ove rotacije fiksne i svode se na preusmeravanje veza, pa ne troše logiku.



Slika 3.10: Korak ρ : ciklična rotacija trake, prikazana na skraćenoj traci od 8 bitova za ofset 3. Tri istaknuta bita prelaze sa desnog kraja na poziciju tri mesta levlje. Bitovi koji izađu sa levog kraja ulaze sa desnog

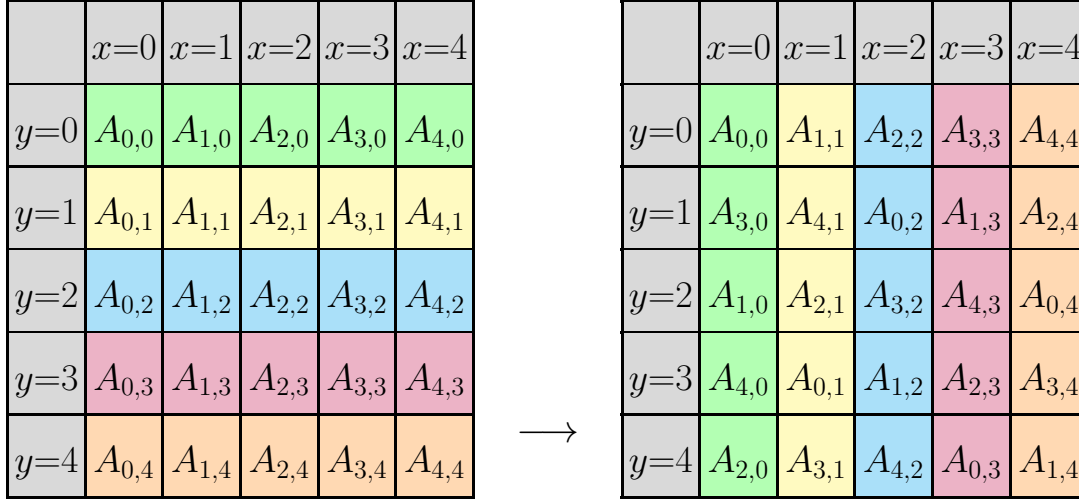
3.2.5.3 Korak π

Korak π premešta trake na nove pozicije u 5×5 matrici:

$$A'_{y, (2x+3y) \bmod 5} \leftarrow A_{x,y}. \quad (3.18)$$

Dok ρ meša bitove unutar traka, π meša same trake između vrsta i kolona, čime se sprečava da se difuzija zadrži u lokalnim šablonima. Preslikavanje svih 25 traka prikazano je na slici 3.11. Iz izraza za odredište vidi se pravilnost koju slika čini očiglednom. Prvi indeks odredišta je y , pa sve trake jedne vrste polaznog stanja završavaju u istoj koloni novog stanja. Korak π je, dakle, transpozicija praćena preslikavanjem $x \mapsto 2x + 3y$ unutar kolone. Traka $A_{0,0}$ jedina ostaje na svom mestu,

a duž prve vrste novog stanja poređala se dijagonala polaznog. I ovaj korak je u hardveru besplatan. Kao i ShiftRows transformacija AES algoritma, o kojoj će biti reči u odeljku 3.3.4, i ovde je reč o čistom preusmeravanju veza.



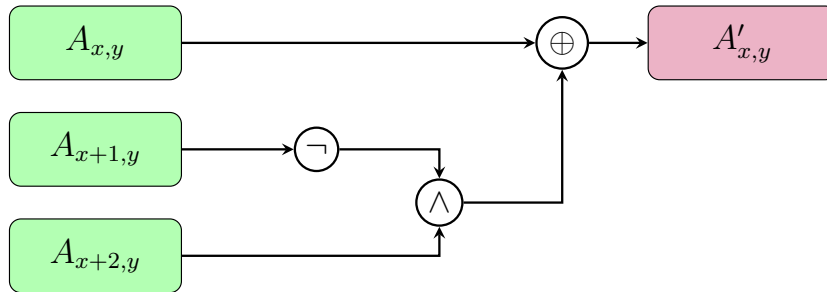
Slika 3.11: Korak π : preslikavanje svih 25 traka. Levo je polazno, desno novo stanje. U svakoj ćeliji novog stanja upisana je traka koja je na to mesto došla, a boja prati polaznu vrstu y

3.2.5.4 Korak χ

Korak χ je jedina nelinearna operacija permutacije, analogon S-boksa AES algoritma, o kome će biti reči u odeljku 3.3.4, ali izuzetno jeftin. Deluje po vrstama stanja, dakle za fiksno y , tako što se svaka traka sabira sa proizvodom komplementa prve i vrednosti druge naredne trake u istoj vrsti:

$$A_{x,y} \leftarrow A_{x,y} \oplus (\overline{A_{x+1,y}} \wedge A_{x+2,y}). \quad (3.19)$$

Operacija je bitska. Logička mreža sa slike 3.12 deluje na svaki od 64 bita trake nezavisno i paralelno, pa se u hardveru sastoji od 1600 identičnih kopija jednog NOT, jednog AND i jednog XOR kola, što je ekvivalent nelinearne supstitucije nad svih 1600 bitova stanja odjednom, bez ijedne memorijske tabele.



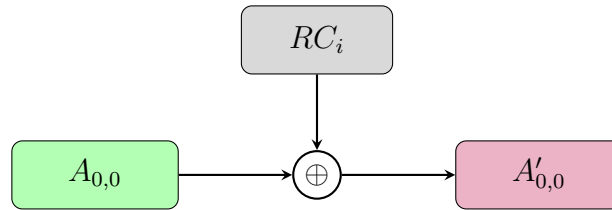
Slika 3.12: Korak χ : logika za jednu traku

3.2.5.5 Korak ι

Poslednji korak runde sabira konstantu runde sa jednom jedinom trakom:

$$A_{0,0} \leftarrow A_{0,0} \oplus RC_i, \quad (3.20)$$

gde je RC_i konstanta runde i , definisana standardom (sve 24 vrednosti date su u dodatku D). Bez ovog koraka sve runde bile bi identične, pa bi permutacija imala simetrije koje Oskar može iskoristiti. Konstante, različite u svakoj rundi, tu simetriju narušavaju, a delovanje je prikazano na slici 3.13.



Slika 3.13: Korak ι

Sa stanovišta hardverske realizacije, presudno je da se svi koraci svode na bitske operacije XOR, AND, NOT i fiksne rotacije, pri čemu fiksne rotacije u hardveru ne koštaju ništa. Jedna runda je stoga plitka kombinaciona mreža, a cela permutacija se prirodno realizuje iterativno, sa jednom rundom po taktu.

3.2.6 Varijante standarda

FIPS 202 definiše četiri heš funkcije fiksnog izlaza i dve XOF funkcije, sve nad istom permutacijom Keccak- $f[1600]$, a međusobno različite samo po brzini r , kapacitetu c i dužini izlaza, kako je dato u tabeli 3.3.

Tabela 3.3: Varijante SHA-3 standarda [16]

Funkcija	r [bit]	c [bit]	Izlaz d [bit]	Kolizije [bit]	Original [bit]
SHA3-224	1152	448	224	112	224
SHA3-256	1088	512	256	128	256
SHA3-384	832	768	384	192	384
SHA3-512	576	1024	512	256	512
SHAKE128	1344	256	proizvoljan	$\min(d/2, 128)$	$\min(d, 128)$
SHAKE256	1088	512	proizvoljan	$\min(d/2, 256)$	$\min(d, 256)$

Činjenica da sve varijante dele istu permutaciju izuzetno je povoljna za hardver, jer je jedno jezgro permutacije dovoljno za ceo standard, a varijanta se bira samo upravljačkom logikom koja određuje koliko se bitova stanja puni, odnosno čita. Uočava se i prirodan kompromis. Varijante sa većim kapacitetom pružaju veću sigurnost, ali obrađuju manje bitova poruke po pozivu permutacije, dakle imaju manju propusnost.

3.2.7 Izvedene funkcije: cSHAKE i KMAC

Nad istom sponge konstrukcijom NIST je standardom SP 800-185 [25] definisao familiju izvedenih funkcija (engl. *derived functions*), od kojih su za ovaj rad značajne cSHAKE i KMAC.

cSHAKE (engl. *customizable SHAKE*) proširuje SHAKE funkcije sa dva dodatna parametra: imenom funkcije N , rezervisanim za NIST-ove definicije, i oznakom

namene (engl. *customization string*) S , kojom korisnik razdvaja različite upotrebe iste funkcije. Parametri se kodiraju u prefiksni blok od jednog ili više celih r -bitnih blokova (za ime i oznaku namene iz ovog rada tačno jedan), koji se upija pre same poruke, a sufiks za razdvajanje domena iz tabele 3.2 menja se u 00 umesto 1111 kod SHAKE funkcija, dakle u bajt 0x04 umesto 0x1F.

Slučaj kada su i N i S prazni standard uređuje posebnim pravilom, a ne kao posledicu konstrukcije. Tada je po definiciji $\text{cSHAKE}(X, L, "", "") = \text{SHAKE}(X, L)$ [25]. To znači da se u tom slučaju prefiksni blok uopšte ne formira i da se koristi SHAKE sufiks 1111, dakle bajt 0x1F. Bez tog izuzetka cSHAKE sa praznim parametrima ne bi davao isti izlaz kao SHAKE, upravo zato što bi sufiks bio drugačiji.

Pre same definicije KMAC funkcije potrebne su četiri pomoćne funkcije kodiranja iz standarda [25]. Parametri koji ulaze u konstrukciju dve su vrste: brojevi (dužine) i stringovi (ime funkcije, oznaka namene, ključ), a ove četiri funkcije ih prevode u zapis iz koga se svaki sastavni deo može jednoznačno pročitati. Time se sprečava da dva različita skupa parametara daju isti ulazni niz. Sve četiri sažeto su date u tabeli 3.4.

Tabela 3.4: Pomoćne funkcije kodiranja iz standarda SP 800-185 [25]

Funkcija	Rezultat	Uloga
$\text{left_encode}(x)$	dužina zapisa, pa sam broj x (brojač dužine ispred)	dužina koja prethodi podatku
$\text{right_encode}(x)$	broj x , pa dužina zapisa (brojač dužine iza)	dužina koja sledi iza podatka
$\text{encode_string}(S)$	$\text{left_encode}(\text{len}(S)) \parallel S$	string kome prethodi dužina
$\text{bytepad}(X, w)$	$\text{left_encode}(w) \parallel X$, dopunjeno nulama do umnoška w bajtova	poravnanje na granicu bloka

Dve varijante zapisa dužine postoje zbog dva različita mesta na kojima zapis stoji. Kada dužina prethodi podatku, čitalac je pročitao pa zna koliko bajtova posle nje da uzme. Tako rade prefiksni blok i blok ključa, iza kojih sledi još sadržaja. Kodirana dužina izlaza, međutim, stoji na samom kraju ulaznog niza, iza poruke čija dužina unapred nije poznata, pa se do nje dolazi čitanjem sa kraja. Poslednji bajt kaže koliko bajtova broja stoji ispred njega, pa ona zato koristi right_encode .

KMAC (engl. *Keccak Message Authentication Code*) je funkcija sa tajnim ključem izgrađena nad cSHAKE funkcijom sa imenom $N = \text{"KMAC"}$:

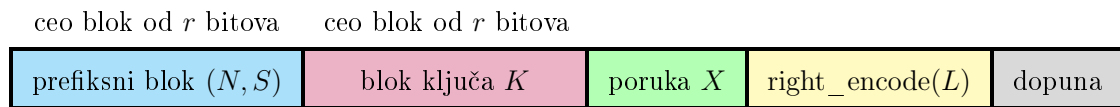
$$\begin{aligned} \text{KMAC}[K, X, L, S] = & \text{cSHAKE}(\text{bytepad}(\text{encode_string}(K), r/8) \\ & \parallel X \parallel \text{right_encode}(L), \\ & L, \text{"KMAC"}, S), \end{aligned} \quad (3.21)$$

gde se ključ K kodira i dopuni u ceo blok koji se upija pre poruke X , a $\text{right_encode}(L)$ na kraj poruke dodaje kodiranu dužinu izlaza L (varijanta KMACXOF, sa $\text{right_encode}(0)$, daje izlaz proizvoljne dužine u XOF režimu).

Postavlja se pitanje zašto je posebna konstrukcija uopšte potrebna, kada sunder nema svojstvo produženja poruke (odeljak 3.2.4), pa naivni oblik $H(K \parallel X)$ nad SHA-3 funkcijom nije ugrožen napadom koji kod Merkle–Damgardove klase nameće HMAC. KMAC ipak donosi tri svojstva koja taj oblik nema. Prva je kodiranje ključa sa dužinom upisanom ispred njega, čime granica ključa postaje deo samog niza, pa dva različita para ključa i poruke nikada ne daju isti ulaz. Druga je razdvajanje

domena kroz ime N i oznaku namene S , čime se isti ključ bezbedno koristi za više namena. Treća je kodirana dužina izlaza, čime skraćen izlaz nije prefiks dužeg. Rezultat je standardizovana pseudoslučajna funkcija (engl. *pseudorandom function*, PRF) čija sigurnost počiva neposredno na kapacitetu sponge konstrukcije, odobrena i kao MAC i kao funkcija za izvođenje ključeva, i sve to jednim pozivom heš funkcije, bez dva ulančana poziva koja traži HMAC.

Iako izraz (3.21) deluje glomazno, on ne uvodi nikakvo novo izračunavanje. Sve što opisuje jeste redosled kojim se polja nadovezuju u ulazni niz pre upijanja. Kako sastavljeni niz izgleda prikazuje slika 3.14. Bitno je uočiti da su prva dva polja celi r -bitni blokovi. Prefiksni blok nosi ime funkcije i oznaku namene, a blok ključa je dopunjen funkcijom bytepad do granice bloka. Tek posle njih dolazi sama poruka, pa kodirana dužina izlaza, i na kraju dopuna iz odeljka 3.2.3 sa sufiksom 00, jer je reč o cSHAKE funkciji.



Slika 3.14: Uokvirivanje ulaznog niza KMAC funkcije pre upijanja

Primer 3.2. U ulozu koju KMAC256 ima u ovom radu, u izvođenju ključeva, ulogu ključa K ima so (engl. salt) od 32 bajta, poruka X nosi zajedničku tajnu i kontekst razmene, ovde 72 bajta tajne i 40 bajtova konteksta, ukupno 112, oznaka namene S je "KDF", kako za ovu ulogu propisuje SP 800-56C [26], a dužina izlaza $L = 256$ bitova odgovara ključu za AES-256 (raspodela parametara data je u tabeli 3.5). Brzina upijanja je $r/8 = 136$ bajtova, pa je left_encode(136) = 01 88, i tim parom počinju oba uokvirena bloka:

Bajтови	Zapis	Značenje
01 88	left_encode(136)	brzina upijanja $r/8$
01 20	left_encode(32)	dužina imena, $4 \cdot 8$ bitova
4B 4D 41 43	"KMAC" u ASCII zapisu	ime funkcije N
01 18	left_encode(24)	dužina oznake, $3 \cdot 8$ bitova
4B 44 46	"KDF" u ASCII zapisu	oznaka namene S
00 $\times$ 123	dopuna nulama	do granice prvog bloka
01 88	left_encode(136)	brzina upijanja $r/8$
02 01 00	left_encode(256)	dužina ključa, $32 \cdot 8$ bitova
K , 32 bajta	—	so, u ulozu ključa
00 $\times$ 99	dopuna nulama	do granice drugog bloka
X , 112 bajtova	—	zajednička tajna i kontekst razmene
01 00 02	right_encode(256)	dužina izlaza L
04 00 ... 00 80	sufiks i dopuna	do granice trećeg bloka

Dužine stringova izražene su u bitovima, ne u bajtovima, pa ime od četiri znaka daje 0x20, a so od 32 bajta vrednost 256, koja se više ne smešta u jedan bajt. U ovom primeru se, sasvim nezavisno od toga, i za dužinu izlaza slučajno bira istih 256 bitova, a mogla je biti i drugačija. Baš zato što je broj isti, na njegova dva

zapisa vidi se razlika iz tabele 3.4. Zapis 02 01 00 u bloku ključa čita se unapred, sa brojem bajtova ispred, a 01 00 02 na kraju niza unazad, sa brojem bajtova iza.

Primer pokazuje i donju granicu cene. Prefiksni blok i blok ključa su celi blokovi bez obzira na to koliko su N , S i K kratki, a poruka sa kodiranom dužinom izlaza i završnom dopunom traži bar još jedan, pa KMAC nikada ne košta manje od tri poziva permutacije, ni kada su svi parametri prazni. Kod kratkih ulaza, kakvi se u izvođenju ključeva i javljaju, dva od ta tri bloka nose samo okvir. Isti raspored donosi i pogodnost. Poruka X uvek počinje na poznatoj granici bloka, kod KMAC256 posle tačno 272 bajta okvira, pa strana koja ulazni niz sastavlja upisuje unapred poznate bajtove na unapred poznata mesta i ne mora da prati pomeraj unutar bloka.

Raspodela parametara iz primera 3.2 upravo je ona koju KMAC ima u hibridnom sistemu iz odeljka 2.4, a to je izvođenje ključeva po jednokoračnoj KDF šemi iz NIST preporuke SP 800-56C [26]. Ona nije ista kao pri upotrebi za autentifikaciju poruka, a razliku uporedno prikazuje tabela 3.5. Suština je u tome što se tajnost seli iz ključa u poruku. Zajednička tajna Z dobijena ECDH razmenom ulazi u poruku X , nadovezana sa fiksnim kontekstom razmene (engl. *FixedInfo*), koji obuhvata identitete strana i oznaku protokola, dok je ključ K so, koja po standardu sme biti i unapred dogovorena javna vrednost. Oznaku namene S standard pritom fiksira na vrednost "KDF", čime je upotreba KMAC-a za izvođenje ključeva razdvojena od svih ostalih upotreba iste funkcije. Ključ sesije i inicijalne vrednosti seku se iz jednog izlaza tražene dužine, a njihov kontekst nosi FixedInfo.

Tabela 3.5: Uloge parametara KMAC funkcije: autentifikacija poruka naspram izvođenja ključeva

Parametar	KMAC kao MAC	KMAC kao KDF (SP 800-56C)
ključ K	tajni ključ koji strane dele	so, sme biti i javna vrednost
poruka X	podatak koji se autentifikuje	$Z \parallel \text{FixedInfo}$, tajna je u Z
oznaka S	razdvajanje namena istog ključa	standard je fiksira na "KDF"
izlaz	autentifikaciona oznaka	ključni materijal

U radu je korišćen KMAC256. Ovim izborom ceo lanac pokriven je NIST standardima: uspostavljanje zajedničke tajne po SP 800-56A [23], izvođenje ključeva po SP 800-56C [26], a zaštita saobraćaja po SP 800-38D [9].

Celokupno KMAC uokvirivanje, dakle prefiksni blok, blok ključa i kodirana dužina izlaza, jeste čisto formatiranje ulaznog niza, jer nijedan od tih koraka ne dira permutaciju niti stanje sundera. Za KMAC je zato dovoljna svaka realizacija cSHA-KE funkcije. Kako je to iskorišćeno u akceleratoru opisano je u odeljku 4.3.6.

3.3 AES-GCM

Treću komponentu sistema čini AES u GCM režimu rada, algoritam za simetrično šifrovanje kojim se štiti korisnički saobraćaj. U ovom odeljku najpre je opisan sam AES algoritam, njegova arhitektura, transformacije u rundi i ekspanzija ključa, a zatim GCM režim koji od blok šifre gradi autentifikovano šifrovanje. Detaljna analiza AES algoritma, sa kompletnim matematičkim osnovama i izvođenjima nad poljem $\text{GF}(2^8)$, data je u [27].

Izlaganje prati notaciju standarda [4, 9] uz jedno odstupanje. Otvoreni tekst, šifrat, autentifikaciona oznaka i pridruženi podaci (engl. *Additional Authenticated Data*) označeni su sa PT, CT, ICV i AAD, umesto sa P , C , T i A . Razlog je dvostruk. Takve oznake prate uobičajenu terminologiju mrežnih protokola, a simboli T i A su već zauzeti: T za T-tabele u odeljku 3.3.5, a A za stanje Keccak permutacije i operande množenja.

3.3.1 Nastanak standarda

NIST je 1997. godine raspisao javni konkurs za novi standard šifrovanja, kao zamenu za zastareli DES čija je dužina ključa od 56 bitova postala nedovoljna pred rastućom računarskom snagom. Nakon dve runde svetske evaluacije, u kojima su kandidati ocenjivani po sigurnosti, brzini u softveru i hardveru i jednostavnosti realizacije, za pobednika je 2000. godine proglašen algoritam Rijndael belgijskih kriptografa Joana Daemena i Vincenta Rijmena [28]. Standardizovana verzija, objavljena 2001. godine kao FIPS 197 [4], sužava fleksibilnost izvornog algoritma. Dužina bloka fiksirana je na 128 bitova, a dozvoljene dužine ključa su 128, 192 i 256 bitova, po kojima varijante nose nazive AES-128, AES-192 i AES-256.

3.3.2 Arhitektura algoritma

AES obrađuje blok od 128 bitova predstavljen kao matrica 4×4 bajtova, koja se naziva matrica stanja (engl. *state*). Bajtovi ulaznog bloka pune matricu po kolonama, a svaka transformacija algoritma ažurira stanje novim vrednostima. Broj kolona matrice stanja označava se sa N_b i za AES iznosi $N_b = 128/32 = 4$. Raspored bajtova u matrici prikazan je na slici 3.15.

$s_{0,0}$	$s_{0,1}$	$s_{0,2}$	$s_{0,3}$
$s_{1,0}$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$
$s_{2,0}$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$
$s_{3,0}$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$

Slika 3.15: Matrica stanja AES algoritma

Svaki bajt matrice stanja tumači se kao element polja $GF(2^8)$ sa redukcionim polinomom:

$$m(x) = x^8 + x^4 + x^3 + x + 1, \quad (3.22)$$

pa su sve operacije nad bajtovima zapravo operacije u tom polju. Sabiranje je XOR, a množenje se izvodi po modulu $m(x)$. Analogno matrici stanja, ključ se predstavlja matricom od N_k kolona, gde je N_k dužina ključa podeljena sa 32. Broj rundi N_r zavisi od dužine ključa, kako je prikazano u tabeli 3.6.

Tabela 3.6: Varijante AES algoritma [4]

Varijanta	Dužina ključa [bit]	N_k	N_r
AES-128	128	4	10
AES-192	192	6	12
AES-256	256	8	14

U ovom radu korišćen je AES-256.

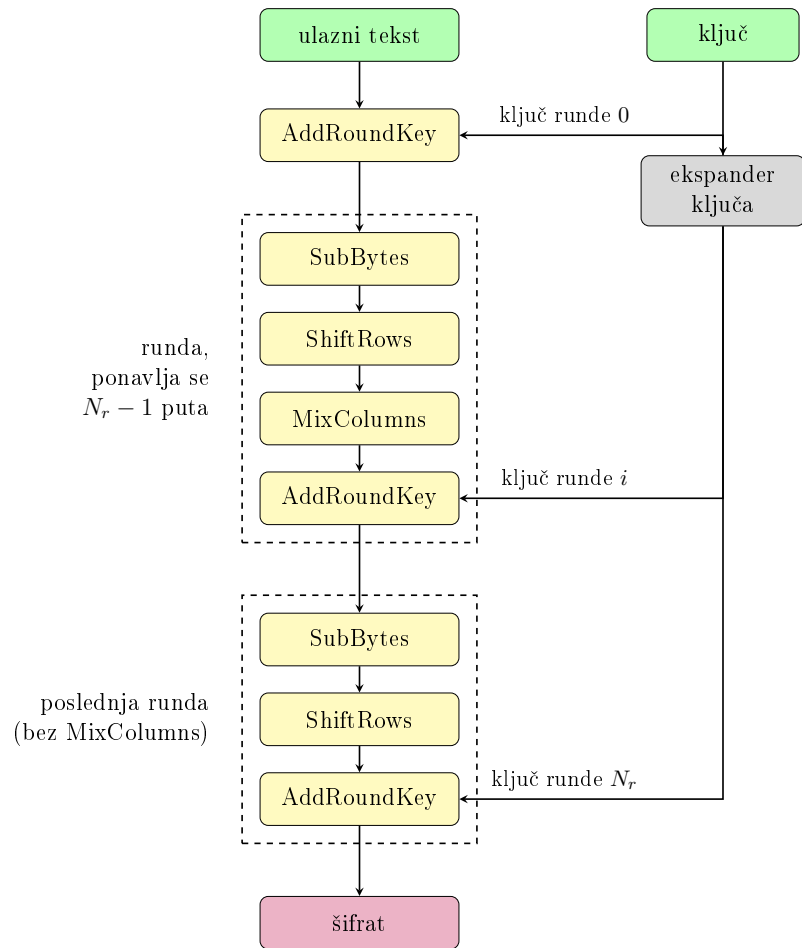
3.3.3 Tok šifrovanja

Šifrovanje počinje početnom operacijom `AddRoundKey`, kojom se ulazni blok sabira sa prvim ključem runde operacijom XOR. Zatim sledi N_r rundi, od kojih svaka primenjuje redom četiri transformacije: `SubBytes`, `ShiftRows`, `MixColumns` i `AddRoundKey`, s tim što se u poslednjoj rundi transformacija `MixColumns` izostavlja. Svaka runda koristi svoj ključ, dobijen postupkom ekspanzije ključa opisanim u odeljku 3.3.6.

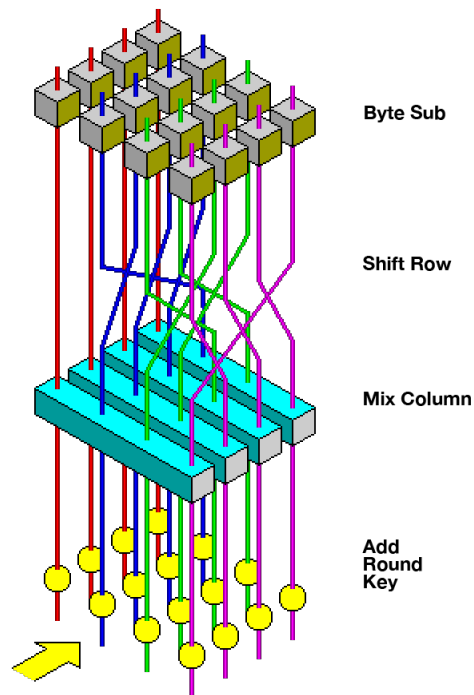
Svaka od četiri transformacije ima jasno odvojenu ulogu, a raspoređene su po klasičnoj podeli na konfuziju i difuziju:

- `SubBytes` je nosilac konfuzije i jedina nelinearna operacija u algoritmu. Bez nje bi ceo AES bio linearno preslikavanje, pa i trivijalno rešiv.
- `ShiftRows` i `MixColumns` zajedno čine linearni sloj, zadužen za difuziju. `MixColumns` meša četiri bajta unutar kolone, dok `ShiftRows` ne kombinuje nijedan bajt sa drugim, nego ih samo premešta tako da bajtovi jedne vrste završe u različitim kolonama.
- `AddRoundKey` unosi tajni materijal ključa u stanje. I ona je linearna operacija i sama ne doprinosi ni konfuziji ni difuziji. Njena je uloga da nelinearnost naredne runde ima na šta da deluje.

Podela posla unutar linearnog sloja objašnjava zašto su potrebne obe transformacije. `MixColumns` sam po sebi nikada ne bi izašao iz granica jedne kolone, a `ShiftRows` sam po sebi ne stvara nikakvu zavisnost među bajtovima. Tek zato što `ShiftRows` prethodno razmesti bajtove među kolone, `MixColumns` u narednoj rundi meša bajtove koji su prvobitno pripadali različitim kolonama. Posledica je da već posle dve runde svaki bajt stanja zavisi od svih šesnaest bajtova ulaza, a ponavljanjem kroz sve runde svaki bit izlaza postaje složena funkcija svih bitova ulaza i svih bitova ključa. Grafički prikaz celokupnog toka dat je na slici 3.16, dok slika 3.17 prikazuje delovanje transformacija jedne runde na matricu stanja.



Slika 3.16: Koraci u AES šifrovanju



Slika 3.17: Prikaz jedne runde AES šifrovanja. Preuzeto iz [29].

3.3.4 Transformacije u rundi

3.3.4.1 SubBytes

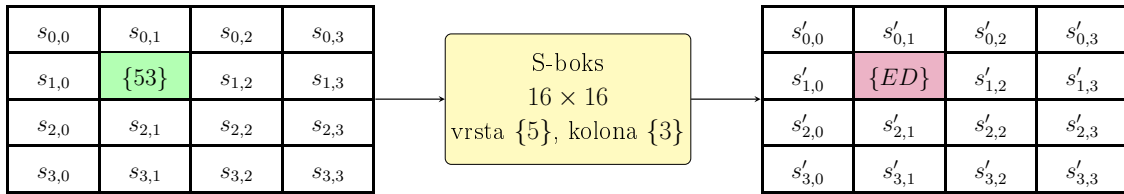
SubBytes zamenjuje svaki bajt matrice stanja vrednošću iz supstitucione tabele, S-boksa. Ta tabela nije proizvoljna, već je definisana kompozicijom dve operacije nad bajtom b :

1. multiplikativna inverzija u polju $\text{GF}(2^8) - b \mapsto b^{-1}$, uz dogovor $0 \mapsto 0$;
2. afina transformacija nad bitovima rezultata:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i, \quad (3.23)$$

gde je $c = \{63\} = 01100011_2$.

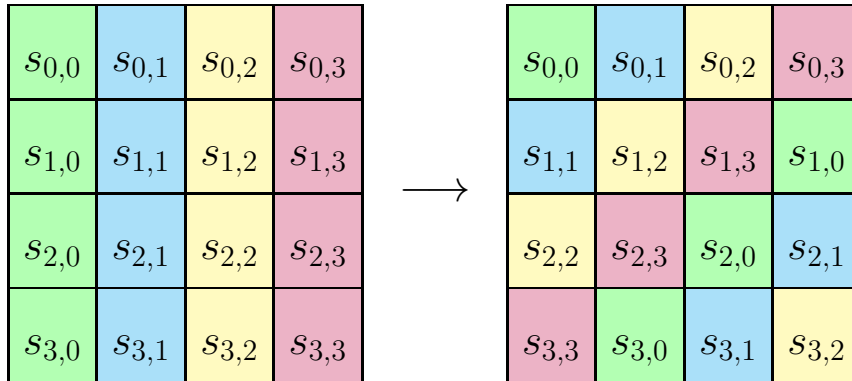
Inverzija u polju obezbeđuje visoku nelinearnost i otpornost na diferencijalnu i linearnu kriptanalizu, dok afina transformacija otklanja fiksne tačke. U realizacijama se S-boks najčešće ostvaruje kao tabela od 256 vrednosti, memorijska ili kombinatorna, mada je moguće i direktno izračunavanje inverzije dekompozicijom polja, što je izbor koji neposredno utiče na površinu i kritičnu putanju hardvera. Transformacija deluje na svaki bajt nezavisno, pa se matrica stanja preslikava element po element, kako prikazuje slika 3.18: gornja četiri bita bajta biraju vrstu, a donja četiri kolonu S-boksa. Na slici je istaknut bajt $\{53\}$, koji se preslikava u $\{ED\}$. Kompletan tabelu S-boksa data je u dodatku B.



Slika 3.18: SubBytes transformacija na primeru bajta $\{53\} \mapsto \{ED\}$

3.3.4.2 ShiftRows

ShiftRows ciklično pomera vrste matrice stanja ulevo. Vrstu r pomera se za r mesta, dakle nulta vrsta ostaje nepromenjena. Transformacija je prikazana na slici 3.19. Kolone polazne matrice označene su bojama, pa se u rezultatu jasno vidi za koliko je mesta svaka vrsta pomerena.



Slika 3.19: ShiftRows transformacija

U hardveru je ShiftRows besplatna operacija, jer se svodi na preusmeravanje veza, bez logičkih elemenata i bez takta latencije.

3.3.4.3 MixColumns

MixColumns transformiše svaku kolonu matrice stanja množenjem fiksnom matricom nad $\text{GF}(2^8)$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} \{02\} & \{03\} & \{01\} & \{01\} \\ \{01\} & \{02\} & \{03\} & \{01\} \\ \{01\} & \{01\} & \{02\} & \{03\} \\ \{03\} & \{01\} & \{01\} & \{02\} \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \quad c = 0, \dots, 3, \quad (3.24)$$

gde vitičaste zagrade označavaju bajt u heksadecimalnom zapisu, tumačen kao element polja.

Koeficijenti matrice nisu izabrani proizvoljno, nego tako da istovremeno ispune dva zahteva. Prvi je da difuzija bude najbolja moguća. Matrica je MDS (engl. *Maximum Distance Separable*), pojam preuzet iz teorije kodova, koji ovde znači da je njena razgranatost (engl. *branch number*) jednaka 5, pa zbir broja izmenjenih bajtova na ulazu i na izlazu ne može biti manji od 5. Praktična posledica je da promena makar jednog bajta u ulaznoj koloni menja sva četiri bajta izlazne kolone, što je najviše što se matricom 4×4 može postići [28].

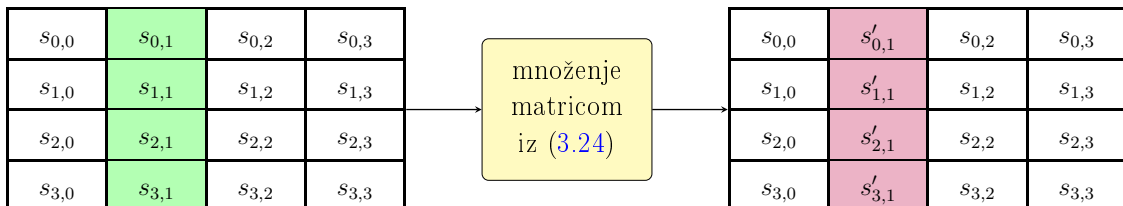
Drugi zahtev je da množenja budu jeftina. Transformacija koristi samo koeficijente $\{01\}$, $\{02\}$ i $\{03\}$, a sva tri se svode na množenje sa $\{02\}$, koje u polju odgovara množenju polinoma sa x . Pošto bit i bajta predstavlja koeficijent uz x^i , to je prosto pomeranje bajta ulevo. Ako pritom najstariji bit ispadne iz bajta, javlja se član x^8 , pa se rezultat redukuje po modulu $m(x)$, odnosno sabira sa $\{1B\}$:

$$\{02\} \cdot a = \begin{cases} a \ll 1, & a_7 = 0, \\ (a \ll 1) \oplus \{1B\}, & a_7 = 1. \end{cases} \quad (3.25)$$

Množenje sa $\{03\}$ ne traži ništa novo, jer je $\{03\} \cdot a = \{02\} \cdot a \oplus a$. Cela MixColumns transformacija time se u hardveru svodi na pomeranja, uslovno sabiranje sa konstantom i XOR operacije, bez ijednog množača i bez tabele u memoriji.

Primer 3.3. Kod množenja bajta $\{57\}$ sa $\{02\}$ najstariji bit od 01010111_2 je nula, pa je rezultat samo pomeranje ulevo, $\{57\} \cdot \{02\} = 10101110_2 = \{AE\}$. Kod množenja $\{AE\}$ sa $\{02\}$ najstariji bit od 10101110_2 je jedinica, pa se posle pomeranja (01011100_2) dodaje $\{1B\}$, dakle $\{AE\} \cdot \{02\} = 01011100_2 \oplus 00011011_2 = 01000111_2 = \{47\}$.

Slika 3.20 prikazuje delovanje transformacije. Svaka kolona nezavisno prolazi kroz blok koji izvršava množenje matricom iz izraza (3.24), pa se u hardveru MixColumns prirodno realizuje sa četiri identična modula koji rade paralelno.



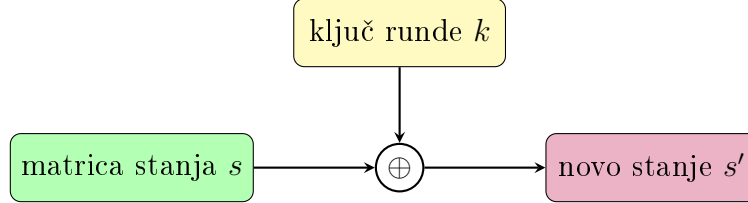
Slika 3.20: MixColumns transformacija nad jednom kolonom matrice stanja

3.3.4.4 AddRoundKey

AddRoundKey sabira matricu stanja sa ključem tekuće runde operacijom XOR:

$$s'_{r,c} = s_{r,c} \oplus k_{r,c}. \quad (3.26)$$

Operacija je involutivna, jer ponovna primena istog ključa poništava njeno dejstvo, pa je pri dešifrovanju identična kao pri šifrovanju. Šematski prikaz dat je na slici 3.21.



Slika 3.21: AddRoundKey transformacija

3.3.5 Fuzija runde: T-tabele

Cela runda, osim poslednje, može se stopiti u jednu tabelarnu operaciju. Ova tehnika, data već u izornoj specifikaciji Rijndael algoritma [28], prvobitno je smišljena za softverske realizacije na 32-bitnim procesorima, ali je jednako primenljiva u hardveru, gde je i korišćena u realizaciji opisanoj u poglavlju 4.

Posmatrajmo jednu kolonu izlaza runde. Kombinovanjem pomeranja vrsta sa slike 3.19 i množenja iz izraza (3.24), kolona c posle SubBytes, ShiftRows i MixColumns iznosi:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} \{02\} & \{03\} & \{01\} & \{01\} \\ \{01\} & \{02\} & \{03\} & \{01\} \\ \{01\} & \{01\} & \{02\} & \{03\} \\ \{03\} & \{01\} & \{01\} & \{02\} \end{bmatrix} \begin{bmatrix} S[s_{0,c}] \\ S[s_{1,c+1}] \\ S[s_{2,c+2}] \\ S[s_{3,c+3}] \end{bmatrix}, \quad (3.27)$$

gde $S[\cdot]$ označava S-boks, a pomereni indeksi vrsta ($c+1$, $c+2$, $c+3$, po modulu 4) potiču od ShiftRows transformacije. Pošto je matrica konstantna, proizvod se može rastaviti na četiri sabirka, od kojih svaki zavisi od tačno jednog ulaznog bajta:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = T_0[s_{0,c}] \oplus T_1[s_{1,c+1}] \oplus T_2[s_{2,c+2}] \oplus T_3[s_{3,c+3}], \quad (3.28)$$

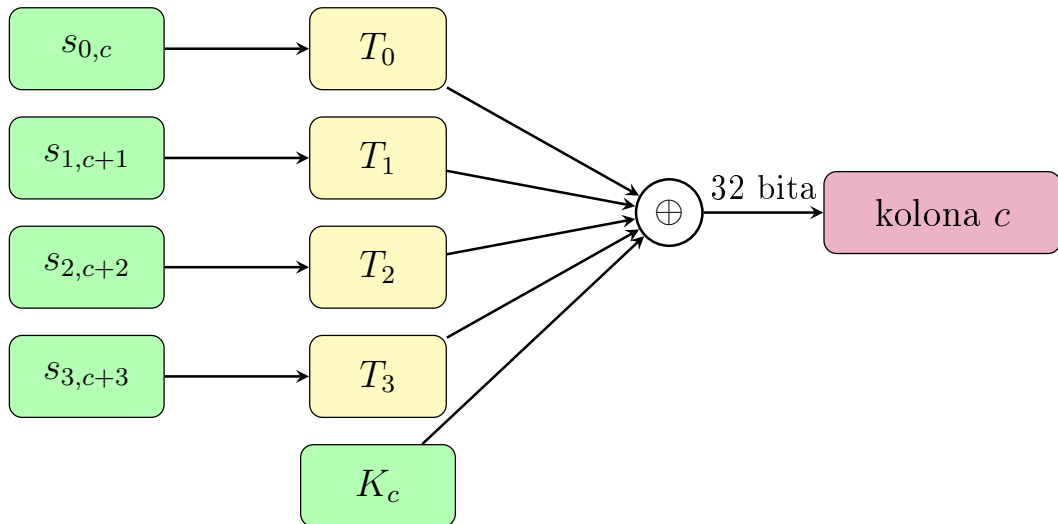
gde su T_0 do T_3 tabele sa po 256 unosa od 32 bita, definisane kao:

$$\begin{aligned}
 T_0[x] &= \begin{bmatrix} \{02\} \cdot S[x] \\ S[x] \\ S[x] \\ \{03\} \cdot S[x] \end{bmatrix}, & T_1[x] &= \begin{bmatrix} \{03\} \cdot S[x] \\ \{02\} \cdot S[x] \\ S[x] \\ S[x] \end{bmatrix}, \\
 T_2[x] &= \begin{bmatrix} S[x] \\ \{03\} \cdot S[x] \\ \{02\} \cdot S[x] \\ S[x] \end{bmatrix}, & T_3[x] &= \begin{bmatrix} S[x] \\ S[x] \\ \{03\} \cdot S[x] \\ \{02\} \cdot S[x] \end{bmatrix}.
 \end{aligned} \tag{3.29}$$

Dodavanjem ključa runde dobija se kompletna runda kao XOR pet 32-bitnih vrednosti:

$$\text{kolona } c = T_0[s_{0,c}] \oplus T_1[s_{1,c+1}] \oplus T_2[s_{2,c+2}] \oplus T_3[s_{3,c+3}] \oplus K_c. \tag{3.30}$$

Postupak je prikazan na slici 3.22.



ShiftRows je sadržan
u izboru indeksa

Slika 3.22: Fuzija runde T-tabelama

Tri svojstva čine ovu transformaciju značajnom:

- *ShiftRows nestaje*. Ne izvršava se kao operacija, nego određuje samo koja četiri bajta stanja ulaze u koju kolonu. Ovaj dobitak pripada pre svega softveru. Na procesoru koji radi nad 32-bitnim rečima ShiftRows bi se izvodio nizom pomeranja i maskiranja pojedinačnih bajtova, dok je u hardveru i bez T-tabela bio puko preusmeravanje veza.

- *Tabele su međusobno rotacije.* Iz definicije (3.29) neposredno sledi da je $T_i[x]$ jednako $T_0[x]$ ciklično rotiranom za i bajtova, pa je dovoljno čuvati samo T_0 . Ostale tri su preuređenje bajtova, dakle besplatne.
- *Cena je memorija umesto računa.* Jedna tabela zauzima 256×32 bita, odnosno 1 KB. Kolona se svodi na četiri čitanja i XOR umesto na lanac supstitucije i množenja u polju, a cela runda na šesnaest čitanja.

Poslednja runda nema MixColumns, pa se u njoj fuzija ne može primeniti. Ona se uvek realizuje neposredno. Hardverske posledice ovog izbora, uključujući pitanje koliko memorijskih blokova zaista košta, razmotrene su u odeljku 4.4.2.5.

3.3.6 Ekspanzija ključa

Za početnu operaciju i svaku od N_r rundi potreban je poseban 128-bitni ključ runde, ukupno $N_b(N_r+1)$ kolona od po 32 bita. Za AES-256 to je 60 kolona, odnosno 15 ključeva rundi.

Prošireni ključ se posmatra kao niz kolona $W_0, W_1, W_2, \dots$, gde je svaka kolona reč od 32 bita, dakle četiri bajta. Pojedinačni bajt označava se sa $W_{i,j}$, pri čemu je $i \in \{0, 1, 2, 3\}$ redni broj bajta unutar kolone, a j redni broj same kolone. Ključ runde r čine četiri uzastopne kolone, W_{4r} do W_{4r+3} , tako da prvih N_k kolona ne daje nužno ceo ključ prve runde. Kod AES-256 ulazni ključ od osam kolona daje ključ runde 0 i ključ runde 1.

Kolone generiše ekspander ključa rekurzijom. Prvih N_k kolona je sam ulazni ključ, a za $j \geq N_k$:

$$W_j = \begin{cases} W_{j-N_k} \oplus \text{SubWord}(\text{RotWord}(W_{j-1})) \oplus \text{Rcon}_{j/N_k}, & j \bmod N_k = 0, \\ W_{j-N_k} \oplus \text{SubWord}(W_{j-1}), & N_k > 6 \wedge j \bmod N_k = 4, \\ W_{j-N_k} \oplus W_{j-1}, & \text{inače,} \end{cases} \quad (3.31)$$

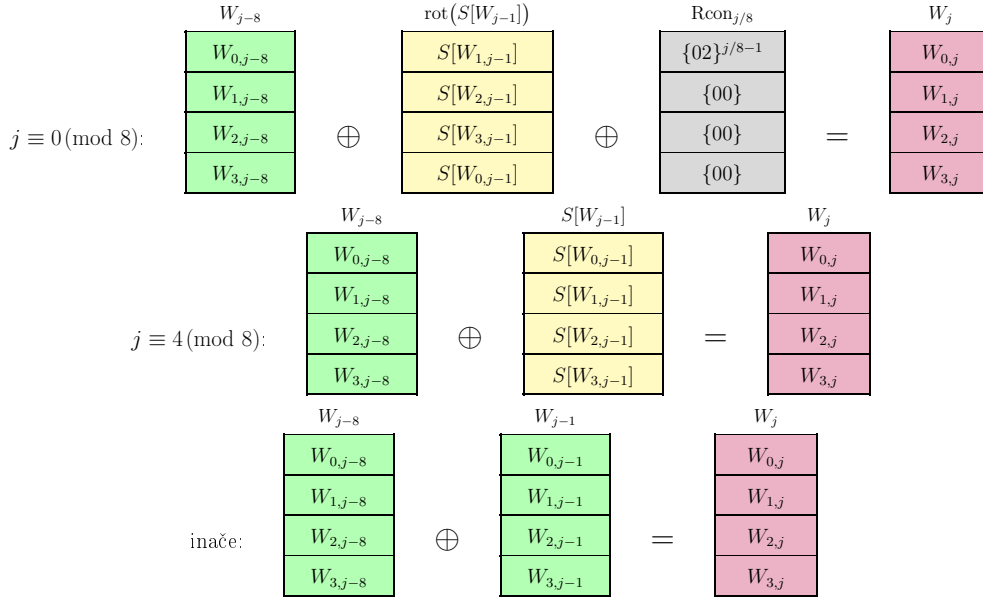
Ovde RotWord ciklično rotira kolonu za jedan bajt naviše, a SubWord primenjuje S-boks na svaki od četiri bajta kolone.

Konstanta runde Rcon_i je kolona kojoj su tri donja bajta nule, a gornji bajt je $(i-1)$ -ti stepen elementa $\{02\}$ u polju $\text{GF}(2^8)$:

$$\text{Rcon}_i = (\{02\}^{i-1}, \{00\}, \{00\}, \{00\}). \quad (3.32)$$

Pošto je množenje sa $\{02\}$ upravo pomeranje ulevo iz izraza (3.25), niz konstanti počinje kao $\{01\}, \{02\}, \{04\}, \{08\}, \{10\}, \{20\}, \{40\}, \{80\}$, a zatim, kada pomeranje izbaci najstariji bit, sledi redukcija: $\{80\} \cdot \{02\} = \{1B\}$, pa $\{36\}$ i tako dalje. Uloga ovih konstanti je da razbiju pravilnosti ekspanzije. Bez njih bi se svaka grupa od N_k kolona računala potpuno istim pravilom, a ekspanzija bi uz to bila invarijantna na rotaciju bajtova unutar kolone. Rcon je jedina veličina vezana i za konkretnu grupu i za konkretan bajt, pa obe pravilnosti nestaju. Detaljnije obrazloženje dato je u [28], a sve vrednosti konstanti u dodatku B.2.

Srednja grana rekurzije postoji samo kod AES-256 ($N_k = 8$) i unosi dodatnu nelinearnost u sredinu svake grupe od osam kolona. Struktura rekurzije prikazana je na slici 3.23. Nova kolona se uvek dobija sabiranjem kolone udaljene N_k mesta i, po potrebi transformisane, prethodne kolone, pri čemu $S[\cdot]$ označava primenu S-boksa na bajt.



Slika 3.23: Ekspanzija ključa za AES-256: izračunavanje kolone W_j , $j \geq 8$, po slučajevima

3.3.7 Dešifrovanje

Sve transformacije AES algoritma su invertibilne, pa standard [4] uz šifrovanje definiše i inverznu šifru. Transformacije `InvSubBytes`, `InvShiftRows` i `InvMixColumns` primenjuju se obrnutim redosledom, sa ključevima rundi uzetim od poslednjeg ka prvom. Za ovaj rad je bitno da režimi CTR familije, kojoj pripada i GCM, blok šifru pozivaju isključivo u smeru šifrovanja, i pri šifrovanju i pri dešifrovanju, kako je pokazano u odeljku 3.3.8.3. Inverzna šifra zato u realizaciji nije ni potrebna.

3.3.8 Režimi rada blok šifara

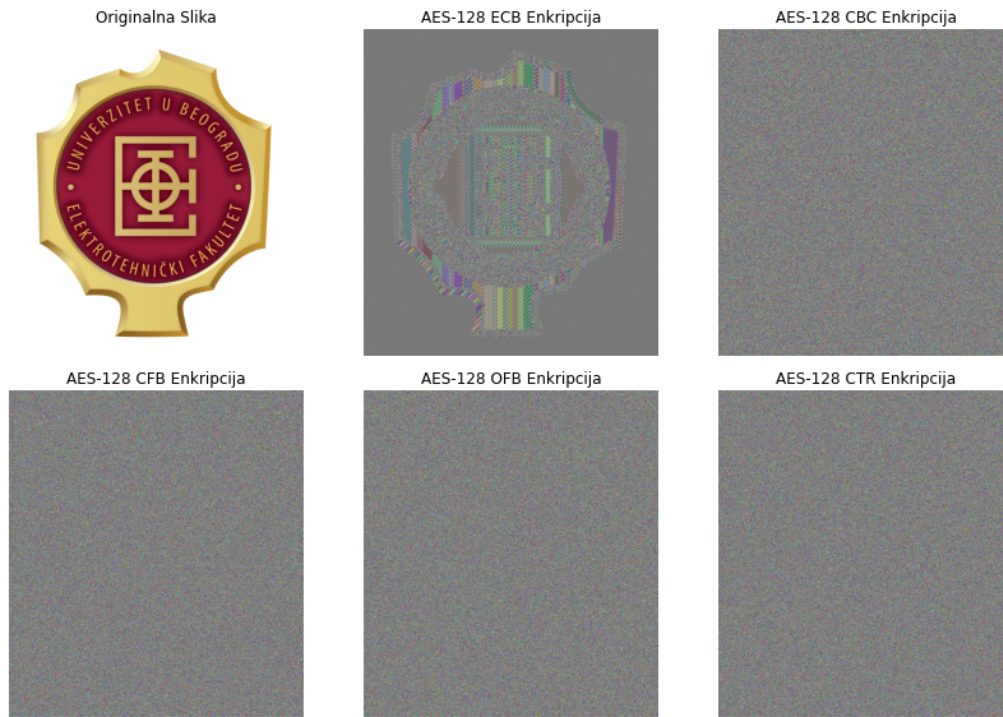
AES definiše transformaciju samo nad jednim blokom od 128 bitova. Način primene šifre na poruke proizvoljne dužine određuje režim rada. Izbor režima određuje i sigurnosna svojstva koja sistem pruža i strukturu hardvera koji ga realizuje. U nastavku su razmotrena tri karakteristična režima, redosledom koji vodi ka GCM-u.

3.3.8.1 ECB režim

ECB (engl. *Electronic Codebook*) je najjednostavniji režim, u kome se svaki blok šifruje nezavisno istim ključem,

$$\text{CT}_i = E_K(\text{PT}_i). \quad (3.33)$$

Posledica je da identični blokovi otvorenog teksta daju identične blokove šifrata, pa statistička struktura poruke ostaje vidljiva u šifratu. Slika 3.24 ovo čini očiglednim. Na slici pečata šifrovanj u ECB režimu pozadina se preslikava u identične vrednosti i kontura pečata se jasno nazire, dok se CBC i CTR šifratu ne razlikuju od šuma. Detaljna kritička analiza ECB režima data je u [27].



Slika 3.24: AES-128 šifrovanje slike u različitim režimima rada. Preuzeto iz [27].

3.3.8.2 CBC režim

CBC (engl. *Cipher Block Chaining*) otklanja slabost ECB režima ulančavanjem blokova, pri kome se svaki blok otvorenog teksta pre šifrovanja sabira sa prethodnim blokom šifrata,

$$CT_i = E_K(PT_i \oplus CT_{i-1}), \quad CT_0 = IV, \quad (3.34)$$

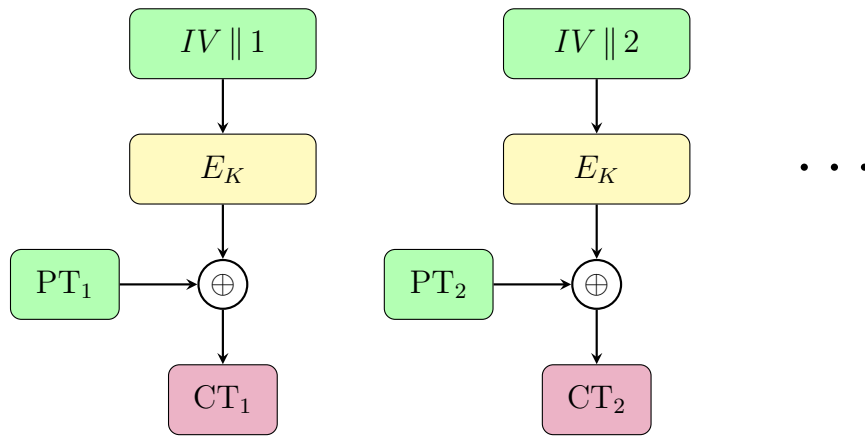
gde je IV nasumični inicijalizacioni vektor. Identični blokovi otvorenog teksta sada daju različite šifrate, ali se cena plaća upravo u onome što je za hardver najvažnije, u propusnosti. Šifrovanje bloka i ne može početi pre nego što se završi blok $i - 1$. Povratna sprega onemogućava protočnu obradu, pa AES jezgro sa latencijom od L taktova postiže tek jedan blok na svakih L taktova, ma koliko resursa bilo na raspolaganju. Pored toga, CBC zahteva dopunu poruke do umnoška bloka i i koristi i smer dešifrovanja.

3.3.8.3 CTR režim

CTR (engl. *Counter*) režim ide drugim putem. Umesto ulančavanja podataka šifruje se niz vrednosti brojača, a dobijeni pseudoslučajni niz sabira sa otvorenim tekstom,

$$CT_i = PT_i \oplus E_K(IV \parallel i), \quad (3.35)$$

čime se blok šifra efektivno pretvara u protočnu šifru, a šema režima prikazana je na slici 3.25.



Slika 3.25: CTR režim rada: blokovi se obrađuju nezavisno i paralelno

Svojstva CTR režima čine ga idealnim polazištem za hardver visoke propusnosti. Blokovi su potpuno nezavisni, jer brojački blok ne zavisi od podataka, pa je moguća i savršena protočnost i paralelizam. Poruka ne zahteva dopunu, jer se od poslednjeg bloka pseudoslučajnog niza uzima tačno onoliko bitova koliko je potrebno. Uz to, dešifrovanje je identičan postupak. Primalac istim ključem i istim brojačkim blokovima proizvede isti pseudoslučajni niz i sabere ga sa šifratom, pa se blok šifra i pri dešifrovanju poziva samo u smeru šifrovanja, a inverzna šifra iz odeljka 3.3.7 nije potrebna. Kritičan zahtev je da se vrednost brojača nikada ne ponovi sa istim ključem.

3.3.8.4 Od CTR do GCM

CTR režim, međutim, ne pruža nikakvu zaštitu integriteta. Štaviše, izrazito je savitljiv (engl. *malleable*). Pošto je $CT_i = PT_i \oplus ks_i$, ako Oskar promeni proizvoljan bit šifrata, promenio je tačno taj bit dešifrovanog teksta, predvidljivo i neprimetno. Šifrovana poruka se stoga u praksi mora štititi i kodom za autentifikaciju, a GCM je upravo standardizovan odgovor na tu potrebu, sa CTR režimom za poverljivost i GHASH funkcijom za integritet, uz obradu pridruženih podataka. Tabela 3.7 sumira poređenje.

Tabela 3.7: Poređenje CTR i GCM režima rada

Svojstvo	CTR	GCM
poverljivost	da	da
integritet i autentifikacija poruke	ne	da
pridruženi podaci (AAD)	ne	da
paralelna obrada blokova	da	da za šifrovanje, a autentifikacija je serijska, jedno množenje po bloku (odeljak 3.3.9.5)
koristi samo smer šifrovanja	da	da
dodatni hardver	—	množač u $GF(2^{128})$ (GHASH)

3.3.9 GCM režim rada

GCM (engl. *Galois/Counter Mode*) [9, 30] je AEAD režim rada koji objedinjuje dva gradivna elementa: CTR režim šifrovanja, zadužen za poverljivost, i GHASH funkciju autentifikacije, zaduženu za integritet. Ulaz u GCM šifrovanje čine:

- tajni ključ K (u ovom radu 256-bitni),
- nonce vrednost IV , jedinstvena za svaku poruku šifrovanu istim ključem,
- otvoreni tekst PT (engl. *plaintext*) proizvoljne dužine,
- pridruženi podaci AAD , koji se ne šifruju, ali ulaze u autentifikaciju. Tipično su to zaglavlja mrežnih paketa, koja moraju ostati čitljiva usput, a ne smeju biti izmenjena.

Izlaz su šifrat CT (engl. *ciphertext*), iste dužine kao PT , i autentifikaciona oznaka ICV (engl. *Integrity Check Value*).

Postupak počinje izvođenjem dve vrednosti. Prva je heš potključ (engl. *hash subkey*):

$$H = E_K(0^{128}), \quad (3.36)$$

dobijen šifrovanjem nultog bloka, koji parametrizuje GHASH funkciju, a druga je početni brojački blok J_0 . Za preporučenu dužinu nonce vrednosti od 96 bitova formira se jednostavnim nadovezivanjem,

$$J_0 = IV \parallel 0^{31} \parallel 1, \quad (3.37)$$

dok se za ostale dužine IV izvodi GHASH funkcijom nad dopunjenim IV . Upravo zbog jednostavnosti izraza (3.37) hardverske realizacije po pravilu fiksiraju dužinu nonce vrednosti na 96 bitova.

3.3.9.1 CTR šifrovanje

Otvoreni tekst deli se na $n = \lceil \text{len}(PT)/128 \rceil$ blokova $PT_1, \dots, PT_n$, pri čemu poslednji blok može biti nepotpun. *Brojački blokovi* (engl. *counter block*) CB_i generišu se funkcijom inc_{32} , koja inkrementira najnižih 32 bita bloka po modulu 2^{32} , ne dirajući gornjih 96 bitova:

$$CB_1 = \text{inc}_{32}(J_0), \quad CB_i = \text{inc}_{32}(CB_{i-1}), \quad i = 2, \dots, n. \quad (3.38)$$

Svaki brojački blok se šifruje, a rezultat sabira operacijom XOR sa odgovarajućim blokom otvorenog teksta:

$$CT_i = PT_i \oplus E_K(CB_i), \quad i = 1, \dots, n-1, \quad (3.39)$$

dok se za poslednji, potencijalno nepotpun blok dužine u bitova koristi samo najznačajnijih u bitova šifrovanog brojača:

$$CT_n = PT_n \oplus \text{MSB}_u(E_K(CB_n)). \quad (3.40)$$

Brojački niz počinje od J_0 , ali šifrovanje podataka počinje tek od CB_1 , jer sam J_0 nikada ne šifruje podatke, nego je njegov šifrat $E_K(J_0)$ rezervisan za masku autentifikacione oznake u izrazu (3.45). Time nijedan blok pseudoslučajnog niza nije upotrebljen dvaput, jer se maska oznake ne poklapa ni sa jednim blokom koji štiti podatke. Grana za poverljivost je, dakle, običan CTR režim nad blokovima $CB_1, \dots, CB_n$, pa za nju važe sva svojstva iz odeljka 3.3.8, pre svega protočna obrada bez povratne sprege.

3.3.9.2 GHASH funkcija

Autentifikacija u GCM režimu zasniva se na funkciji GHASH, definisanoj nad poljem $\text{GF}(2^{128})$ sa redukcionim polinomom:

$$f(x) = x^{128} + x^7 + x^2 + x + 1. \quad (3.41)$$

Blokovski poruke tumače se kao koeficijenti polinoma nad tim poljem, a sažetak je vrednost tog polinoma u tački H . Za niz 128-bitnih blokova $X_1, X_2, \dots, X_m$ to znači:

$$\text{GHASH}_H(X) = X_1 \cdot H^m \oplus X_2 \cdot H^{m-1} \oplus \dots \oplus X_m \cdot H. \quad (3.42)$$

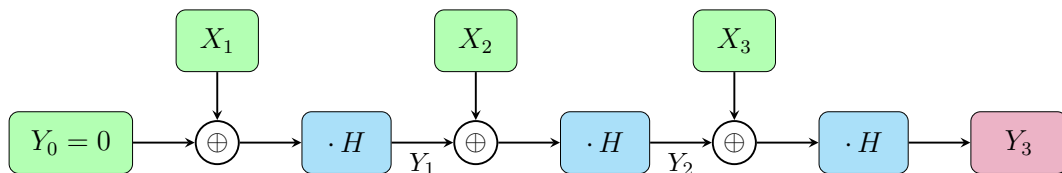
Pošto je H tajan, Oskar ne zna u kojoj se tački polinom računa, pa ne može da nađe drugu poruku sa istom vrednošću. Konstrukcije ovog tipa u literaturi se zovu univerzalne heš funkcije (engl. *universal hash function*). Za razliku od SHA-3, one nisu sigurne same po sebi, nego tek uz tajni parametar i jednokratnu masku.

Izraz (3.42) se ne izračunava neposredno, jer bi tražio sve stepene $H^2, \dots, H^m$, nego Hornerovom šemom, u kojoj se svaki blok sabere sa akumulatorom, a zbir pomnoži sa H :

$$Y_0 = 0^{128}, \quad Y_i = (Y_{i-1} \oplus X_i) \cdot H, \quad \text{GHASH}_H(X) = Y_m. \quad (3.43)$$

Time je po bloku potrebno jedno jedino množenje u polju, i to sa uvek istim činiocem H . Sekvencijalni tok izračunavanja prikazan je na slici 3.26.

Ostaje još jedan detalj zapisa. Standard [9] bitove 128-bitnog bloka pridružuje stepenima ogledalski u odnosu na uobičajeni binarni zapis. Krajnji levi bit bloka je koeficijent uz x^0 , a krajnji desni uz x^{127} . Na matematička svojstva polja to nema uticaja, a u hardveru se svodi na preuređenje veza na ulazu i izlazu množača, bez ijednog logičkog elementa. Jedini uslov je da ono bude dosledno sprovedeno.



Slika 3.26: GHASH funkcija kao Hornerova šema

Ulazni niz GHASH funkcije formira se nadovezivanjem pridruženih podataka AAD i šifrata CT, svakog dopunjenog nulama do umnoška 128 bitova, i završnog bloka koji sadrži 64-bitne zapise njihovih dužina:

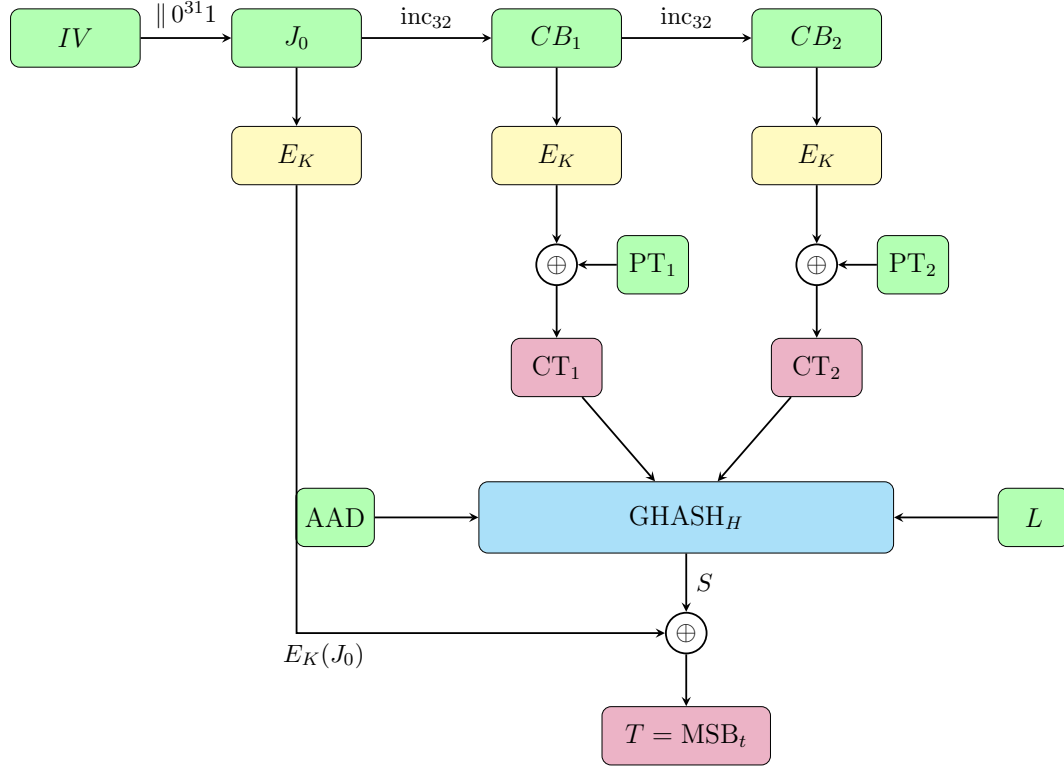
$$S = \text{GHASH}_H(\text{AAD} \parallel 0^v \parallel \text{CT} \parallel 0^w \parallel \text{len}_{64}(\text{AAD}) \parallel \text{len}_{64}(\text{CT})). \quad (3.44)$$

gde su v i w najmanji brojevi nula koji dužine AAD i CT dovode do umnoška 128 bitova. Uključivanje dužina u poslednji blok sprečava napade produžavanjem i preraspodelom granice između AAD i CT. Konačno, oznaka se dobija šifrovanjem vrednosti S blokom J_0 :

$$\text{ICV} = \text{MSB}_t(E_K(J_0) \oplus S), \quad (3.45)$$

gde je t dužina oznake (pun format je $t = 128$). Ovaj korak je i razlog zašto sama GHASH funkcija ne mora biti kriptografski sigurna. Vrednost S nikada ne napušta jezgro nezaštićena, nego se sabira sa maskom $E_K(J_0)$, koja je za svaku poruku

druga. Bez poznavanja ključa Oskar ne može izračunati tu masku, pa ni konstruisati ispravnu oznaku za izmenjenu poruku. Celokupan postupak GCM šifrovanja, sa CTR granom i GHASH granom kao odvojenim funkcionalnim celinama, prikazan je na slici 3.27 na primeru poruke od dva bloka. Blok L na slici je završni blok sa dužinama, $L = \text{len}_{64}(\text{AAD}) \parallel \text{len}_{64}(\text{CT})$, iz izraza (3.44).



Slika 3.27: Blok šema GCM šifrovanja poruke od dva bloka: CTR grana (žuto/zeleno) i GHASH grana (cijan)

Primer 3.4. Za poruku sa jednim blokom pridruženih podataka AAD_1 i dva bloka šifrata CT_1, CT_2 ulazni niz ima četiri bloka, pa izraz (3.42) daje:

$$S = \text{AAD}_1 \cdot H^4 \oplus \text{CT}_1 \cdot H^3 \oplus \text{CT}_2 \cdot H^2 \oplus L \cdot H,$$

gde je $L = \text{len}_{64}(\text{AAD}) \parallel \text{len}_{64}(\text{CT})$ završni blok sa dužinama. Isti rezultat Hornerova šema (3.43) dobija u četiri koraka, sa jednim množenjem po koraku.

3.3.9.3 Množenje u $\text{GF}(2^{128})$

Sve što GCM režim traži od aritmetike svodi se na jednu operaciju, množenje dva elementa polja $\text{GF}(2^{128})$ iz izraza (3.43). Sabiranje je XOR i praktično je besplatno, a šifrovanje obavlja AES, pa je ovo množenje jedina netrivialna računaska operacija u režimu. Kako se izvršava po jednom za svaki blok poruke, ono i određuje gornju granicu propusnosti autentifikacije.

Množenje se razlaže na dva koraka: množenje polinoma bez prenosa, koje daje nesvedeni proizvod stepena do 254, i redukciju po modulu polinoma (3.41).

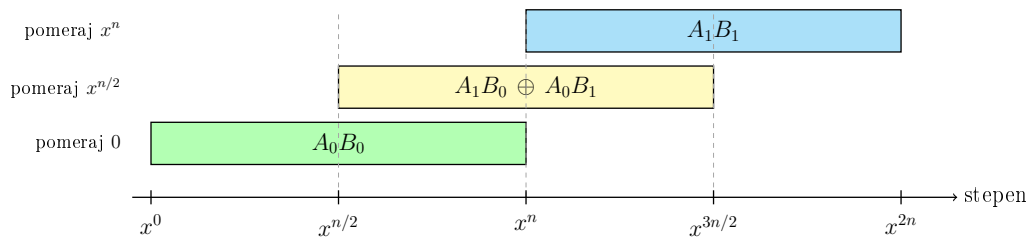
Množenje polinoma. Školski postupak, u kome se svaki bit jednog činioca množi svakim bitom drugog, traži $128^2 = 16\,384$ parcijalnih proizvoda. U polju karakteristike 2 svaki od njih je jedno logičko I kolo, a njihovo sabiranje jedno stablo XOR kola. Kvadratni rast sa dužinom operanda čini ovaj pristup izuzetno rasipnim za polja ovakve veličine.

Povoljniji postupak daje Karatsubin algoritam [31]. Operandi se rastave na polovine, $A = A_1x^{n/2} + A_0$ i $B = B_1x^{n/2} + B_0$, pa je proizvod:

$$A \cdot B = A_1B_1x^n \oplus [(A_1 \oplus A_0)(B_1 \oplus B_0) \oplus A_1B_1 \oplus A_0B_0]x^{n/2} \oplus A_0B_0. \quad (3.46)$$

Raspored tri proizvoda po rezultatu prikazan je na slici 3.28. Svaki je širok najviše $n - 1$ bitova, a pomereni su za 0, $n/2$ i n mesta, pa se susedni preklapaju po polovini svoje dužine. Ključna je ušteda u srednjem članu. Po školskom postupku on iznosi $A_1B_0 \oplus A_0B_1$ i traži dva množenja. U izrazu (3.46) dobija se iz jednog novog proizvoda i dva koja su za spoljne članove ionako već izračunata. Umesto četiri množenja polovina potrebna su, dakle, tri, po cenu nekoliko dodatnih sabiranja.

Primenjen rekursivno, postupak svodi složenost sa $\mathcal{O}(n^2)$ na $\mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1,585})$.



Slika 3.28: Karatsubin algoritam

Za binarna polja ovaj algoritam je povoljniji nego za obične cele brojeve, i to iz dva razloga. Prvi je da dodatna sabiranja koja Karatsubin postupak unosi ovde nisu sabiranja sa prenosom, nego XOR operacije, jedan logički nivo, bez prostiranja prenosa duž reči. Drugi je da se time cela struktura svodi na čistu kombinacionu mrežu, čija dubina raste logaritamski sa n . Dubina rekurzije je pritom kompromis između broja osnovnih množenja i dubine mreže. Sistematska analiza data je u [32].

Redukcija. Nesvedeni proizvod stepena do 254 vraća se u polje smenom $x^{128} = x^7 + x^2 + x + 1$, koja sledi neposredno iz (3.41). Svaki postavljen bit iznad stepena 127 tako se preslikava u najviše četiri niža stepena. Pošto su svi članovi redukcionog polinoma osim vodećeg ispod x^8 , jedan prolaz spušta najviši stepen sa 254 na 133, a drugi na 12. Dva prolaza su, dakle, dovoljna, i oba su plitke mreže XOR kola. Malobrojni i niski članovi redukcionog polinoma upravo su ono što redukciju čini ovako plitkom, i po tome se ovakvi polinomi i biraju za kriptografska polja.

Konkretna dubina rekurzije, broj osnovnih množenja i uticaj na kritičnu putanju u realizaciji ovog rada dati su u odeljku 4.4.3.

3.3.9.4 Dešifrovanje i provera oznake

Pri dešifrovanju primalac najpre istim postupkom, nad primljenim šifratom i pridruženim podacima, izračunava očekivanu oznaku ICV' , pa je poredi sa primljenom oznakom ICV . Poruka se prihvata samo ako je $ICV' = ICV$, a dešifrovani sadržaj ne sme biti prosleđen dalje, ni primaocu ni bilo kom potrošaču nizvodno, pre nego što provera uspe. Realizacije koje poruku obrađuju u prolazu, ne zadržavajući ceo njen sadržaj, taj zahtev ispunjavaju tako što uz dešifrovani tok isporuče i ishod provere na kraju poruke, pa obaveza odbacivanja prelazi na prvog potrošača – tako postupa i realizacija iz poglavlja 4. Sam šifrat se dešifruje identičnim CTR postupkom kao pri šifrovanju, jer operacija XOR sa istim pseudoslučajnim nizom poništava svoje

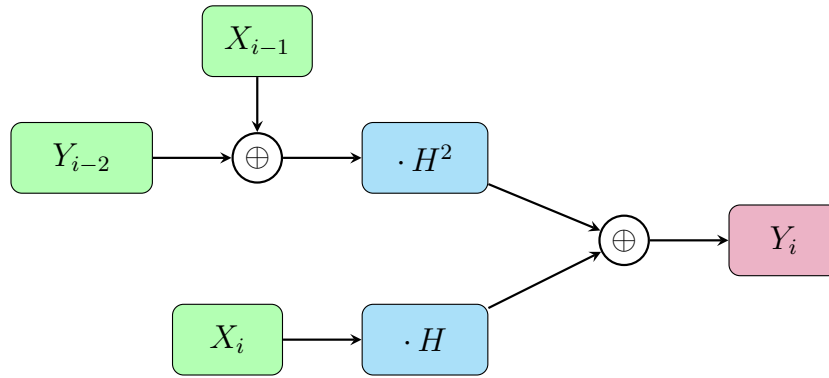
dejstvo. Otuda i posledica najavljena u odeljku 3.3.7: i pri dešifrovanju se šifra poziva samo u smeru šifrovanja, a razlika postoji jedino u završnom koraku, koji oznaku u jednom slučaju generiše, a u drugom proverava.

3.3.9.5 Paralelizacija GHASH funkcije

Za razliku od CTR grane, koja je savršeno paralelna, Hornerova šema (3.43) nameće striktno sekvencijalnu obradu. Svaki blok zahteva jedno množenje sa H pre prihvatanja narednog, pa jedan množač koji množenje izvršava u c taktova ograničava propusnost na jedan blok svakih c taktova. Ovo ograničenje, međutim, pripada načinu izračunavanja, a ne samoj funkciji. Polinomijalni oblik (3.42) nema nikakvu povratnu spregu, pa se grupisanjem po p uzastopnih blokova, uz unapred izračunate stepene $H^2, H^3, \dots, H^p$, dobija:

$$Y_i = (Y_{i-p} \oplus X_{i-p+1}) \cdot H^p \oplus X_{i-p+2} \cdot H^{p-1} \oplus \dots \oplus X_i \cdot H, \quad (3.47)$$

pa p paralelnih množača obrađuje p blokova istovremeno, umnožavajući propusnost autentifikacije p puta. Slika 3.29 prikazuje strukturu za $p = 2$. Dva množača rade paralelno, jedan sa stepenom H^2 nad akumulatorom i neparnim blokom, drugi sa H nad parnim blokom, a njihovi rezultati se sabiraju u novi akumulator.



Slika 3.29: Paralelizovani GHASH za $p = 2$

Broj množača i latencija množenja izbor su realizacije. Kompromis za sistem iz ovog rada kvantifikovan je u odeljku 4.4.3.2.

3.3.10 Sigurnosna svojstva i ograničenja

GCM režim nasleđuje snagu AES algoritma, ali njegova sigurnost kritično zavisi od pravilne upotrebe nonce vrednosti, jer se ista vrednost IV nikada ne sme upotrebiti dva puta sa istim ključem. Ponavljanje nonce vrednosti u CTR režimu dovodi do ponavljanja pseudoslučajnog niza, čime Oskar dobija XOR dva otvorena teksta. Kod GCM-a posledice su još teže, jer razlika dve oznake dobijene istim brojačkim blokom daje jednačinu po potključu H , iz koje se dobija svega nekoliko kandidata, a u praktičnim slučajevima i sam H . Time postaje moguće falsifikovanje oznaka za proizvoljne poruke [33]. Zbog toga je generisanje nonce vrednosti, tipično brojačem ili kombinacijom fiksnog polja i brojača poruka, sastavni deo pravilne upotrebe režima i mora biti pažljivo rešeno i na nivou hardverskog interfejsa akceleratora.

Pored jedinstvenosti nonce vrednosti, standard [9] ograničava dužinu poruke na najviše $2^{39} - 256$ bitova, propisuje minimalne preporučene dužine oznake u zavisnosti

od primene i ograničava ukupan broj poziva sa istim ključem. U sistemu realizovanom u ovom radu jedan ključ sesije štiti sav saobraćaj svoje sesije. Jedinstvenost nonce vrednosti obezbeđuje deterministička konstrukcija sa brojačem poruka, koja ujedno prostorom brojača ograničava broj poziva sa istim ključem, a sveža ECDH razmena sa izvođenjem novih ključeva dostupna je u svakom trenutku kada sesiju treba obnoviti. Navedena ograničenja time se poštuju konstrukcijom protokola.

Poglavlje 4

Hardverska realizacija

Prethodno poglavlje je algoritme razložilo do operacija i pokazalo koje od njih određuju cenu u hardveru. Ovo poglavlje te operacije pretvara u kolo: tri akceleratora jezgra i način na koji se ona uklapaju u sistem. Izlaganje počinje principima koji su zajednički za sva tri jezgra (interfejsi, upravljanje tokom podataka, način konfiguracije), a zatim se svako jezgro obrađuje pojedinačno. Sve arhitekture realizovane su u jeziku VHDL, alatom AMD Vivado, i verifikovane postupcima opisanim u poglavlju 5.

4.1 Zajednička arhitektura sistema

4.1.1 Ciljna arhitektura i obim rada

U radu su realizovana tri akceleratora jezgra: ECDH, SHA-3 i AES-GCM. Akcelerator iz naslova čini upravo taj skup, a zajednički imenitelj mu je interfejsni ugovor, po kome sva tri jezgra podatke primaju i predaju AXI4-Stream protokolom, a konfigurišu se generičkim parametrima pri sintezi. Nameravana upotreba je hibridni kriptosistem iz odeljka 2.4, u kome jezgra redom daju tajnu Z , ključ sesije K i zaštićen saobraćaj.

Pritom nijedno jezgro ne zavisi od preostala dva. Taj lanac jedna je od njihovih upotreba, a ne uslov njihovog rada, i kako se jezgra u njega povezuju opisano je u odeljku 4.5. Zato su i proverena pojedinačno, svako u okruženju koje odgovara njegovoj nameni, a zajednički rad lanca proverava se simulacijom nad povezanim jezgrima. Pojediniosti po jezgru date su u zaključnom pododeljku njegovog odeljka.

Za arhitekturu jezgara presudno je što dve ravni tog kriptosistema imaju bitno različite vremenske zahteve. *Upravljačka ravan*, u kojoj se uspostavljaju ključevi, izvršava se jednom po sesiji, vodi je softver i njeno trajanje nije vezano rokom. *Putanja podataka* je suprotan slučaj. U celini je smeštena u programabilnoj logici, obrađuje blok po bloku bez posredovanja softvera, i latencija joj je deterministička, unapred poznata iz arhitekture jezgra. Zato SHA-3 i AES-GCM jezgra obrađuju neprekidan tok podataka i mere se propusnošću, dok ECDH izvršava jednu dugu operaciju po pozivu i vrednuje se trajanjem te operacije, dakle brojem razmena ključa koje u jedinici vremena može da opsluži.

4.1.2 AXI4-Stream interfejs

SHA-3 i AES-GCM akcelerator obrađuju podatke kao kontinualan tok. Poruka ulazi, a izlazi transformisana poruka, sažetak ili oznaka. Za takvu obradu prirodan je

streaming interfejs, pa sva jezgra komuniciraju preko AXI4-Stream protokola [34], industrijskog standarda za jednosmerni prenos toka podataka unutar sistema na čipu. Osnovni signali interfejsa dati su u tabeli 4.1.

Tabela 4.1: Osnovni signali AXI4-Stream interfejsa

Signal	Smer	Uloga
tdata	proizvođač → potrošač	reč podataka širine DATA_WIDTH
tvalid	proizvođač → potrošač	na tdata je validna reč
tready	potrošač → proizvođač	potrošač je spreman da prihvati reč
tlast	proizvođač → potrošač	tekuća reč je poslednja u paketu (poruci)
tkeep	proizvođač → potrošač	maska validnih bajtova u reči

Prenos jedne reči (engl. *beat*) dešava se u taktu u kome su **tvalid** i **tready** istovremeno aktivni. Ova jednostavna konvencija nosi dve ključne osobine. Prvo, *protivpritisak* (engl. *backpressure*). Potrošač koji trenutno ne može da primi podatak jednostavno spusti **tready**, i proizvođač zadržava reč, tok se zaustavlja bez gubitka podataka i bez ikakve dodatne signalizacije. Drugo, granica poruke je deo protokola. Signal **tlast** označava poslednju reč paketa, pa jedan TLAST-ograničen paket predstavlja jednu poruku (kod SHA-3 jedan sažetak, kod AES-GCM jedan šifrovani paket sa oznakom), a **tkeep** na poslednjoj reči dozvoljava poruke čija dužina nije umnožak širine reči.

Isti valid/ready mehanizam korišćen je i unutar SHA-3 i AES-GCM jezgara, između njihovih stepena obrade. Time stepeni rade paralelno, a spregnuti su samo rukovanjem. Svaki stepen obrađuje svoj deo posla svojim tempom, i celina ostaje ispravna pri bilo kom rasporedu zastoja na ulazu ili izlazu. ECDH jezgro, sa jednom dugom operacijom po pozivu, unutrašnje celine sprega parom start i done signala. Konkretni mehanizam upravljanja tokom razlikuje se, međutim, od jezgra do jezgra. Tamo gde bi kombinaciona putanja **tready** signala postala kritična umeću se *skid* baferi, registarski stepeni koji registruju rukovanje u oba smera po cenu jedne reči skladišta (GCM sloj akceleratora, odeljak 4.4), dok se u delovima organizovanim po rundama tok vodi brojačima validnih podataka, po ugledu na iterativnu arhitekturu iz [27].

4.1.3 Konfiguracija i platforma

Sva jezgra opisana su na nivou prenosa među registrima (engl. *Register Transfer Level*, RTL) i konfigurišu se generičkim parametrima u vreme sinteze (varijanta algoritma, širina interfejsa, dubina protočnosti), a ne upisom u konfiguracione registre u radu. Ovakav izbor pojednostavljuje i hardver i bezbednosnu analizu, nepromenljiva konfiguracija ne može biti zloupotrebljena u radu, a fleksibilnost se zadržava tamo gde joj je mesto, u parametrizovanom RTL opisu iz koga se sintetiše željena varijanta. Ispravnost kombinacija parametara proverava se tvrdnjama pri elaboraciji, pa besmislene kombinacije (npr. širina reči koja ne deli veličinu bloka) obaraju sintezu, a ne rad sistema.

Ciljna platforma je razvojni sistem Kria KR260 sa Zynq UltraScale+ čipom, koji objedinjuje procesorski sistem (PS, ARM Cortex-A53 jezgra) i programabilnu logiku (PL). Akceleratori su smešteni u programabilnoj logici, a jedina veza sa okolinom su im AXI4-Stream tokovi iz prethodnog odeljka, pa je jezgru svejedno da li na drugom kraju toka stoji drugo jezgro, mrežni put ili procesorski sistem. Kako izgleda vezivanje jezgra za procesorski sistem pokazano je na primeru SHA-3 jezgra, uz merenja na ploči u odeljku 5.3.5.

4.2 ECDH akcelerator

ECDH akcelerator izračunava skalarno množenje kP na krivoj B-571, po Montgomeryjevim merdevinama sa Lópezovim i Dahabovim formulama iz odeljka 3.1.7. Sa okolinom je povezan AXI4-Stream tokovima, ali tok ne nosi sadržaj koji se obrađuje u prolazu, nego dostavi operande i preuzme rezultat. Kratak paket uđe, usledi računanje koje traje nekoliko redova veličine duže od samog prenosa, pa kratak paket izađe. Merilo zato nije propusnost toka, nego latencija (engl. *latency*), vreme od zadatih operanada do gotovog rezultata.

Sam izbor krive B-571 hardverski je motivisan. Aritmetika nad velikim binarnim poljem zahtevan je i zanimljiv hardverski problem, jer sabiranje bez prenosa i kvadriranje kao linearna operacija daju prostor za arhitekturna rešenja kojih kod prostih polja nema, a upravo je to predmet ovog rada. Realizaciju stoga treba posmatrati kao istraživački akcelerator i osnovu za poređenje arhitektura, a ne kao preporuku parametara za nov produkcionni sistem. Za takvu upotrebu bi se ista arhitektura zamenila krivom nad prostim poljem, uz bitno drugačiji množač.

Ceo akcelerator je jedno IP jezgro sa dva parametra koja menjaju arhitekturu, uz širinu magistrale i parametre krive zadate pri sintezi. Generik `G_D` zadaje koliko bitova operanda množač u polju obradi po taktu, o čemu je reč u odeljku 4.2.2. Generik `G_LOW_LATENCY` bira jedno od dva jezgra ugrađena u isti IP. *Osnovno* je-zgro (`ecd_core_basic`) svu aritmetiku obavlja jednom deljenom jedinicom i cilja najmanju površinu, a *paralelno* (`ecd_core_low_latency`) iteraciju merdevina izvršava na tri uporedna množača i cilja najmanju latenciju. Izbor menja samo raspored resursa, ne i rezultat računa, a sintetiše se samo izabrana grana.

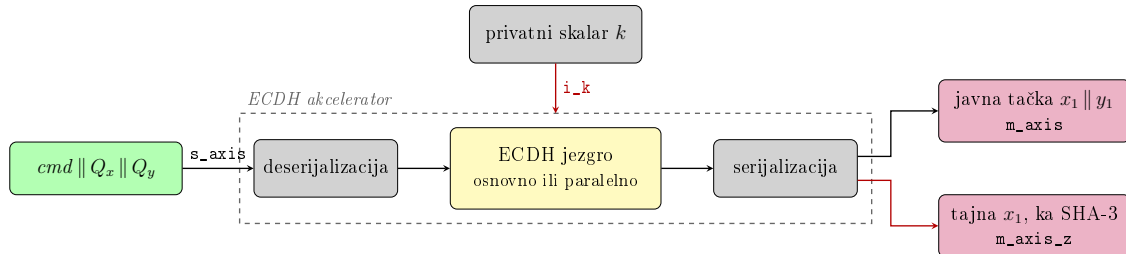
4.2.1 Organizacija akceleratora

Akcelerator je organizovan kao lanac tri stepena, prikazan na slici 4.1: *deserijalizacija* sklapa reči ulaznog paketa u operande pune širine polja, *jezgro* izvršava skalarno množenje, a *serijalizacija* rastavlja rezultat u reči i šalje ga na odgovarajući izlazni tok. Ulazni paket nosi komandu i koordinate tačke, $cmd \parallel Q_x \parallel Q_y$, a komanda razlikuje dve upotrebe istog računa. Prva je generisanje javnog ključa, kada je tačka u paketu generator krive, a rezultat cela javna tačka. Druga je izračunavanje zajedničke tajne, kada je tačka tuđ javni ključ, a od rezultata se zadržava samo x -koordinata, jer po protokolu iz odeljka 3.1.8 samo ona ulazi u izvod ključa.

Privatni skalar k ne putuje kroz tok podataka, nego ulazi bočnim kanalom sa sopstvenim strobom, i jednom upisan važi za obe komande. Skalar tako ne mora da prođe ni kroz procesorski sistem ni kroz magistralu kojom stižu operandi.

Izlazna toka su dva, razdvojena po poverljivosti. Javna tačka izlazi na `m_axis`, ka procesorskom sistemu i dalje ka mreži. Zajednička tajna izlazi na `m_axis_z`, namenjen izvođenju ključa KMAC funkcijom u SHA-3 jezgru, u lancu koji će biti

prikazan u odeljku 4.5. Neizabrani tok pritom ne proglašava nijednu reč validnom, pa nijedna reč tajne ne biva prihvaćena na javnom toku, niti javni rezultat na tajnom. Celim lancem upravlja jednostavna mašina stanja koja dopušta jednu operaciju u letu. U mirovanju primi paket, po potrebi sačeka skalar, pokrene jezgro, isprati slanje rezultata i tek se onda vrati u prijem, dok naredni paket čeka na magistrali, zadržan protivpritiskom. Ceo ovaj sloj ne zavisi od izbora jezgra, koji menja samo instancu u sredini lanca.



Slika 4.1: Tok podataka kroz ECDH akcelerator. Crvene strelice nose tajni materijal, a skalar se pamti u omotaču i jednom upisan važi za obe komande

Moduli sa slike, redom kojim podatak putuje:

Deserijalizacija (`ecdh_deserializer`) prihvata paket reč po reč i upisuje ih po rednom broju u komandu, Q_x i Q_y , od najniže reči ka najvišoj. Paket pogrešne dužine tiho odbacuje, pa do jezgra stiže samo potpun par operanada, a dok operacija traje ne prima ništa, pa sledeći paket čeka na magistrali.

Serijalizacija (`ecdh_serializer`) na pokretanje upiše u izlazni pomerački registar $x_1 || y_1$ za javni ključ, odnosno samo x_1 za zajedničku tajnu, pa šalje reč po reč, od najniže. Reč se pritom vodi na oba izlaza istovremeno, a upamćena komanda bira koji od njih reč proglašava validnom i koliko reči paket ima. Neizabrani izlaz drži spušten `tvalid` do kraja paketa.

Akcelerator je parametrizovan genericima iz tabele 4.2, a kriva je fiksirana pri sintezi, jer sistem cilja jednu krivu. Polinom polja pritom ulazi u samu strukturu kvadratora i redukcije, pa se logika gradi oko njega pri elaboraciji, dok koeficijent b ostaje ulaz jezgra kome je pri sintezi dovedena stalna vrednost.

Tabela 4.2: Generici ECDH akceleratora

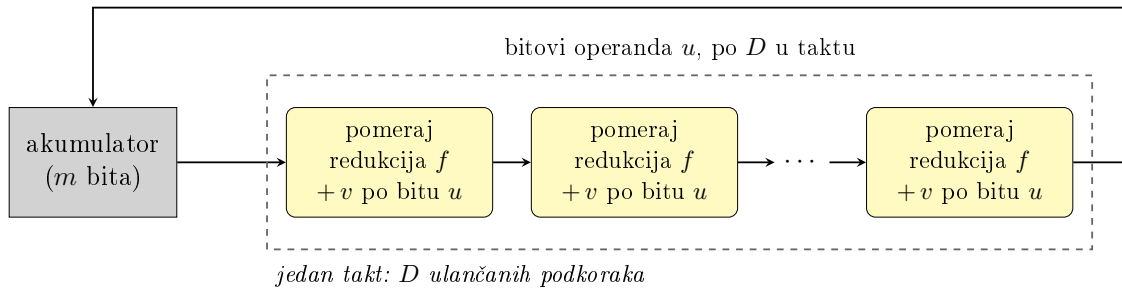
Generik	Vrednost	Uloga
G_M	571	širina polja m
G_D	1 do m (mereno do 128)	broj bitova operanda obrađenih u taktu (digit D), množenje traje $\lceil m/D \rceil$ taktova
G_F	pentanom krive B-571	ireducibilni polinom f , ugrađen u redukciju
G_B	koeficijent b (dodatak C)	parametar krive u udvostručenju
DATA_WIDTH	umnožak osam bitova	širina AXI4-Stream reči
G_LOW_LATENCY	false / true	osnovno ili paralelno jezgro

4.2.2 Operacije u polju

Cela aritmetika nad $GF(2^{571})$ svodi se na četiri operacije iz odeljka 3.1.3, a njihova cena u hardveru je izrazito nejednaka. Sabiranje (`gf_add`) je XOR po bitovima iz odeljka 3.1.3, pa staje u jedan nivo LUT logike bez obzira na širinu polja. Ni

kvadriranje (`gf_sqr`) nije mnogo skuplje, jer je linearno po istom odeljku. Pentanom krive ima svega tri srednja člana niskog stepena, pa je svaki izlazni bit XOR najviše četiri ulazna, jer redukcija svaku poziciju iznad m spusti na nekoliko nižih, a lanci spuštanja se za ovaj pentanom ne gomilaju. Celo kvadriranje ostaje kombinaciono i sam račun stane u jedan takt, a poziv aritmetičke jedinice oko njega, sa predajom operanada i preuzimanjem rezultata, košta četiri. Skupe operacije su množenje i inverzija, a inverzija se, po Itoh–Tsuijiijevoj metodi iz odeljka 3.1.5, i sama svodi na kvadriranja i množenja. Cena množenja tako određuje cenu svega.

Množać `gf_mul` računa $u \cdot v \bmod f$ Hornerovom šemom, od najvišeg bita. U svakom podkoraku akumulator se pomeri za jedno mesto, redukuje polinomom f ako je vodeći bit ispao, pa mu se doda v ako je tekući bit operanda u jedinica. Redukcija je tako umetnuta u svaki podkorak, pa akumulator nikada ne prelazi širinu polja. Generik `G_D` zadaje koliko se ovakvih podkoraka ulanča u jedan takt, kako prikazuje slika 4.2. Množenje zato traje $q = \lceil m/D \rceil$ taktova, uvek isto i nezavisno od vrednosti operanada.



Slika 4.2: Digit-serijski množać polja `gf_mul`

Veće D smanjuje broj taktova množenja, a cena je logika, jer svaki dodatni bit obrađen u taktu ulančava još jedan parcijalni proizvod sa redukcijom. Sa dužim kombinacionim lancem očekivano opada i najviša učestanost, pa je izbor generika `G_D` kompromis između broja taktova i periode takta. Paralelno jezgro, druga od dve grane generika `G_LOW_LATENCY`, do kraće latencije ne dolazi većim `G_D`, nego paralelizmom na nivou iteracije. Množać ostaje isti, a iteracija merdevina koristi tri uporedo, kako je pokazano u odeljku 4.2.4.

Množać u $GF(2^m)$ bira se po dve ose. Prva je raspored u vremenu. Bit-serijski množać ($D = 1$) troši m taktova uz najmanju površinu, digit-serijski tu ravnotežu pomera brojem bitova obrađenih u taktu, a potpuno paralelan izračuna proizvod u jednom taktu, po ceni logike koja raste sa kvadratom širine polja. Druga osa je sam algoritam množenja, gde školski raspored parcijalnih proizvoda nije jedini izbor. Alternativama je posvećen deo poglavlja 6. ECDH jezgro množi hiljade puta u jednoj razmeni ključa, ali se razmena dešava jednom po sesiji, pa je izabran digit-serijski množać sa školskim rasporedom parcijalnih proizvoda. Primer drugačijeg izbora po obe ose daje GHASH jedinica akceleratora AES-GCM, o kojoj će više reči biti u odeljku 4.4.3.

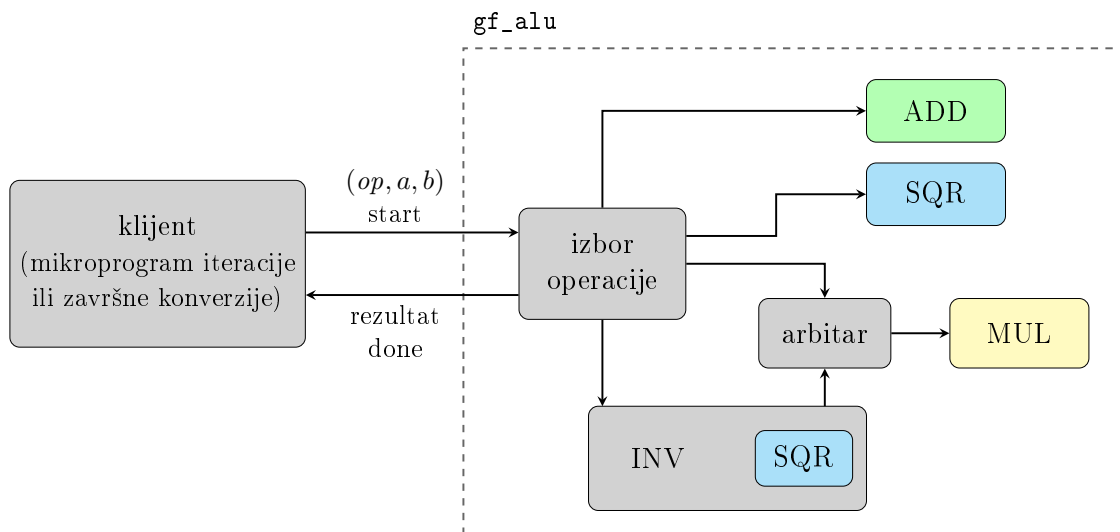
Inverzija je hardverski realizovana modulom `gf_inv`, koji neposredno sprovodi metodu iz odeljka 3.1.5. Mašina stanja prolazi kroz bitove eksponenta i za svaki izvede udvostručenje lanca, a po potrebi i produženje za jedan. Kvadrator modula, vezan u petlju sa registrom lanca, obavi po jedno od $m - 1 = 570$ kvadriranja u taktu, a svako od trinaest množenja iz izraza (3.8) košta jedno puno množenje od q taktova. Cela inverzija tako traje približno $570 + 13q$ taktova, ne računajući taktove

upravljačke logike, a koliko je to skupo najbolje se vidi kada se izrazi vremenom jednog množenja. Pri $D = 1$ množenje traje 571 takt, pa inverzija košta koliko četrnaest množenja. Pri $D = 128$ množenje traje svega pet taktova, pa ista inverzija vredi već sto dvadeset sedam množenja. Cilj realizacije je stoga da se inverzija izbegne gde god je moguće, što Lópezove i Dahabove koordinate iz odeljka 3.1.7 upravo i omogućavaju. Iteracija merdevina iz odeljka 4.2.4 zahvaljujući njima ne sadrži nijednu inverziju, pa jedina ostaje u završnoj konverziji, gde se i ona dodatno proređuje, o čemu je reč u odeljku 4.2.5.

4.2.3 Aritmetička jedinica

Četiri operacije iz prethodnog odeljka objedinjuje aritmetička jedinica `gf_alu`, koja po pozivu izvršava tačno jednu operaciju. Klijent postavi kod operacije i operande, pulsira start i po javljenom završetku preuzme rezultat. Jedinica pritom nema registarsku banku, međurezultate računa čuva klijent, a ona pamti samo operande tekućeg posla i poslednji rezultat. Takav izvršilac je jednostavan za upravljanje, a ceo račun nad tačkama svodi se na nizove mikroinstrukcija koje klijenti izdaju jedinici. Taj obrazac nosi ostatak ovog odeljka.

U jedinici postoji tačno jedan množač, koji dele neposredni put za množenje i inverter sa svojih trinaest poziva. Koji od dva potrošača ga trenutno drži odlučuje arbitar, prema stanju jedinice, kako prikazuje slika 4.3. Sabiranje i kvadriranje su kombinacioni i privatni, pa kod njih nema deljenja.



Slika 4.3: Aritmetička jedinica `gf_alu`: sabiranje (ADD), kvadriranje (SQR), množenje (MUL) i inverzija (INV) iza zajedničkog izbora operacije

4.2.4 Iteracija merdevina u hardveru

Jedna iteracija merdevina objedinjuje udvostručenje (`mdouble`, (3.10)) i diferencijalno sabiranje (`madd`, (3.11)). Iz tekućeg para (P_1, P_2) ona izračuna $2P_1$ i $P_1 + P_2$. Sabiranju je pritom potrebna i x -koordinata razlike dve tačke, po odeljku 3.1.7 uvek jednake baznoj tački P , pa se koristi njena unapred poznata afina koordinata. Po tabeli 3.1 iteracija staje šest množenja, pet kvadriranja i tri sabiranja, a kako se kvadriranja i sabiranja obave u po jednom taktu, dok je D mali praktično sve je u ceni množenja, a pri širokoj obradi glavninu preuzme režija poziva (odeljak 5.2.3).

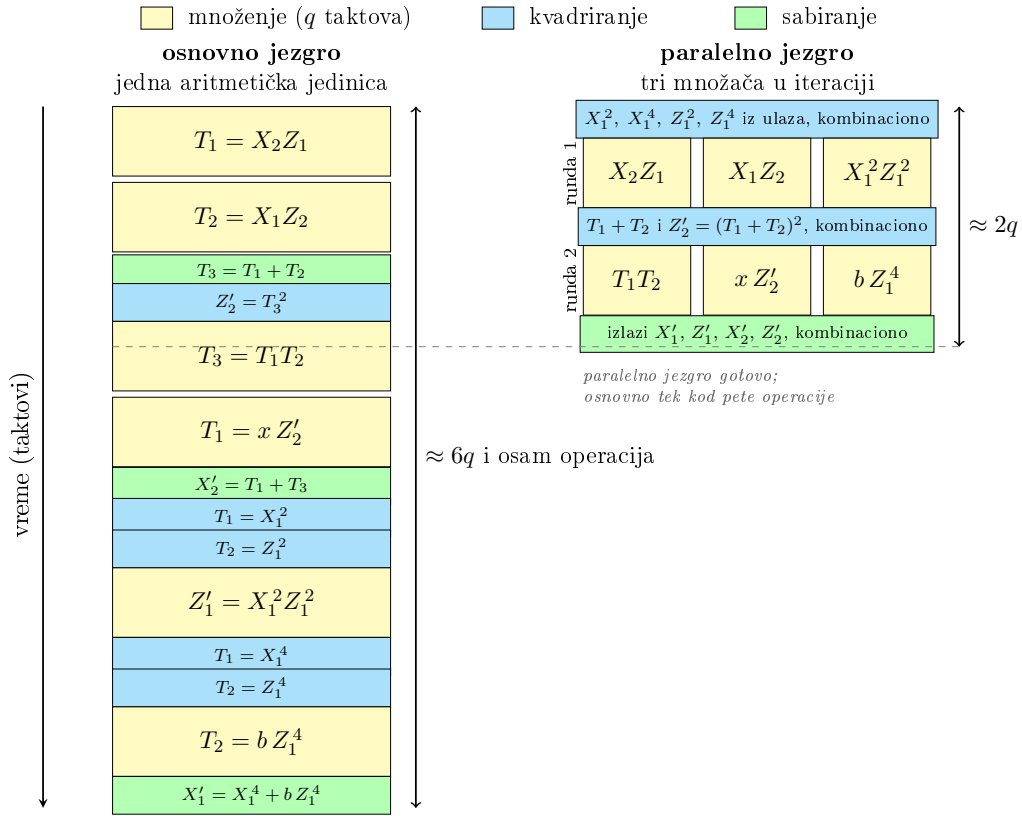
U osnovnom jezgru iteraciju izvršava modul `ec_step_mxy` kao mikroprogram od četrnaest operacija na jednoj aritmetičkoj jedinici, dat u tabeli 4.3. Brojač instrukcija prolazi kroz tabelu, za svaku operaciju izda mikroinstrukciju jedinici i njen rezultat upiše u odredišni registar te instrukcije. Šest množenja teče jedno za drugim, pa iteracija traje približno $6q$ taktova. Ostatak su kvadriranja, sabiranja i po koji takt poziva i preuzimanja rezultata po operaciji.

Tabela 4.3: Mikroprogram jedne iteracije merdevina u osnovnom jezgru

Br.	Operacija	Račun	Deo iteracije
1	MUL	$T_1 = X_2 Z_1$	madd
2	MUL	$T_2 = X_1 Z_2$	
3	ADD	$T_3 = T_1 + T_2$	
4	SQR	$Z'_2 = T_3^2$	
5	MUL	$T_3 = T_1 T_2$	
6	MUL	$T_1 = x Z'_2$	
7	ADD	$X'_2 = T_1 + T_3$	
8	SQR	$T_1 = X_1^2$	mdouble
9	SQR	$T_2 = Z_1^2$	
10	MUL	$Z'_1 = X_1^2 Z_1^2$	
11	SQR	$T_1 = X_1^4$	
12	SQR	$T_2 = Z_1^4$	
13	MUL	$T_2 = b Z_1^4$	
14	ADD	$X'_1 = X_1^4 + b Z_1^4$	

Paralelno jezgro istu iteraciju izvršava modulom `ec_point_step_par`, koji nije klijent aritmetičke jedinice, nego poseduje tri sopstvena množača, pet kvadratora i sabiranja u čistoj logici. Graf zavisnosti šest množenja ima dubinu dva, pa se ona rasporede u dve runde po tri. U prvoj rundi tri množenja zavise samo od ulaza iteracije (redovi 1, 2 i 10 tabele 4.3), u drugoj dva zavise od rezultata prve (redovi 5 i 6), a treće mesto u drugoj rundi popunjava množenje nezavisno od svega ostalog (red 13). Sva tri su tako stalno zauzeta, a kvadriranja i sabiranja, kombinaciona i jeftina po odeljku 4.2.2, ne troše nijedan takt, jer se obave u prolazu, između rundi.

Slika 4.4 poredi dva rasporeda u vremenu. Iteracija osnovnog jezgra traje približno $6q$, a paralelnog približno $2q$ taktova, pa je dobitak na množenjima trostruk. Ukupan odnos zavisi od širine obrade. Pri malom D ga završna konverzija, zajednička obema realizacijama, spušta neznatno ispod trostrukog, a sa većim D prevagne režija osnovnog jezgra, osam serijskih kvadriranja i sabiranja sa pozivom i preuzimanjem rezultata po svakoj operaciji, čega kod paralelnog nema, pa odnos raste iznad trostrukog (odeljak 5.2.3). Broj množača u iteraciji određuje sam graf zavisnosti. Ispod dve runde se ne može, jer druga runda zavisi od prve, a dve runde po tri množenja tačno popune tri množača, pa dodatni u iteraciji ne bi imao šta da radi.

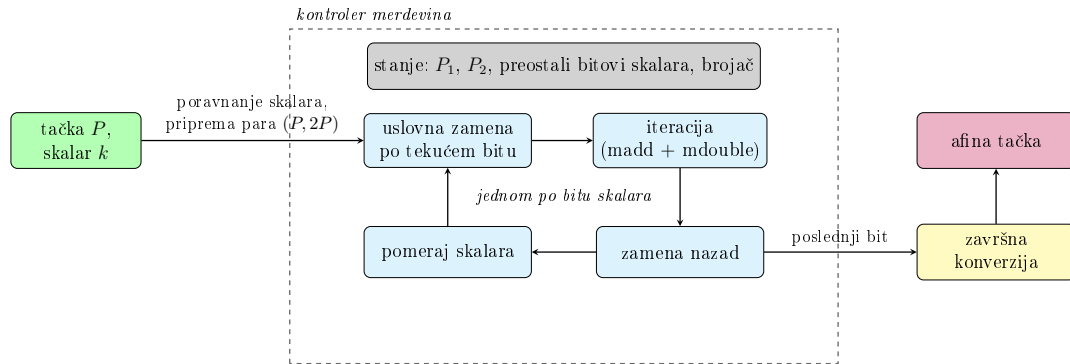


Slika 4.4: Ista iteracija merdevina u dva rasporeda resursa. Visina bloka odgovara trajanju u taktovima, uz $q = \lceil m/D \rceil$

4.2.5 Kontroler merdevina i završna konverzija

Iteracija iz prethodnog odeljka između dva poziva ništa ne pamti. Ona primi tekući par tačaka, izračuna novi i preda ga nazad. Sve što traje kroz celo skalarno množenje drži *kontroler merdevina*, a to su tekuće tačke P_1 i P_2 , preostali bitovi skalara i brojač iteracija.

Pre petlje kontroler poravna skalar tako što vodeće nule odbaci pomeranjem, bit po taktu. Merdevine zato kreću od najvišeg postavljenog bita, a broj iteracija odgovara stvarnoj dužini skalara, ne širini registra. Po svakom bitu kontroler zatim prolazi kroz isti redosled, prikazan na slici 4.5. Uslovnom zamenom uredi par prema vrednosti tekućeg bita, pozove iteraciju nad uređenim parom, vrati zamenu i pomeri skalar za jedno mesto. Ni za pripremu početnog para $(P, 2P)$ ne postoji posebna logika. Kontroler iteraciji prosledi par (P, P) i zadrži samo polovinu rezultata koja odgovara udvostručenju, pa i početni par pripremi isti hardver iteracije.



Slika 4.5: Petlja kontrolera merdevina. Prikaz je zajednički za oba jezgra, a razlika u pakovanju konverzije data je u odeljku 4.2.6

Uslovnu zamenu izvodi `ec_cswap`, četiri uporedna multipleksera dva-u-jedan, čisto kombinaciona. U softveru bi zamena po tajnom bitu bila grananje, curenje koje se tamo suzbija maskiranjem. U hardveru su oba ulaza multipleksera uvek fizički prisutna i vođena, bit samo bira koji prolazi, pa su tok podataka i trajanje identični za obe vrednosti. Konstantno trajanje po bitu, koje odeljak 3.1.7 traži od realizacije, ovde dakle ne košta ništa, jer je multiplekser već takav.

Po poslednjem bitu merdevine ostavljaju projektivno stanje, a afinu tačku vraća završna konverzija, kojoj trebaju tri inverzije. Dve dele X imeniocem Z za obe tačke para, a treća, inverz x -koordinate bazne tačke, ulazi u rekonstrukciju y -koordinate javne tačke, zatvorenim izrazom nad projektivnim stanjem i obema koordinatama bazne tačke, bez ponavljanja merdevina [22, 35]. Po ceni iz odeljka 4.2.2 svaka inverzija nosi 570 kvadriranja, pa se umesto tri nezavisne koristi Montgomeryjeva simultana inverzija. Dva množenja slože tri imenioca u jedan proizvod, invertuje se samo on, a još četiri množenja iz proizvoda vrata sva tri pojedinačna inverza. Zamena dve inverzije za šest množenja algebarsko je preuređenje, isto u oba jezgra. Ceo mikroprogram konverzije dat je u tabeli 4.4, a čini ga osamnaest operacija, od toga jedanaest množenja, pet sabiranja, jedno kvadriranje i jedna jedina inverzija.

Tabela 4.4: Mikroprogram završne konverzije: x i y su koordinate bazne tačke, x_1 , x_2 i y_1 affine koordinate rezultata, a s_i pomoćne vrednosti

Br.	Operacija	Račun	Deo konverzije
1	MUL	$p = Z_1 Z_2$	simultana inverzija
2	MUL	$u = p x$	
3	INV	$t = u^{-1}$	
4	MUL	$x^{-1} = t p$	
5	MUL	$t = t x$	
6	MUL	$Z_2^{-1} = t Z_1$	
7	MUL	$Z_1^{-1} = t Z_2$	
8	MUL	$x_1 = X_1 Z_1^{-1}$	afine koordinate
9	MUL	$x_2 = X_2 Z_2^{-1}$	
10	ADD	$s_1 = x_1 + x$	rekonstrukcija y
11	ADD	$s_2 = x_2 + x$	
12	MUL	$s_3 = s_1 s_2$	
13	SQR	$s_4 = x^2$	
14	ADD	$s_3 = s_3 + s_4$	
15	ADD	$s_3 = s_3 + y$	
16	MUL	$s_3 = s_1 s_3$	
17	MUL	$s_3 = s_3 x^{-1}$	
18	ADD	$y_1 = s_3 + y$	

4.2.6 Osnovno i paralelno jezgro

Dva jezgra se razlikuju samo po pakovanju opisanih delova. U osnovnom jezgru (`ecdh_core_basic`) kontroler je deo samog jezgra, a iteracija i završna konverzija dele modul `ec_step_mxy`, njegove registre i mašinu stanja, razdvojene jednim bitom režima. Sve mikroistrukcije idu na jednu deljenu aritmetičku jedinicu kojoj je taj modul jedini klijent.

U paralelnom jezgru (`ecdh_core_low_latency`) merdevine su samostalan modul `ec_scalar_mult_par`, koji iteraciju poverava modulu `ec_point_step_par` sa tri sopstvena množača, a završnu konverziju vodi zaseban modul `ec_mxy_batch`. Aritmetička jedinica, koju merdevine više ne koriste, ostaje dodeljena samo konverziji. Paralelno jezgro tako u zbiru nosi četiri množača, tri u iteraciji i jedan u aritmetičkoj jedinici, s tim što četvrti radi samo na samom kraju, tokom završne konverzije. Deljenje tri množača merdevina sa završnom konverzijom, koje bi četvrti učinilo nepotrebnim, razmatra se u poglavlju 6.

4.2.7 Uklapanje u sistem i provera

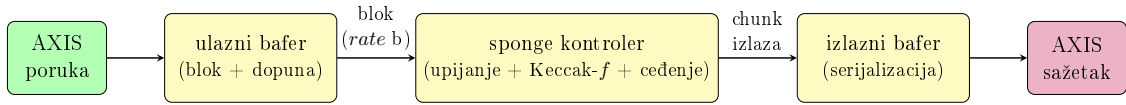
Jezgro se poziva dvaput po razmeni ključa, za javnu tačku i za zajedničku tajnu, pa u sistemu stoji kao koprocesor. Pozivalac dostavi tačku paketom, skalar bočnim portom, i preuzme javnu tačku ili x -koordinatu zajedničke tajne. Takva postavka odgovara poslu upravljačke ravni iz odeljka 4.1.1, jednoj dugoj operaciji po pozivu.

Jezgro je provereno na tri nivoa, od vektorskih testova sloja polja do celog akceleratora kroz AXI4-Stream omotač, pri čemu obe konfiguracije prolaze iste vektore sa identičnim rezultatom. Postupci i svi rezultati dati su u odeljku 5.2.

4.3 SHA-3 akcelerator

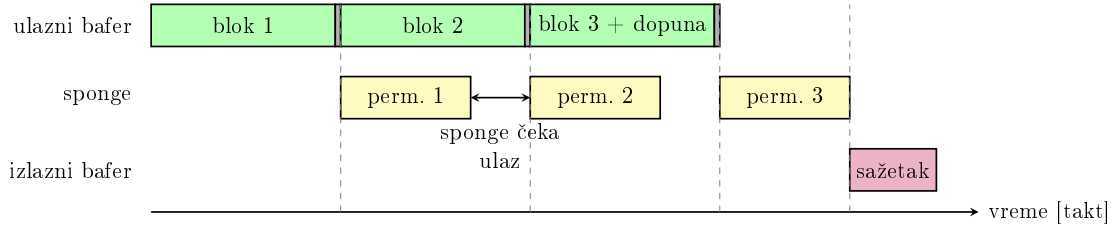
4.3.1 Arhitektura jezgra

SHA-3 akcelerator organizovan je kao lanac tri stepena, prikazan na slici 4.6. *Ulazni bafer* sklapa AXI4-Stream reči u blokove veličine *rate*, kako se brzina *r* iz odeljka 3.2.2 zove u RTL-u, i primenjuje dopunu. *Sponge kontroler* upija blokove u 1600-bitno stanje, izvršava permutaciju Keccak- $f[1600]$ i cedi izlaz. *Izlazni bafer* serijalizuje iscedeni izlaz nazad u AXI4-Stream reči. Stepni su spregnuti isključivo valid/ready predajama (blok, odnosno *chunk* izlaza), pa svaki napreduje čim ima spreman ulaz i mesto za rezultat. Njihov rad se zato preklapa, pa dok sponge kontroler permutuje tekući blok, ulazni bafer već sklapa naredni. To preklapanje određuje propusnost jezgra, a formalno ga razmatra model uskog grla u odeljku 5.3.3.



Slika 4.6: Tri stepena SHA-3 akceleratora

Vremenski dijagram na slici 4.7 to ilustruje na primeru poruke od tri bloka, u konfiguraciji kod koje punjenje bloka traje duže od same permutacije. Permutacija se u celini sakriva iza punjenja narednog bloka, pa propusnost određuje ulaz, a sponge deo vremena čeka. Koja od dve veličine dominira zavisi od odnosa broja ulaznih reči po bloku ($rate/DATA_WIDTH$) i trajanja permutacije u taktovima.



Slika 4.7: Preklapanje rada stepena za SHA3-256 uz $DATA_WIDTH = 32$, gde 34 ulazne reči i takt predaje bloka (uski tamni segment) stoje prema 24 takta permutacije

Jezgro je u potpunosti parametrizovano genericima, datim u tabeli 4.5. Jedan isti RTL opis daje svih šest funkcija standarda i cSHAKE konfiguraciju, kako svojstvo iz odeljka 3.2.6 i obećava. Izbor varijante svodi se na raspodelu stanja sponge konstrukcije ($rate = 1600 - 2d$ za SHA3- d , a 1344, odnosno 1088, za SHAKE128/256) i bajt dopune. Širina interfejsa $DATA_WIDTH$ mora deliti i $rate$ i veličinu izlaza, kombinacije koje to ne ispunjavaju odbijaju se pri elaboraciji.

Tabela 4.5: Generici SHA-3 akceleratora

Generik	Vrednosti	Uloga
ALGORITHM	"SHA3" / "SHAKE" / "CSHAKE"	familija funkcija i bajt dopune (06/1F/04)
SHA3_VERSION	"224" / "256" / "384" / "512"	dužina sažetka d , odakle je $rate = 1600 - 2d$
SHAKE_VERSION	"128" / "256"	$rate = 1344 / 1088$
SHAKE_BITS	umnožak DATA_WIDTH	ukupna dužina XOF izlaza
DATA_WIDTH	8 / 16 / 32 / 64	širina AXI4-Stream reči
ROUNDS_PER_CYCLE	delilac broja 24	broj rundi permutacije po taktu

4.3.2 Ulazni bafer i dopuna

Ulazni bafer akumulira pristigle reči u registar bloka od $rate$ bitova i vodi računa o dopuni poruke. Modul je realizovan mašinom stanja sa četiri stanja: **S_INIT** (početno stanje), **S_FILL** (punjenje bloka), **S_SEND** (predaja bloka sponge kontroleru) i **S_PAD_BLOCK** (generisanje bloka koji sadrži samo dopunu). Ulaz `i_config` je signal omogućavanja jezgra. U trenutnoj realizaciji, gde je varijanta algoritma fiksirana genericima pri sintezi, u AXIS omotaču je vezan na jedinicu, pa jezgro napušta **S_INIT** odmah po resetu. Kada stigne reč sa aktivnim `tlast`, bafer u jednom taktu iznova sastavi reč u kojoj se poruka završava, sa nulama na mestima bajtova koje `tkeep` maskira i bajtom dopune (06 za SHA3, 1F za SHAKE, 04 za cSHAKE) iza poslednjeg važećeg bajta, a kada je poslednja reč poruke puna, bajt dopune otvara narednu reč bloka. Završnu jedinicu šeme bafer postavi zasebnim upisom najvišeg bita bloka, što je tačno dopuna (3.13) iz odeljka 3.2.3, sa bitovima razdvajanja domena. Dva granična slučaja zaslužuju pažnju:

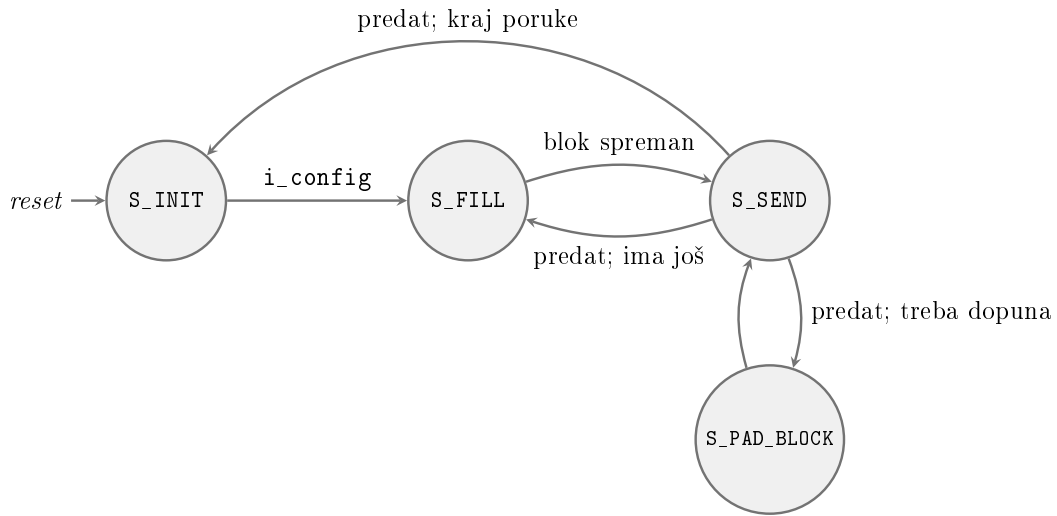
- ako dopuna pada na poslednji bajt bloka, bajt dopune i završni bit se stapaju u jedan bajt (86, 9F, odnosno 84), kako standard i propisuje;
- ako se poruka završi tačno na granici bloka, dopuna nema gde da stane, pa se generiše poseban blok koji sadrži samo dopunu (**S_PAD_BLOCK**).

Od organizacije registra bloka neposredno je zavisilo zauzeće ulaznog bafera. Blok se ne drži kao jedan vektor od $rate$ bitova, nego kao niz reči širine `DATA_WIDTH`, a svaki upis, bilo da nosi podatak ili dopunu, prolazi kroz jedan jedini upisni port, adresiran rednim brojem reči. Dopuna zauzima jedan dodatni takt, u kome bafer ne prima novu reč. Kroz isti port se tada prepravi tačno jedna reč bloka, a završna jedinica šeme postavlja se zasebnim upisom najvišeg bita bloka. Ovakva organizacija izabrana je zato što se pozicija upisa ne zna unapred, jer zavisi od toga koliko je reči već primljeno i koliko je bajtova poslednje reči maskirano, pa dopuna može početi na bilo kom bajtu bloka.

Opiše li se takav upis nad jednim širokim vektorom, pozicija se računa u bitovima, izrazom oblika $w \cdot \text{DATA_WIDTH} - m \cdot 8$, koji nije poravnat na granicu reči, pa dekodovanje mora biti u stanju da pogodi bilo koji bajt bloka. Uz to je i širina isečka kojim se brišu maskirani bajtovi promenljiva, što traži generator maske preko celog bloka. Dekoder i izbor po bajtu tada obuhvataju svih $rate/8$ bajtova odjednom, pa je takav opis samo u ulaznom baferu koštao preko 4600 LUT-ova.

Adresiranje po rečima isti posao svodi na dekodeer 1 od $rate/DATA_WIDTH$, a izbor između bajta poruke, bajta dopune i nule, koji i dalje zavisi od `tkeep` signala, ostaje zatvoren unutar jedne reči i svodi se na multiplekser po bajtu. Bajt dopune se pri tome ne upisuje u već sklopljeni blok, nego se cela reč iznova sastavi iz zasebnog registra koji čuva poslednju primljenu reč, čime se izbegava povratna sprega sa registra bloka na samog sebe. Opseg posla time je sveden sa celog bloka na jednu reč, a zauzeće ulaznog bafera na 865 LUT-ova.

U gustom saobraćaju upravljanje tokom ostaje bezbedno. U taktu u kome se u već pun blok upisuje dopuna, `tready` je spušten, pa ni paketi koji stižu bez razmaka (novi paket odmah po `tlast` prethodnog) ne mogu pregaziti blok u pripremi, svojstvo dokazano namenskim testom. Kompletan mašina stanja ulaznog bafera prikazana je na slici 4.8.

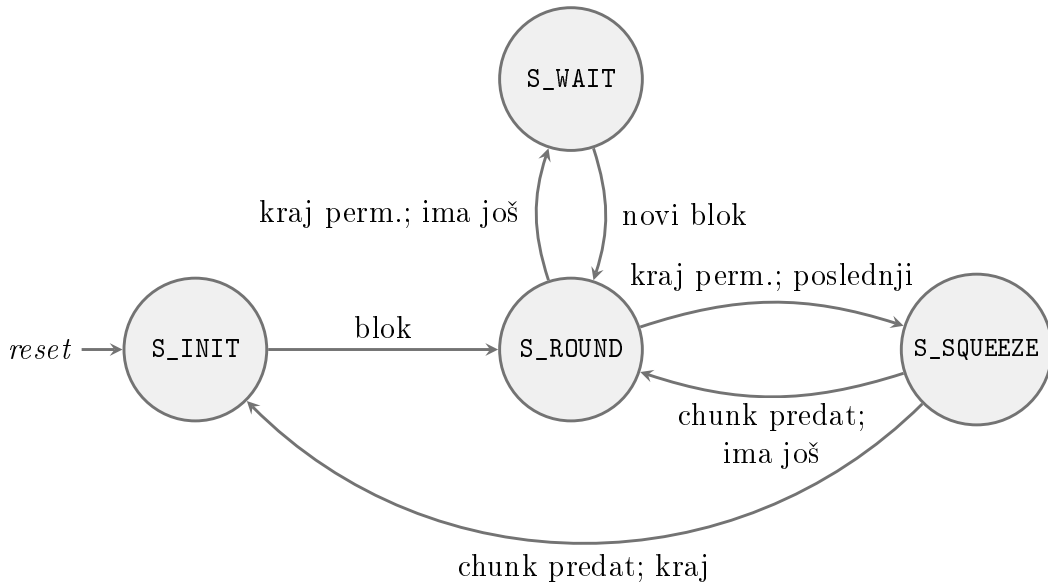


Slika 4.8: Mašina stanja ulaznog bafera SHA-3 akceleratora

4.3.3 Sponge kontroler i Keccak jezgro

Srce akceleratora je sponge kontroler. Stanje od 1600 bitova drži se u registru. Pri prihvatanju bloka prvih $rate$ bitova stanja sabira se sa blokom (upijanje), a zatim mašina stanja (`S_ROUND`) pokreće permutaciju. Tok podataka Keccak jezgra, za obe realizovane arhitekture, prikazan je na slici 4.10 u odeljku 4.3.4. Svih pet koraka jedne runde ($\theta, \rho, \pi, \chi, \iota$) realizovano je kao čiste funkcije u zajedničkom paketu (`keccak_pkg`), pa je runda kombinaciona mreža bez ijednog registra, registruje se samo stanje između taktova. Generik `ROUNDS_PER_CYCLE` određuje koliko se takvih mreža veže u lanac unutar jednog takta, pa stanje `S_ROUND` traje onoliko taktova koliko treba da se izvrše sve 24 runde. Kako taj izbor rađa dve arhitekture jezgra pokazuje odeljak 4.3.4. Konstante rundi uzimaju se iz tabele u paketu (dodatak D).

Kontrolerom upravlja mašina stanja prikazana na slici 4.9. U stanju `S_ROUND` teče permutacija, u `S_WAIT` kontroler čeka naredni blok poruke, a u `S_SQUEEZE` nudi iscedeni chunk izlaznom baferu. U stanju `S_INIT` prvi blok nove poruke upija se u eksplicitno nulovan registar, jer pri gustom saobraćaju, kada nova poruka stigne odmah po prethodnoj, u registru još stoji zatečena vrednost sa kraja prethodne poruke, a upijanje u nju umesto u nule iskvarilo bi sažetak. I ovaj granični slučaj pokriven je testovima gustog saobraćaja.



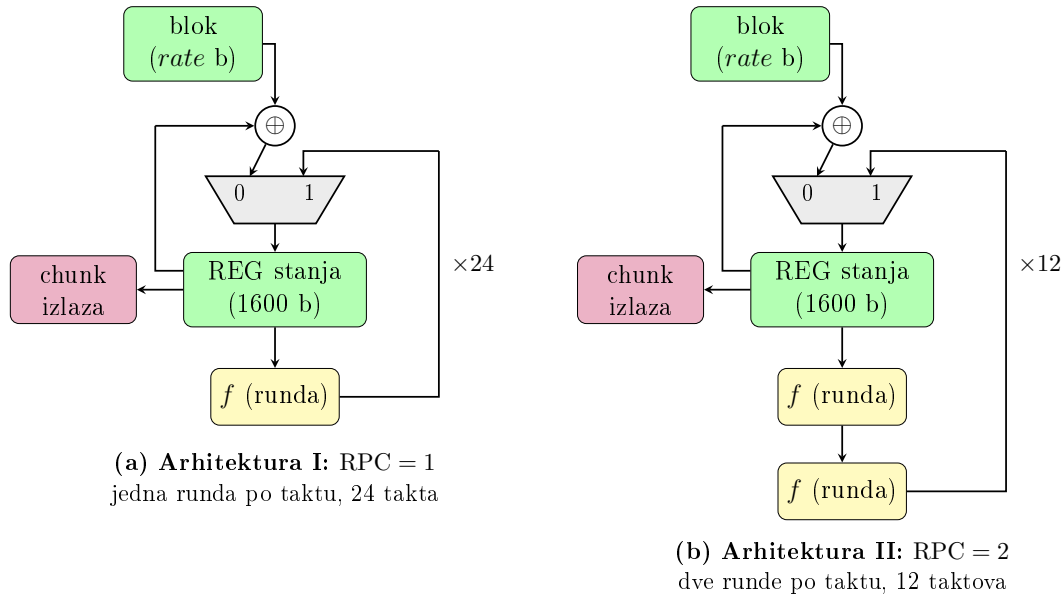
Slika 4.9: Mašina stanja sponge kontrolera

Za XOF funkcije, kod kojih izlaz premašuje *rate*, kontroler posle predaje svakog chunk-a ponovo permutuje stanje ($S_SQUEEZE \rightarrow S_ROUND$). Naredni chunk se pritom permutuje dok izlazni bafer još serijalizuje prethodni, pa dugi XOF izlazi ne unose prazan hod kad god chunk ima bar onoliko reči koliko permutacija traje taktova, resursi ne rastu sa dužinom izlaza, samo latencija (jedna permutacija po chunk-u).

4.3.4 Dve realizacije jezgra

Izbor mikroarhitekture jezgra zasnovan je na analizi prostora arhitektura SHA-3 algoritma koju su dali Sundal i Chaves [36]. Potpuno protočne (engl. *unrolled/pipelined*) arhitekture ostvaruju gotovo jedan sažetak po taktu tek kada se protočni registri popune, što donosi dobit samo kod mnoštva kratkih, međusobno nezavisnih poruka. Nijedna upotreba jezgra u ovom radu ne izgleda tako. Poruka se prima kao jedan tok, pa naredni blok ionako mora sačekati svih 24 runde nad prethodnim i protočnost ne pomaže, a pri izvođenju ključeva, ulozi koju jezgro ima u sistemu, obrađuje se jedna kratka poruka po sesiji. Suprotan pristup, *folded* arhitekture koje po taktu obrađuju samo deo stanja, smanjuje širinu puta podataka, ali usporava sistem približno četiri puta uz nesrazmerno malu uštedu površine zbog dodatne upravljačke logike [36]. Zato je odabrano iterativno jezgro nad celim stanjem. Svih 1600 bitova obrađuje se paralelno, a runde se izvršavaju jedna po taktu, ili dve. Generik `ROUNDS_PER_CYCLE`, broj rundi po taktu (dalje RPC), time postaje osnova dve zasebno sintetisane i ispitane arhitekture, prikazane na slici 4.10:

- *Arhitektura I* ($RPC = 1$) – jedna kombinaciona runda u povratnoj petlji. Permutacija traje 24 takta, ali je kritična putanja najkraća moguća, dubina jedne runde (θ do ι);
- *Arhitektura II* ($RPC = 2$) – dve runde vezane u lanac unutar istog takta. Permutacija traje 12 taktova, po cenu približno dvostruko duže kritične putanje i dvostruke količine kombinacione logike.



Slika 4.10: Tok podataka Keccak jezgra za obe arhitekture, prilagođeno prema [36]. Multiplexer bira između upijanja novog bloka (ulaz 0, blok sabran sa registrovanim stanjem, kod prvog bloka poruke nultim) i rezultata runde (ulaz 1). (a) jedna runda po taktu, (b) dve vezane runde po taktu

Arhitektura II na prvi pogled nudi dvostruku propusnost, ali kompromis ima dva ograničenja. Prvo, duža kritična putanja obara radnu učestanost, pa deo dobitka u taktovima poništi produženi period, a vrednosti generika iznad 2 iz istog razloga očekivano daju sve manji dobitak, jer putanja raste brže nego što broj taktova opada. Koliko učestanost zaista pada izmereno je u odeljku 5.3.2. Drugo, kod konfiguracija kod kojih punjenje bloka traje duže od permutacije (`DATA_WIDTH` znatno manji od *rate*), skraćanje permutacije ne menja ništa, jer jezgro ionako čeka ulaz. Zbog toga su obe arhitekture izmerene nad svih sedam kombinacija varijante i širine koje elaboracija dopušta, ukupno četrnaest konfiguracija (odeljak 5.3.2).

4.3.5 Izlazni bafer

Izlazni bafer prihvata chunk od sponge kontrolera i serijalizuje ga u `DATA_WIDTH` reči na master AXI4-Stream interfejsu, uz `tlast` na poslednjoj reči ukupnog izlaza. Bafer drži sopstvenu kopiju chunk-a, pa se iznošenje tekućeg preklapa sa računanjem narednog u kontroleru, a chunk koji stigne na vreme nadovezuje se bez praznog takta između reči. Svaka reč, uključujući poslednju, ostaje validna dok je `tready` ne prihvati, pa je izlaz bezbedan i pod proizvoljnim protivpritisom potrošača.

4.3.6 Podrška za cSHAKE i KMAC

Odeljak 3.2.7 pokazao je da je celokupno KMAC uokvirivanje samo formatiranje ulaznog niza, u kome se prefiksni blok sa imenom funkcije, blok ključa i kodirana dužina izlaza redom nadovezuju pre poruke.

Iz toga sledi da KMAC nema svoj režim u hardveru. Jezgro realizuje samo cSHAKE, a od funkcije SHAKE ga razlikuje jedino bajt dopune, koji konfiguracija `ALGORITHM = "CSHAKE"` postavlja na 04 umesto na 1F. Pošto je izbor utvrđen generikom, jezgro za prazne N i S ne prelazi u SHAKE, kako standard za taj slučaj propisuje, i tu se od njega razilazi. Sve ostalo stiže kao obična poruka, čiji sastav

jezgro ne proverava. Uokvirivanje je time ostavljeno strani koja podatke dovodi, na kojoj je i obaveza da režim koristi sa nepraznim prefiksnim blokom, kod KMAC-a uvek ispunjena, jer ime funkcije propisuje sam standard (odjeljak 3.2.7). U proveru iz ovog rada to je referentni model, dok bi u sistemu jednako poslužio upravljački program ili jednostavan upravljački blok ispred jezgra.

Podela je proverena namenskim testom sa poznatim odgovorima, nad KMAC256 i KMACXOF256 vektorima koje je proizveo referentni model, program u jeziku Python koji iste funkcije izračunava nezavisno od jezgra i sam je prethodno upoređen sa standardnom bibliotekom (odjeljak 5.1).

4.3.7 Uklapanje u sistem i provera

Jezgro je projektovano kao koprocesor iza kontrolera za neposredan pristup memoriji (engl. *Direct Memory Access*, DMA). Procesorski sistem preda poruku u memoriju, DMA je propusti kroz jezgro, a sažetak se vrati u memoriju. Takva postavka odgovara grupnom heširanju, kod koga je merilo propusnost nad dugom porukom, dok je za izvođenje ključeva merodavna latencija kratke poruke, izmerena u odeljku 5.3.3. Koja veličina postavlja granicu propusnosti za svaku konfiguraciju, i gde udvostručenje broja rundi po taktu ima smisla, pokazuje model uskog grla iz istog odeljka.

Jezgro pritom nije vezano za ulogu u lancu, pa jednako služi za opšte heširanje podataka i kao izvor pseudoslučajnog niza proizvoljne dužine.

Jezgro je provereno na tri nivoa, od vektora sa poznatim odgovorima do rada na ploči. Postupci i svi rezultati dati su u odeljku 5.3.

4.4 AES-GCM akcelerator

4.4.1 Organizacija akceleratora

AES-GCM akcelerator projektovan je kao umetnuta zaštita paketa. Paket uđe u jezgro i isti taj paket izađe, sa zaštićenim korisnim sadržajem i nepromenjenim zaglavljima. To ga razlikuje od SHA-3 jezgra, koje poruku potroši i vrati sažetak. Ovde ulaz i izlaz imaju istu strukturu, pa jezgro može stajati usred puta kojim paket ionako prolazi. Paket se posmatra kao niz segmenata, datih u tabeli 4.6.

Tabela 4.6: Segmenti paketa u AES-GCM akceleratoru

Segment	Uloga
BYPASS	Prenosi se nepromenjen, mimo kriptografskog puta (npr. zaglavlja nižih slojeva).
AAD	Autentifikuje se, ali se ne šifrjuje, pa ulazi u GHASH i izlazi nepromenjen.
PT / CT	Korisni sadržaj, koji se i šifrjuje i autentifikuje. Dužina se otkriva u radu, iz <code>tlast</code> .
ICV	Autentifikaciona oznaka od 16 bajtova. Dodaje se pri šifrovanju, proverava pri dešifrovanju.

Šifrovanje time preslikava `BYPASS || AAD || PT` u `BYPASS || AAD || CT || ICV`, a dešifrovanje obrnuto, uz dodatni izlazni signal koji govori da li je oznaka ispravna. Dužine segmenata `BYPASS` i `AAD` zadaju se generički, dok se dužina korisnog sadržaja ne zna unapred.

Akcelerator je izgrađen od tri sloja, koji odgovaraju i podeli na IP jezgra:

1. *Kriptografsko jezgro* (`AES_algorithm`) – sama šifra;
2. *GCM sloj* (`gcm_enc_glue`, `gcm_dec_glue`) – sve ostalo iz GCM režima: GHASH, blok sa dužinama, formiranje i provera oznake;
3. *Sistemska omotač* – razdvajanje i ponovno sastavljanje segmenata stvarnog paketa.

Granica između prvog i drugog sloja namerno je postavljena kao fiksna lista portova. Sloj ne zna koju šifru vodi, a jezgro ne zna kojim smerom sloj radi, jer je XOR sam sebi inverzan, pa ista instanca jezgra stoji i na putu šifrovanja i na putu dešifrovanja. GCM režim je pritom ugrađen u samo jezgro, koje formira brojački blok i računa potključ H i masku oznake. Ta granica je i osnova za dinamičku rekonfiguraciju, o čemu je reč u odeljku 4.4.6.

4.4.2 AES jezgro

4.4.2.1 Šta jezgro računa

Iz opisa GCM režima (odeljak 3.3.9) sledi da su jezgru potrebne tri vrste blokova:

1. $H = E_K(0^{128})$ – heš potključ za GHASH. Zavisi samo od ključa, pa se računa jednom po ključu i izlazi bočnim kanalom;
2. $E_K(J_0)$ – maska autentifikacione oznake. Zavisi od nonce vrednosti, pa se računa jednom po paketu, takođe bočnim kanalom;
3. $E_K(J_0 + 1), E_K(J_0 + 2), \dots$ – niz koji se sabira sa korisnim sadržajem, jedan blok po bloku podataka.

Sve tri su obična šifrovanja jednog bloka. Blok šifra se nikada ne invertuje, pa jezgro ne sadrži nijednu inverznu transformaciju, isti modul, bez ijedne izmene, koristi se i na putu šifrovanja i na putu dešifrovanja, što je već najavljeno u odeljku 3.3.7. Kroz samu šifru pritom prolaze isključivo ovi blokovi, dok se korisni sadržaj sa nizom sabira tek na izlazu omotača i u jezgro nikada ne ulazi.

Brojački blok se formira u hardveru. Iz 96-bitnog ulaza `i_nonce` gradi se $J_0 = \text{nonce} \parallel 0^{31} \parallel 1$, a dalji blokovi inkrementiranjem donjih 32 bita, dok gornjih 96 ostaje zaključano. Onaj ko jezgro pokreće bira, dakle, samo nonce, a ne i sam brojač, pa se brojački blokovi unutar paketa ne mogu ni ponoviti ni preskočiti. Ponavljanje niza između paketa svodi se time na ponavljanje nonce vrednosti, o čijoj je dodeli reč u odeljku 3.3.10. Ovakav sastav nonce vrednosti poklapa se sa načinom na koji je definišu IPsec [37] i MACsec, kod kojih jedan deo nonce vrednosti ostaje isti tokom čitavog trajanja veze, a ostatak je brojač paketa.

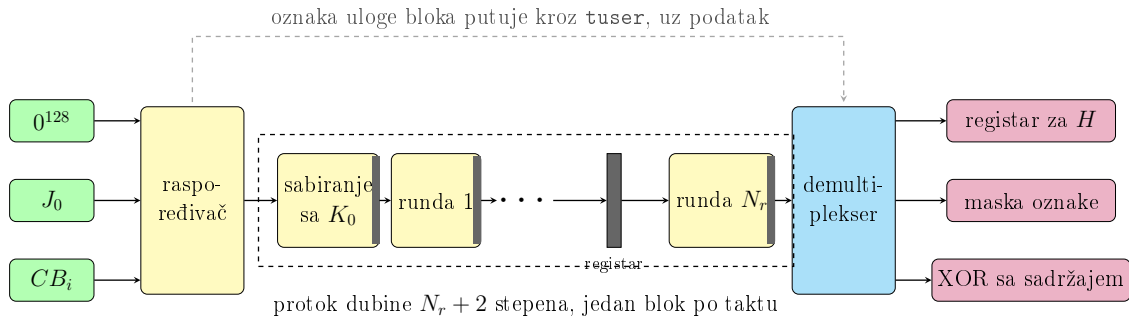
4.4.2.2 Dve arhitekture jezgra

Kao i kod SHA-3 jezgra, ista funkcionalnost realizovana je u dve arhitekture, koje se biraju generikom `WRAPPER_KIND`.

Protočna arhitektura (`UNROLLED`) sadrži jedno potpuno protočno AES jezgro dubine $N_r + 2$ stepena, koje u ustaljenom stanju daje jedan blok po taktu.

Rundi ima N_r , a stepena dva više. Prvi dodatni je početno sabiranje sa ključem, registrovano kao i svaki drugi stepen. Drugi je stepen koji se sastoji samo od registra, umetnut neposredno pred poslednju rundu. On postoji zbog spoja koji se javlja samo na tom mestu. Poslednja runda nema MixColumns, pa se jedina uvek realizuje kombinacionom logikom (odjeljak 4.4.2.5), a kod BRAM stila ispred nje stoji izlaz memorije sa T-tabelama, osetno sporiji od izlaza običnog registra. Bez ovog stepena izlaz memorije, XOR sloj runde i cela logika poslednje runde našli bi se na istoj putanji, koja ne ispunjava vremenske uslove. Registar je seče na dva kratka takta. Kod LUT stila takve putanje nema, pa je tamo ovaj stepen samo cena uniformnosti, 128 flip-flova i jedan takt latencije, kojom dubina protoka i upravljačka logika ostaju iste za oba stila.

Kroz taj isti protok prolaze tri vrste blokova, i to bez ikakvog razdvajanja u faze. Raspoređivač (engl. *dispatcher*) ih propušta jedan za drugim, a svakom pridružuje dvobitnu oznaku uloge koja putuje uz podatak, u korisničkom signalu `tuser` AXI4-Stream interfejsa. Na izlazu demultiplekser po njoj usmerava blok, u registar za H , u registar za masku oznake, ili u XOR sa korisnim sadržajem, dok je protok samo prenosi, ne ispitujući je. Postupak je prikazan na slici 4.11.



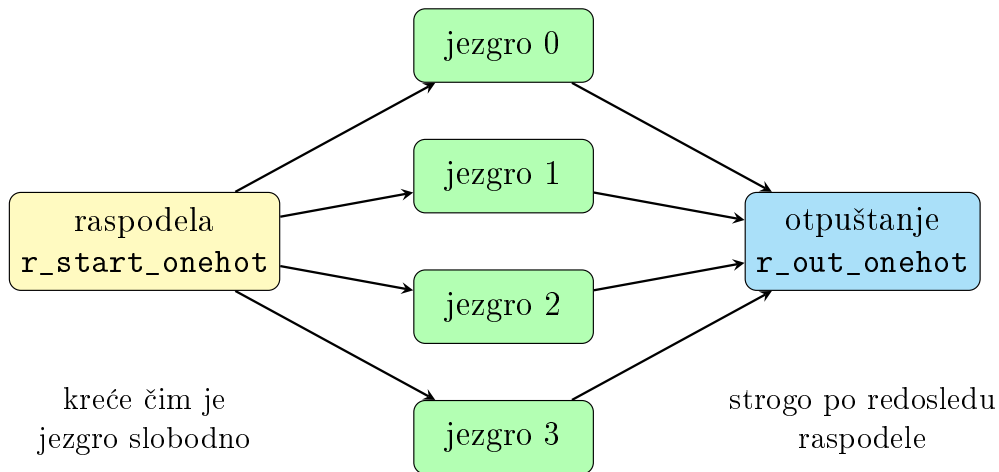
Slika 4.11: Protočna arhitektura AES jezgra. Svaki stepen protoka završava se svojim registrom (tamna traka uz desnu ivicu), a stepen registar sastoji se samo od njega

Višejezgarna arhitektura (MULTICORE) sadrži `NUM_CORES` iterativnih jezgara. Jezgro računa blok N_r taktova, a zatim rezultat drži još jedan takt na registarskom izlazu, dok ga omotač ne preuzme. Blok, dakle, zauzima jezgro $N_r + 1$ taktova, pa je zbirna propusnost

$$B_{\text{takt}} = \min\left(\frac{\text{NUM_CORES}}{N_r + 1}, 1\right) \text{ blokova po taktu,} \quad (4.1)$$

gde gornju granicu postavlja jedan izlazni port, koji ne može otpustiti više od jednog bloka po taktu. Jedan blok po taktu postiže se, dakle, sa $N_r + 1$ jezgara, što kod AES-256 znači petnaest jezgara. Raspodela posla je kružna (engl. *round-robin*), sa dva nezavisna pokazivača, prikazana na slici 4.12:

- `r_start_onehot` – koje jezgro sledeće prima blok, napreduje na svako prihvaćeno pokretanje;
- `r_out_onehot` – čiji se rezultat sledeće otpušta, napreduje na svako otpuštanje.



Slika 4.12: Kružna raspodela u višejezgarnoj arhitekturi

Dva pokazivača idu nezavisno, oba kružnim redom. Ulazni pokreće naredno jezgro čim je ono prazno, nezavisno od toga dokle je izlazni stigao. Zaustavlja se jedino kad ga sustigne, jer sme da pokrene samo prazno jezgro. Pokretanje pritom ne čeka ni da reč podataka stigne, pa niz nastaje unapred. Izlazni otpušta rezultate strogo redom, jezgro po jezgro, pa jezgro koje završi ranije čeka svoj red. Niz koji na kraju paketa ostane neiskorišćen odbacuje se pri pražnjenju. Oba pokazivača su kružne one-hot dozvole koje se pomeraju na sam događaj, prihvaćeno pokretanje odnosno otpuštenu reč, pa izbor jezgra ne traži dekodera. Uz izlaznu dozvolu napreduje i mali binarni pratilac, koji služi samo kao izbor multipleksera rezultata, jer je za širinu od 128 bitova binarni izbor jeftiniji od one-hot mreže.

Šta je otpušteni blok, H , maska oznake ili niz za šifrovanje, ne pamti se po jezgru, već sledi iz dve zastavice. Dok H za tekući ključ nije zabeležen, prvi gotov rezultat je H . Dok maska za tekući paket nije zabeležena, sledeći je maska. Posle toga svi su blokovi niza. Prva dva otpuštaju se bezuslovno, jer ih niko nizvodno ne mora prihvatiti, dok se blok niza otpušta samo uz potpuno rukovanje sa oba kraja.

4.4.2.3 Ekspanzija ključa u hodu

Ključevi rundi iz odeljka 3.3.6 mogu se izračunati unapred i zadržati u registrima, ili generisati u hodu (engl. *on-the-fly*), uporedo sa samim rundama. Ovde je izabrano drugo, a presudan je rad sa više ključeva odjednom. Na primer, čvor koji nosi N IPsec tunela ima po jedan ključ za svaki tunel, i paketi različitih tunela smenjuju se na istom akceleratoru.

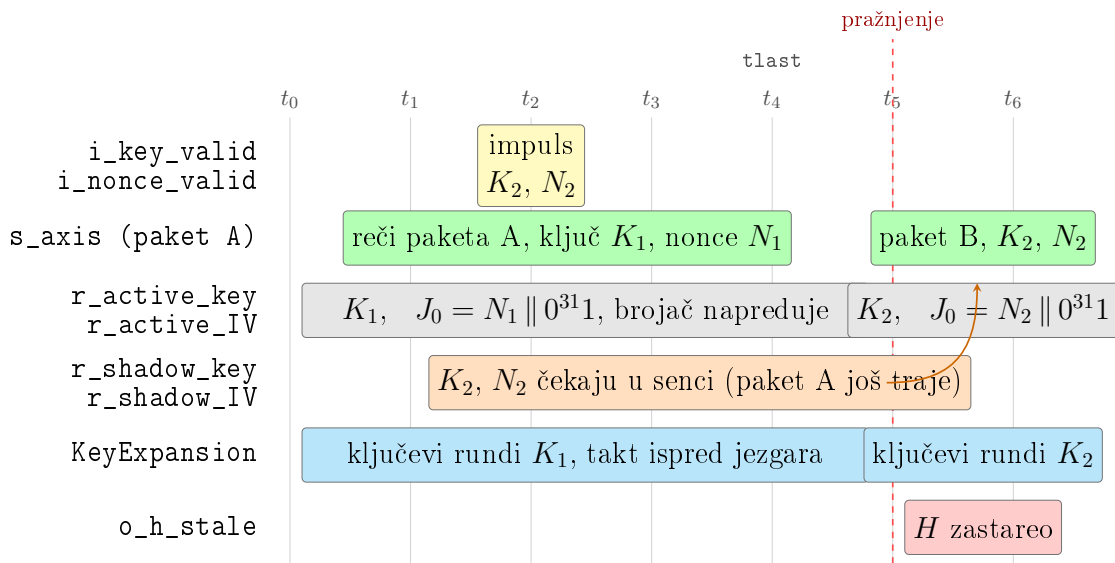
Sa unapred izračunatim ključevima takav rad ostavlja dva loša izbora: ili se pri svakoj promeni ključa čeka nova ekspanzija, pa protok pada, ili se prošireni ključevi keširaju, po 240 bajtova registara za svaki, pa zauzeće raste linearno sa brojem tunela. Generisanje u hodu uklanja oba, jer se za svaki ključ čuva samo polazni oblik, prošireni postoji jedino za ključ u upotrebi, a jedina cena promene ključa je jedno šifrovanje bloka više, kojim se preračunava potključ H za GHASH.

Sama ekspanzija pritom ne unosi nikakav zastoje. Jezgro nema fazu „proširi ključ, pa počni“ niti rukovanje tipa „ključevi spremni“. Ekspander proizvodi ključ runde K_i tačno jedan takt pre runde koja ga koristi, u koraku sa jezgrima, pa promena ključa ne košta nijedan prazan paket. Jedino što mora biti usklađeno jeste trenutak prve raspodele, i on kod višejezgarne arhitekture zavisi od dužine ključa. Kod AES-256 su K_0 i K_1 validni već u drugom ciklusu, dok kod AES-128 K_1 postaje validan

tek u trećem, pa se prva raspodela zadržava za jedan takt.

U višejezgarnoj arhitekturi jezgra ključeve rundi ne čitaju iz zajedničkog niza registara. Ekspander ključ svake runde emituje kao upis svim jezgrima, a svako ga upiše u sopstvenu distribuiranu memoriju od šesnaest reči i čita lokalno, po svom brojaču runde. Jezgro tako ne nosi multiplexer za izbor ključa, a raspored ključeva ne putuje preko čipa. Protočna arhitektura, sa jednim protokom, ključeve čita neposredno iz niza ekspandera, svaki stepen ključ svoje runde.

Poseban problem je ključ ili nonce koji stigne usred paketa. Ključevi rundi se u tom trenutku čitaju, a brojački blok je usred niza, pa ni jedno ni drugo ne sme da se promeni dok paket traje. Jezgro zato za obe vrednosti drži po jedan senka registar (engl. *shadow register*). Pristigla vrednost ne dira aktivnu, nego čeka u senci. Na ciklusu pražnjenja, kada se poslednja reč paketa preda, senka postaje aktivna vrednost, ekspander se pokreće ispočetka, a bočni signal javlja da je H zastareo dok se ne preračuna. Vremenski tok prikazan je na slici 4.13.



Slika 4.13: Ključ i nonce pristigli usred paketa (t_2) primenjuju se tek na ciklusu pražnjenja (t_5)

Tri pravila pokrivaju granične slučajeve, i sva tri su potvrđena namenskim testovima:

- *poslednji upis važi* – vrednost koja stigne baš na ciklusu pražnjenja pretiče onu koja već čeka u senci, pa nijedna izmena ne biva tiho izgubljena;
- *nezavisno pristizanje* – nonce pre ključa i ključ bez nonce vrednosti su podjednako dozvoljeni. Paket jednostavno čeka dok oba ne budu na mestu;
- *zaštita od ponovne upotrebe* – ako nijedna nonce vrednost nije u senci, jezgro na kraju paketa zaboravlja tekuću, pa naredni paket ne može krenuti dok se ne dobije sveža. Tiho ponavljanje brojačkog bloka nije dostižno preko interfejsa.

4.4.2.4 Upravljanje tokom: FLOW_STYLE

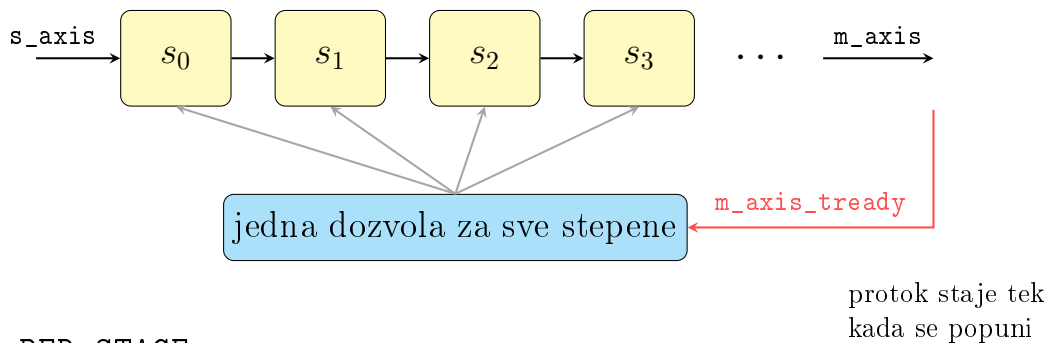
Unutar protočne arhitekture, način na koji se izvode dozvole za pomeranje stepena bira se generikom FLOW_STYLE. Oba izbora vode isti protok i daju isti šifrat.

Pre opisa treba uvesti dve veličine na koje se oba načina oslanjaju. *Vektor validnosti* je niz od $N_r + 2$ bita, po jedan za svaki stepen protoka, i bit i kaže da stepen i trenutno drži važeći blok. *Spremnost nizvodne strane* je signal `m_axis_tready`, kojim potrošač na izlazu javlja da može da primi reč. Prazan stepen usred punog protoka naziva se *mehur*. On ne nosi podatak, ali zauzima mesto i putuje kroz protok kao i svaki blok.

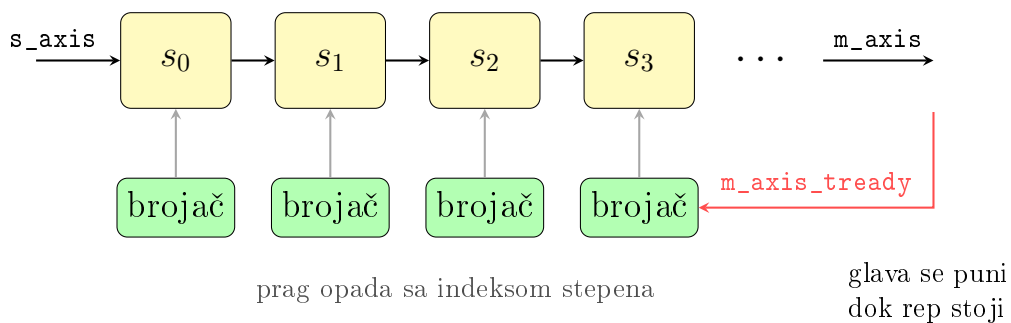
GLOBAL izvodi jednu jedinu dozvolu i razvodi je svim stepenima, pa se ceo protok pomera zajedno ili nikako. Dozvola se računa neposredno iz vektora validnosti i signala `m_axis_tready`, bez ijednog pomoćnog registra. Time je vektor validnosti jedini merodavan zapis o tome šta je u protoku, i ne postoji brojač koji bi se mogao raspapirati sa stvarnim sadržajem. Mana je što mehur nastao u sredini nikada ne nestaje. Pošto se ceo protok pomera zajedno, prazan stepen putuje sve do izlaza. Zastoj na izlazu, međutim, ne zaustavlja odmah i glavu protoka, jer dozvola ostaje aktivna dok god je poslednji stepen slobodan. Protok, dakle, nastavlja da prima dok se ne popuni, i tek tada staje. Dok stoji, poslednji stepen drži gotov blok sve dok ga potrošač ne primi, pa se pod protivpritiskom nijedan rezultat ne gubi.

PER_STAGE, preuzet iz [27], daje svakom stepenu sopstveni brojač zauzeća i prag koji opada sa indeksom stepena, tako da kasniji stepeni prestaju da primaju pre ranijih. Glava protoka može da nastavi sa punjenjem i pošto se poslednji stepen popuni, jer svaki stepen odlučuje sam za sebe, pa se mehuri istiskuju. Cena su $N_r + 2$ brojača i komparatora, i drugi izvor istine, koji mora ostati usklađen sa vektorom validnosti. Upravo taj problem *GLOBAL* nema, jer nema ni brojače. Oba načina prikazana su na slici 4.14.

(a) GLOBAL



(b) PER_STAGE



Slika 4.14: Dva načina upravljanja tokom u protočnoj arhitekturi: (a) jedna zajednička dozvola, pa se protok pomera u celini, a (b) po jedan brojač zauzeća po stepenu, pa glava protoka nastavlja da prima dok rep stoji

U GCM sistemu protok puni jedan raspoređivač, a prazni ga jedan XOR, pa je

po pravilu pun i mehuri su retki. Zbog toga je GLOBAL podrazumevani izbor, a poređenje dva stila pod zastojem dato je u odeljku 5.4.5.

4.4.2.5 Realizacija runde: ROUND_STYLE

Poslednji izbor je kojim resursom se računa runda. Obe varijante imaju isto kašnjenje od jednog takta i daju isti rezultat.

BRAM (engl. *Block RAM*, ugrađeni memorijski blok) koristi T-tabele iz odeljka 3.3.5, pa runda postaje XOR četiri tabelarna čitanja, čitanje iz memorije umesto aritmetike. Svaka od četiri kolone realizovana je zasebnom instancom koja izvršava četiri čitanja, a ShiftRows je utkan u to koja četiri bajta stanja koja instanca čita, kolona 0 čita bajtove 0, 5, 10 i 15, kolona 1 bajtove 4, 9, 14 i 3, i tako dalje. Tabela se čuva samo jednom, jer se T_1 , T_2 i T_3 dobijaju rotacijom pročitane T_0 vrednosti, preuređenjem veza posle registra, bez ijednog logičkog elementa. XOR koji spaja četiri rotirane vrednosti i ključ runde ima pet ulaza po bitu, pa staje u jednu šestoulaznu tabelu.

LUT varijanta ispisuje rundu kao lanac četiri transformacije i registruje ga jednom na kraju. Sve završava u logici, S-boks kao kombinaciona mreža, MixColumns kao stabla XOR kola. Nula memorijskih blokova, ali četiri kombinaciona koraka leže u seriji unutar jednog takta, i ta serija je kritična putanja.

Izbor važi samo za glavne runde. Poslednja nema MixColumns, pa se u njoj nema šta stopiti u tabelu, i ona ostaje u logici bez obzira na vrednost generika. Dve varijante tu logiku ne plaćaju jednako. U LUT varijanti završna runda koristi isti prolaz kroz S-boks koji svaka runda ionako računa, samo bez koraka MixColumns, pa sopstveni sloj S-boksova ne nosi. U BRAM varijanti takav prolaz ne postoji, jer T-tabele drže SubBytes stopljen sa MixColumns, pa svako jezgro nosi zaseban sloj od šesnaest S-boksova u distribuiranoj logici. Bilans dve realizacije runde kvantifikovan je u odeljku 5.4.3.

Pošto se čuva samo T_0 , deluje prirodno zaključiti da se troši četvrtina memorije u odnosu na klasičnu šemu sa četiri tabele. Taj zaključak je pogrešan, jer broj memorijskih instanci ne određuje količina sadržaja nego broj istovremenih čitanja. Memorijski blok ima dva porta za čitanje. Da bi opslužio N istovremenih čitanja, mora biti umnožen $\lceil N/2 \rceil$ puta, bez obzira na to šta u njemu piše.

Jedna runda izvršava 16 istovremenih čitanja (četiri kolone puta četiri bajta), pa poređenje izgleda kako je dato u tabeli 4.7.

Tabela 4.7: Broj kopija memorijskog bloka po rundi za dve organizacije T-tabela

Organizacija	Čitanja po tabeli	Kopija po tabeli	Ukupno kopija
klasična (T_0 - T_3)	$16/4 = 4$	$\lceil 4/2 \rceil = 2$	$4 \times 2 = 8$
samo T_0	16	$\lceil 16/2 \rceil = 8$	$1 \times 8 = 8$

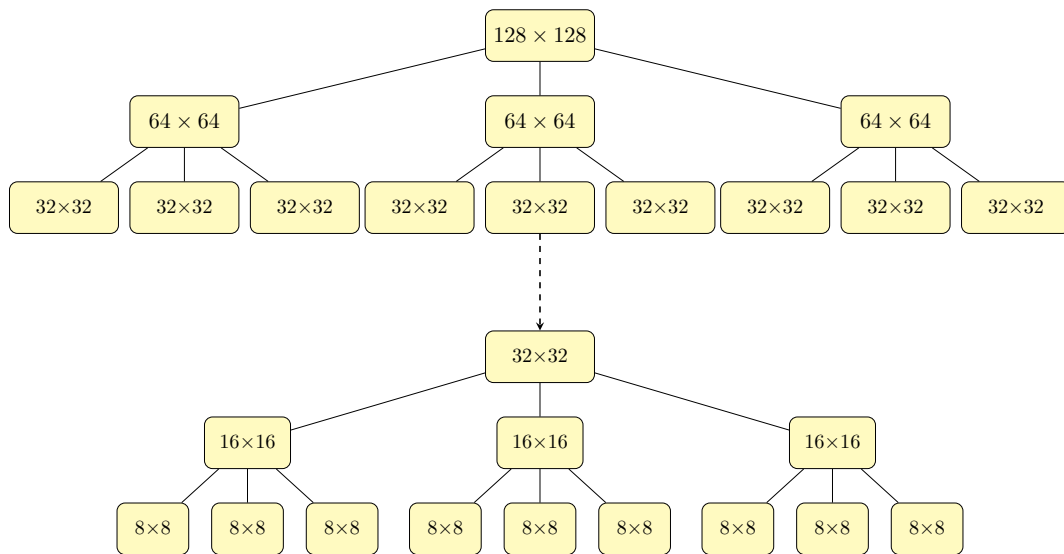
Rezultat je identičan. U klasičnoj šemi svaka od četiri tabele ionako mora biti udvostručena (četiri čitanja premašuju dva porta), pa se sadržaj koji je „ušteden” vraća kao umnožavanje portova. Ne postoji raspored u kome 16 paralelnih čitanja stane u manje portova. Ono što organizacija sa samo T_0 zaista donosi jeste jedan inicijalizator memorije umesto četiri, uz tri tabele svedene na preuređenje veza,

pojednostavljenje opisa, a ne ušteda površine. Pravi pokretač troška, 16 čitanja po rundi, ostaje nepromenjen.

4.4.3 GHASH jedinica

4.4.3.1 Množać u $GF(2^{128})$

Množać je realizovan Karatsubinom dekompozicijom, opisanom u odeljku 3.3.9.3. Izraz (3.46) primenjuje se rekurzivno, na svaki od tri dobijena proizvoda, pa je pri projektovanju trebalo izabrati dubinu, granicu na kojoj se množenje isplati realizovati neposredno. Izabrana su četiri nivoa, $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$, kako prikazuje slika 4.15. Nivoi od 128 do 16 su zasebni moduli, a množenje 8×8 je funkcija u najnižem od njih, pa se dubina menja zamenom jednog nivoa, bez izmene ostatka jedinice.



Slika 4.15: Rekurzivna Karatsubina dekompozicija množača. Nastavak ispod nivoa 32 prikazan je za jedan čvor, a isti je za svih devet

Ušteda se množi po nivoima, kako prikazuje tabela 4.8. U osnovi stabla ostaje $3^4 = 81$ školskih množenja 8×8 umesto $4^4 = 256$, dakle manje od trećine, uz odgovarajući porast broja XOR kola za spajanje parcijalnih proizvoda. Dalja rekurzija ispod 8×8 nije uzeta jer bi dobitak u I kolima počeo da se gubi u sve složenijoj mreži sabiranja i u dužoj kombinacionoj putanji.

Tabela 4.8: Broj množenja po nivoima stabla: školska dekompozicija naspram Karatsubine

Množenja	Školska dekompozicija	Karatsubina dekompozicija
64×64	4	3
32×32	16	9
16×16	64	27
8×8	256	81

Redukcija 255-bitnog nesvedenog proizvoda realizovana je kao kombinaciona mreža XOR kola po postupku iz odeljka 3.3.9.3, s tim što su oba prolaza spoje-

na u jedan silazni, jer bitove koje presavijanje usput vrati iznad stepena 127 isti prolaz kasnije obuhvati.

Konvencija o redosledu bitova iz odeljka 3.3.9.2 rešena je na jednom mestu. Operandi se ogledalski preuređuju pri ulasku u množač, a proizvod pri izlasku. Sam množač time radi u uobičajenoj matematičkoj konvenciji, a preuređenje je samo ožičenje.

4.4.3.2 Dve vremenske varijante

Kompletno množenje realizovano kao jedna kombinaciona mreža daje najkraću latenciju, ali i najdužu kritičnu putanju u celom akceleratoru. Zato je jedinica napravljena tako da se generikom bira jedna od dve vremenske organizacije:

- *jedan takt po množenju* – ceo lanac (preuređenje, dekompozicija, spajanje, redukcija) je jedna kombinaciona mreža, pa jedinica upija blok po taktu;
- *dva takta po množenju* – mreža je presečena registarskom bankom neposredno posle sloja od devet 32×32 množenja, pa je kritična putanja kraća, a jedinica upija blok svaka dva takta.

Dotok tom ritmu prilagođava modul koji jedinicu opslužuje, o čemu je reč u odeljku 4.4.4.

Propusnost akceleratora je minimum dve gornje granice. Granica šifarskog puta pomera se brojem jezgara, dok je granica autentifikacionog puta, sa jednim množačem, zadata upravo ovim izborom. Kako pokazuje tabela 4.9, dodavanje AES jezgara preko onoga što autentifikacioni put može da prihvati ne donosi propusnost, već samo skriva latenciju. Kvantitativno poređenje obe varijante dato je u odeljku 5.4.4.

Tabela 4.9: Uticaj vremenske organizacije GHASH jedinice na granicu propusnosti akceleratora

Takta po množenju	Dotok u GHASH	Granica sistema	Kritična putanja
1	blok po taktu	1 blok/takt	najduža
2	blok svaka dva takta	0,5 blokova/takt	kraća

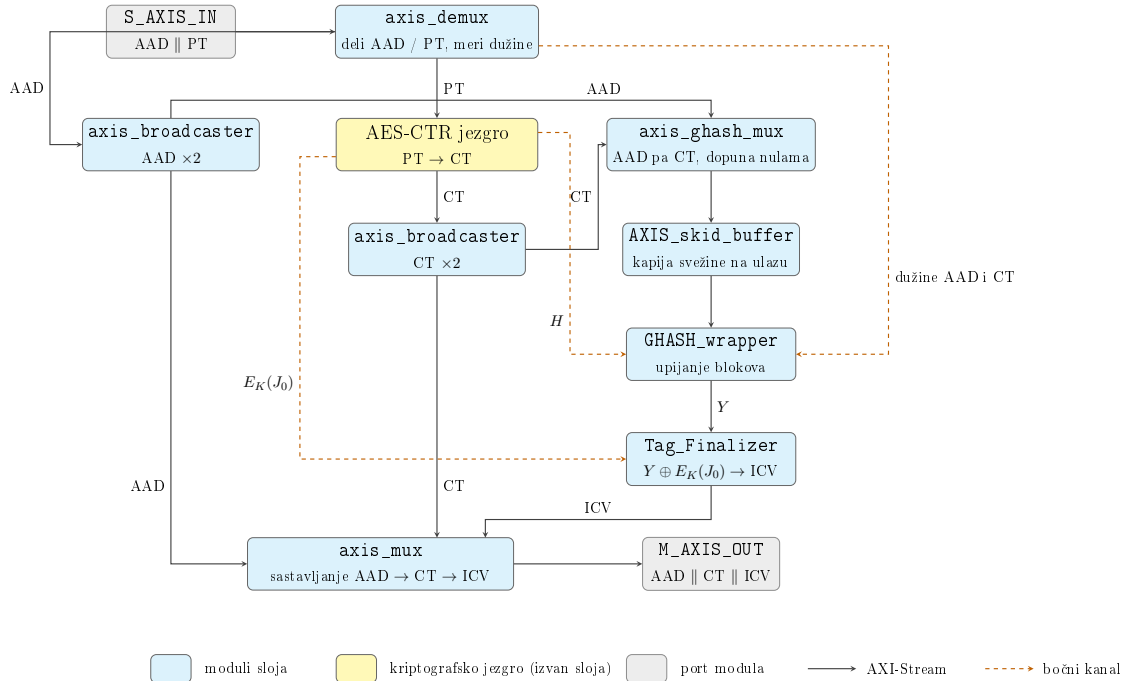
Za mrežni sistem od 10 Gb/s sa 128-bitnom magistralom na taktu od 200 MHz dvotaktna varijanta obezbeđuje 12,8 Gb/s, dakle i dalje sa rezervom, pa se isplati uzeti kraću kritičnu putanju i višu radnu učestanost. Tek za veće brzine ima smisla preći na jednotaktnu varijantu, a isti cilj postiže i paralelizacija sa stepenima H^p iz odeljka 3.3.9.5. Sa dva dvotaktna množača, za $p = 2$, jedinica upija blok po taktu, i to na višoj učestanosti koju dozvoljava kraća kritična putanja.

4.4.4 GCM sloj

Modul koji stoji između AXI4-Stream interfejsa i kriptografskog jezgra sadrži sve iz GCM režima osim šifre: razdvajanje segmenata, GHASH, blok sa dužinama i rad sa oznakom. Postoje dve varijante, za šifrovanje i za dešifrovanje, koje dele istu strukturu i isto jezgro, a razlikuju se u pojedinostima opisanim u daljem tekstu.

4.4.4.1 Tok kroz GCM sloj na putu šifrovanja

Ulazni tok se najpre razdvaja na pridružene podatke i korisni sadržaj, uz brojanje dužina oba segmenta. Oba se zatim granaju, jedna kopija ide u GHASH, a druga u izlazni multiplekser, odnosno preko kriptografske granice u jezgro. Izlaz se sastavlja redom AAD, šifrat, oznaka. Struktura je prikazana na slici 4.16.



Slika 4.16: Moduli GCM sloja na putu šifrovanja

Moduli sa slike, redom kojim podatak putuje kroz sloj:

Razdvajanje segmenata (**axis_demux**) deli ulazni tok na AAD i PT. Granica nije označena u samim podacima. Akcelerator štiti pakete unapred poznatog formata, pa je dužina AAD segmenta generik, utvrđen pri sintezi. Dužinu korisnog sadržaja modul meri u hodu, brojeći reči i važeće bajtove poslednje, i obe dužine šalje bočnim kanalom za završni blok GHASH funkcije.

Grananje (**axis_broadcaster**) pravi dve istovetne kopije toka, bez sopstvenog skladišta. Reč prolazi tek kada su oba potrošača spremna, pa sporiji od njih zadržava i bržeg. Instancirana su dva, jedan za AAD i jedan za šifrat, u oba slučaja sa po jednom kopijom za autentifikacioni put i jednom za izlaz.

Multiplekser autentifikacionog puta (**axis_ghash_mux**) sastavlja redosled koji GHASH zahteva: najpre svi AAD blokovi, zatim svi blokovi šifrata. Usput sprovodi pravilo dopune iz standarda, tako što bajtove koje maska **tkeep** ne pokriva postavlja na nulu, pa nepotpun poslednji blok segmenta ulazi u GHASH kao čist 128-bitni koeficijent. Maskiranje važi samo za ovu granu, kopija šifrata koja putuje ka izlazu ostaje netaknuta.

Skid bafer (**AXIS_skid_buffer**) je registarski stepen sa jednim rezervnim mestom, umetnut po pravilu iz odeljka 4.1.2. Registruje rukovanje u oba smera i time prekida kombinacionu putanju **tready** signala na ulazu u GHASH omotač. Rezervno mesto pritom čuva reč zatečenu u letu kada omotač u dvotaktnoj varijanti spusti spremnost. Na njegovom ulazu i kapija svežine iz odeljka 4.4.4.3 zaustavlja početak novog paketa.

GHASH omotač (`GHASH_wrapper`) vodi jedinicu iz odeljka 4.4.3: upija blokove pristigle sa multipleksera, po poslednjem sam dodaje završni blok sa dužinama, sastavljen od vrednosti primljenih od razdvajanja segmenata, i vraća konačan sažetak Y . Čita isti generik `MULT_CYCLES` kao i jedinica i u dvotaktnoj varijanti spušta spremnost svaki drugi takt. To je modul koji dotok prilagođava ritmu množača, najavljen u odeljku 4.4.3.2.

Završetak oznake (`Tag_Finalizer`) pamti $E_K(J_0)$ čim ga jezgro izračuna, a kada stigne konačno Y , računa $ICV = Y \oplus E_K(J_0)$ i izdaje oznaku kao jednu reč toka, koju drži dok je izlaz ne preuzme.

Sastavljanje izlaza (`axis_mux`) spaja tri toka u izlazni paket utvrđenim redom: AAD, šifrat, oznaka, a `tlast` podiže tek na reči sa oznakom.

4.4.4.2 Prevođenje redosleda bajtova

Dve konvencije se ovde susreću. AXI4-Stream nosi nulti bajt toka u najnižoj poziciji reči. Pri širini magistrale od 128 bita prvi bajt paketa stiže na linijama [7:0], a šesnaesti na [127:120]. AES i GHASH, nasuprot tome, nulti bajt bloka smeštaju na najvišu poziciju. Bez prevođenja, prvi bajt niza za šifrovanje sabirao bi se sa poslednjim bajtom otvorenog teksta.

Prevođenje je zato svedeno na tačno dve tačke, ulazni i izlazni port sloja, gde se reč obrće po bajtovima, a maska `tkeep` po bitovima, tako da preživljava pravilo „bit i označava bajt i “. Tok podatka kroz te dve tačke prikazan je u tabeli 4.10. Time nijedan modul unutar sloja ne mora znati za konvenciju, a nepotpuna poslednja reč prirodno smešta svoje bajtove na najviše pozicije bloka, ostavljajući nule iza njih, što je upravo pravilo dopunjavanja nulama iz GCM standarda.

Tabela 4.10: Redosled bajtova kroz akcelerator, na primeru bloka 00 11 22 ... ff

Tačka u sistemu	Vrednost
u memoriji (redom bajtova)	00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff
na <code>s_axis_tdata[127:0]</code>	ff ee dd cc bb aa 99 88 77 66 55 44 33 22 11 00
posle obrtanja, kao AES blok	00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff
šifrat kao AES blok	8a b2 82 15 ee 2f 29 6c ce 92 f7 66 b7 c0 67 c0
na <code>m_axis_tdata[127:0]</code>	c0 67 c0 b7 66 f7 92 ce 6c 29 2f ee 15 82 b2 8a
u memoriji (redom bajtova)	8a b2 82 15 ee 2f 29 6c ce 92 f7 66 b7 c0 67 c0

Sa prevođenjem na oba porta, prvi bajt niza za šifrovanje sabira se sa prvim bajtom otvorenog teksta, a šifrat koji se vraća u memoriju bajt po bajt jednak je onom koji standard propisuje za iste ulaze.

Upravo se zato ovo mesto proverava testovima sa poznatim odgovorima, o kojima je reč u odeljku 5.1.

4.4.4.3 Kapija svežine heš potključa

Ova kapija postoji zbog jedne posledice koja se lako previdi. GHASH može da počne rad pre nego što je spreman. Pridruženi podaci ulaze u GHASH pre korisnog

sadržaja, a oni ne zavise od šifre, pa sloj nema prirodan razlog da ih zadrži, a na putu dešifrovanja u GHASH ulazi primljeni šifrat, koji kroz jezgro i ne prolazi. Heš potključ H , međutim, zavisi od ključa i dobija se tek pošto jezgro izračuna $E_K(0^{128})$. Pri gustom saobraćaju, kada nov ključ stigne uz nov paket, prve reči tog paketa mogu stići pre te vrednosti.

Posledica bi bila upijanje sa zastarelim H , dakle oznaka izračunata pogrešnim potključem. To je greška koja se ne prijavljuje ni na jednoj strani. Pošiljalac bi proizveo oznaku koju primalac odbacuje, a uzrok bi bio vidljiv samo pod gustim saobraćajem i tek na paketu koji nosi promenu ključa.

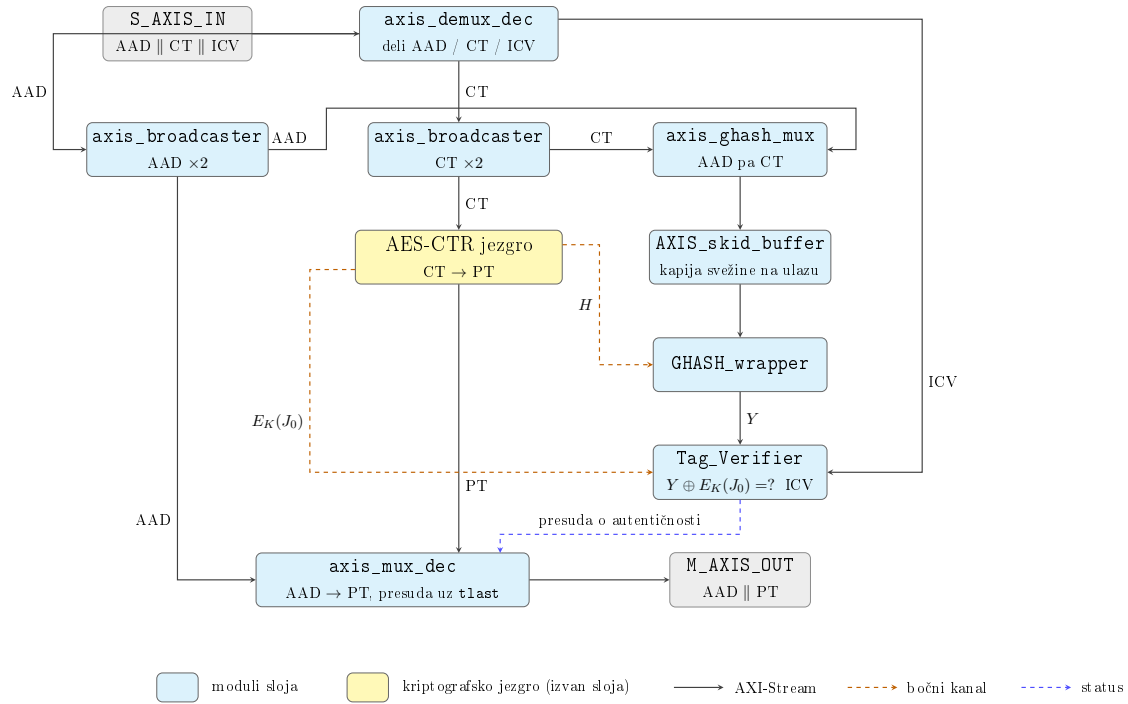
Rešenje je jedan bočni signal i jedno pravilo. Jezgro drži signal aktivnim dok H za novi ključ nije preračunat, a sloj dok signal traje zadržava početak novog paketa na ulazu u GHASH. Paket koji već traje kapija nikada i ne presreće, jer ključ pristigao usred paketa čeka u senci do ciklusa pražnjenja (odjeljak 4.4.2.3), pa se potključ tekućeg paketa ne može promeniti. Kapija, dakle, deluje samo na granici paketa.

4.4.4.4 Razlike puta dešifrovanja

Sloj na putu dešifrovanja zadržava strukturu puta šifrovanja. Grananje, multi-plekser autentifikacionog puta, skid bafer i GHASH omotač isti su moduli, samo grananje šifrata sada stoji ispred kriptografske granice, jer standard propisuje da se autentifikuje ono što je primljeno, a ne ono što je dobijeno dešifrovanjem (odjeljak 3.3.9.2), pa GHASH radi nad primljenim šifratom, pre XOR-a. Tri modula su drugačija:

- *Razdvajanje segmenata* (`axis_demux_dec`) deli ulazni tok na tri segmenta, jer paket pored AAD i šifrata nosi i oznaku. Pošto je poslednja reč šifrata poznata tek kada stigne oznaka, modul u sopstvenom skid baferu zadržava jednu reč šifrata, da bi `tlast` pao na stvarno poslednju reč.
- *Provera oznake* (`Tag_Verifier`) zamenjuje završetak oznake. Izračunatu i primljenu vrednost prihvata bilo kojim redosledom, prva pristigla se pamti dok ne stigne i druga.
- *Sastavljanje izlaza* (`axis_mux_dec`) drži poslednju reč otvorenog teksta dok rezultat provere oznake ne bude poznat, pa je zatim šalje zajedno sa `tlast` i signalom ispravnosti. Potrošač tako saznaje da li je paket autentičan u istom taktu u kome prima njegov poslednji bajt, i može ga odbaciti, čime se poštuje pravilo iz odeljka 3.3.9.2 da dešifrovani sadržaj ne sme biti prosleđen dalje pre uspešne provere.

Kompletnan raspored modula dat je na slici 4.17.



Slika 4.17: Moduli GCM sloja na putu dešifrovanja

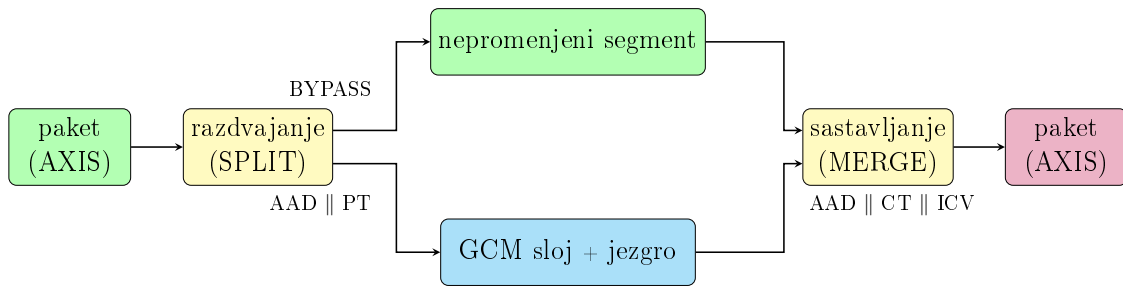
4.4.5 Sistemski omotač

Do sada opisani deo akceleratora radi sa već razdvojenim segmentima. Stvaran paket, međutim, stiže kao jedan neprekidan niz bajtova u kome granice segmenata nisu ničim označene, i po pravilu ne padaju na granicu reči magistrale. Ako je zaglavlje koje se prenosi nepromenjeno dugačko 34 bajta, a magistrala široka 16, treća reč paketa sadrži i poslednja dva bajta zaglavlja i prvih četrnaest bajtova pridruženih podataka. Sistemski omotač rešava upravo taj problem. U osnovi ga čini par jezgara, razdvajanje i sastavljanje, uz skid bafere između njih i, na putu dešifrovanja, jezgro za poravnanje oznake.

Razdvajanje (SPLIT) deli ulazni tok na dva izlaza: segment koji se prenosi nepromenjen i ostatak, pri čemu svaki segment mora početi od nultog bajta svoje reči, jer to GCM sloj očekuje. Pošto granice padaju usred reči, modul sadrži prenosni registar (engl. *gearbox*), koji zadržava bajtove narednog segmenta pristigle u graničnoj reči i poravnava ih uz sledeće.

Sastavljanje (MERGE) je inverzna operacija. Od dva toka pravi jedan neprekidan, tako da je samo poslednja reč nepotpuna. Vredi uočiti asimetriju. Sastavljanju nije potreban nijedan generik sa dužinom, jer oba ulazna toka sama najavljuju svoj kraj signalom `tlast`. Razdvajanju jeste, jer ono mora da preseče jedan tok na mestu koje u podacima ništa ne označava.

Uz njih idu i skid baferi, tri na putu šifrovanja i četiri na putu dešifrovanja, na samom ulazu, na izlazu, na grani BYPASS i, pri dešifrovanju, na kriptografskoj grani pred poravnanjem oznake. Svaki registruje rukovanje u oba smera i time prekida kombinacione putanje između jezgara. Razdvajanje i sastavljanje imaju i generik kojim se segment BYPASS isključuje u potpunosti, za pakete koji počinju neposredno pridruženim podacima. Razdvajanje tada kreće od segmenta AAD, sastavljanje samo prosleđuje kriptografski tok, a portovi kojima segment BYPASS putuje i skid bafer između njih nestaju iz sistema. Lanac na putu šifrovanja prikazan je na slici 4.18.

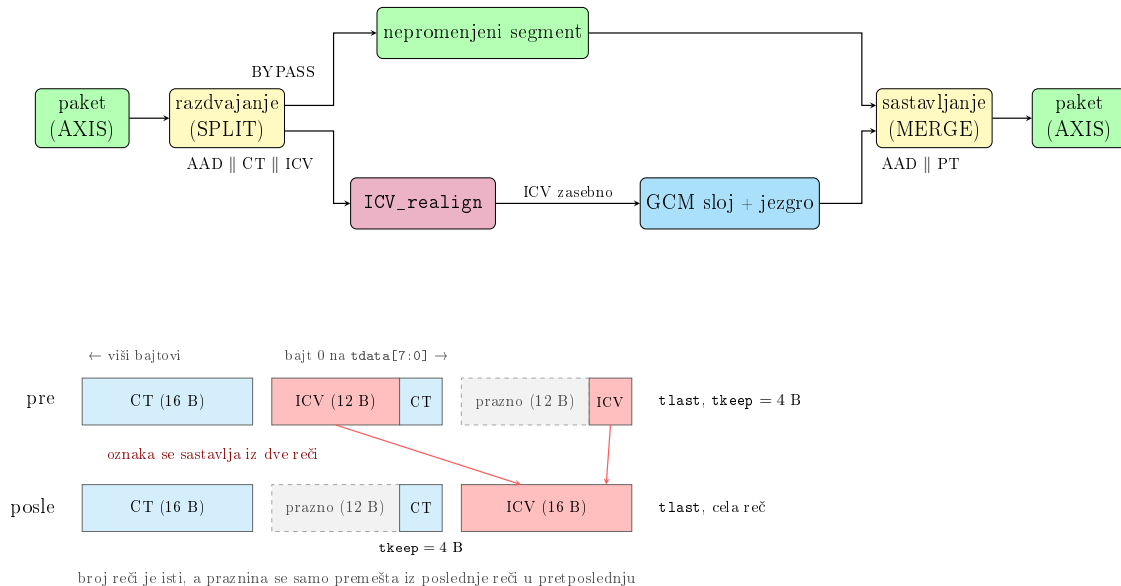


Slika 4.18: Sistemski omotač na putu šifrovanja

Na strani dešifrovanja lanac ima jedno jezgro više, i to zbog razlike koja postoji samo u tom smeru. Pri šifrovanju oznaka nastaje unutar GCM sloja, koji je sam stavlja na kraj. Pri dešifrovanju oznaka stiže spolja, spakovana neposredno uz šifrat. Pošto dužina korisnog sadržaja nije unapred poznata, poslednjih šesnaest bajtova paketa je oznaka, a kako se njen početak najčešće zatekne usred reči, njeni bajtovi su razdeljeni na poslednje dve reči. GCM sloj, međutim, zahteva oznaku samu na poslednjoj reči.

Taj razmak popunjava jezgro `ICV_realign`. Ono zadržava po jednu reč unazad, jer se tek sa signalom `tlast` sazna koliko bajtova poslednje reči pripada oznaci. Ako je k broj važećih bajtova u poslednjoj reči, rep šifrata su donjih k bajtova zapamćene reči, a oznaka se sastavlja od preostalih bajtova te reči i bajtova poslednje.

Bitno je uočiti da se pri tome ništa ne dodaje. Broj reči ostaje isti, kao i broj važećih bajtova. Jedino se praznina, koja je pre preuređenja stajala u poslednjoj reči iza oznake, premešta u pretposlednju, iza repa šifrata. Rezultat je tok u kome poslednja reč šifrata sme biti nepotpuna, a oznaka zauzima celu narednu reč, sa `tlast` na njoj. Postupak je, na primeru repa šifrata od četiri bajta, prikazan na slici 4.19.



Slika 4.19: Sistemski omotač na putu dešifrovanja

Ovakva organizacija ostvaruje ulogu umetnute zaštite paketa sa početka odeljka. Zaglavlja nižih slojeva prolaze netaknuta, a zaštita se primenjuje tačno na deo paketa koji joj pripada.

4.4.6 Pogodnost za dinamičku rekonfiguraciju

Podela na slojeve iz odeljka 4.4.1 ima posledicu koja prevazilazi urednost opisa. Kriptografska granica, lista portova između sloja i jezgra, nema nijedan signal svojstven AES-u, nego samo ključ, nonce, dva toka podataka i četiri bočna kanala (H , maska oznake i dve zastavice stanja). Sve što je vezano za GCM režim (GHASH, blok sa dužinama, oznaka, redosled bajtova, segmenti) stoji sa jedne strane te granice, a sve što je vezano za šifru sa druge.

Iz toga sledi da je akcelerator prirodno podeljen na *statički deo*, koji se ne menja, i *dinamički deo*, samo jezgro šifre. Takva podela je upravo ono što traži dinamička delimična rekonfiguracija (engl. *Dynamic Function eXchange*, DFX). Particija ima nepromenljiv interfejs, jasno definisan protokol na njemu i nema skrivenog stanja koje bi prelazilo granicu. Zamena algoritma u radu, ili čak zamena jezgra iste šifre drugom arhitekturom, na primer prelazak sa višejezgarne na protočnu prema trenutnom opterećenju, ne zahteva izmenu ostatka sistema.

Dva ograničenja ipak postoje. Prvo, sadašnji interfejs pretpostavlja da algoritam sam formira brojački blok iz 96-bitne nonce vrednosti. Algoritam koji zahteva drugačiju konstrukciju (recimo pun inicijalizacioni vektor koji generiše neki drugi deo sistema, ili nonce druge dužine) tražio bi manju izmenu granice. Drugo, ključni port je širine 256 bita, pa bi šifra sa dužim ključem takođe zahtevala proširenje. Oba su proširenja same liste portova, ne i logike sa obe strane.

4.4.7 Uklapanje u sistem i provera

Za razliku od SHA-3 jezgra, AES-GCM akcelerator stoji u samom mrežnom putu, kao umetnuta zaštita paketa iz odeljka 4.4.1. Merilo takvog jezgra je propusnost u ustaljenom stanju uz proizvoljan protivpritisak, a ne trajanje pojedinačnog poziva, pa je i celokupno upravljanje tokom podređeno tome.

Jezgro je provereno na tri nivoa, od vektora sa poznatim odgovorima do rada u postojećem mrežnom sistemu. Postupci i svi rezultati dati su u odeljku 5.4.

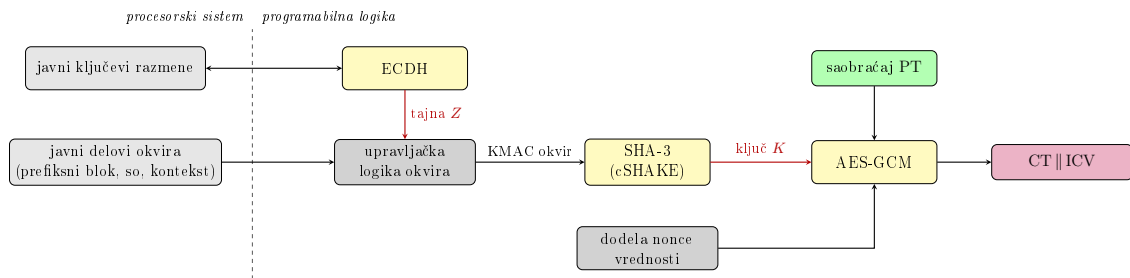
4.5 Integracija jezgara u lanac

Tri jezgra projektovana su da sarađuju onako kako to nalaže hibridni protokol iz odeljka 2.4, pri čemu SHA-3 jezgro ključ sesije K iz tajne Z izvodi u cSHAKE režimu. Pri tom povezivanju Z i K putuju isključivo između jezgara, unutar programabilne logike, i ni u jednom trenutku ne prolaze kroz memoriju procesorskog sistema. Ciljna organizacija lanca prikazana je na slici 4.20.

Ovde se vidi i zašto KMAC nema svoje jezgro. Kako je pokazano u odeljku 3.2.7, KMAC je cSHAKE sa propisanim uokvirivanjem ulaza. Pri tome su svi delovi okvira javni (prefiksni blok, so i kontekst razmene), osim same tajne Z . Zato okvir sklapa upravljačka logika ispred SHA-3 jezgra. Javne delove primi unapred pripremljene, a Z umeće neposredno sa tajnog izlaznog toka ECDH jezgra, koji za tu namenu i postoji. Upravo ta podela čini tvrdnju iz prethodnog pasusa ostvarivom, jer strana koja priprema javne delove okvira tajnu nikada ne vidi.

Upravljačka logika okvira u ovom radu nije realizovana kao zasebno jezgro, pa se ceo lanac sa slike ne sklapa na ploči. Zajednički rad jezgara proverava se simulacijom nad povezanim jezgrima, u kojoj verifikaciono okruženje preuzima upravo tu ulogu: formira KMAC okvir, predaje nonce vrednosti i pokreće jezgra propisanim

redosledom. Time se proverava ono što pojedinačni testovi po prirodi ne mogu, a to je da se izlaz jednog jezgra bez izmene prihvata na ulazu narednog. Rezultati su dati u odeljku 5.5, a realizacija upravljačke logike kao gotovog IP jezgra spada u nadogradnje o kojima je reč u poglavlju 6.



Slika 4.20: Ciljna organizacija lanca izvođenja i upotrebe ključa unutar programabilne logike. Crvene strelice nose tajni materijal

Poglavlje 5

Verifikacija i rezultati

Prethodna dva poglavlja opisala su šta jezgra računaju i kako su građena. Ovo poglavlje odgovara na pitanje koliko to košta i koliko brzo radi. Izlaganje počinje postupkom kojim su brojevi dobijeni, jer bez njega nijedna vrednost nema značenje, zatim ide jezgro po jezgro, a poslednji odeljak spaja tri jezgra i meri njihov zajednički rad.

5.1 Metodologija verifikacije

Provera i merenje odvijaju se na šest nivoa, i svaki odgovara na drugačije pitanje.

Ispravnost algoritma proverava se testovima sa poznatim odgovorima. Ulaz i očekivani izlaz preuzimaju se iz standarda ili se generišu referentnim modelom koji je prethodno upoređen sa standardnom bibliotekom, pa se izlaz jezgra poredi bit po bit. Ovi testovi imaju poseban status. Dosledno primenjena pogrešna konvencija preživela bi svaki povratni test, jer bi se ista greška poništila pri obrnutoj operaciji, a otkriva je jedino poređenje sa objavljenom vrednošću. Oni su zato jedini koji dokazuju usklađenost sa standardom, dok ostali testovi dokazuju doslednost realizacije.

Ponašanje pod opterećenjem proverava se namenskim testovima koji jezgro dovode u granične situacije: zastoje na ulazu, zastoje na izlazu, poruka koja se završava usred reči, promena ključa usred paketa. Ovi testovi ne dokazuju usklađenost sa standardom, nego doslednost realizacije.

Zauzeće resursa i radna učestanost dobijaju se implementacijom van konteksta (engl. *out-of-context*). Jezgro se implementira samo, bez ostatka sistema, tako da alat ne može da optimizuje preko njegovih granica niti da mu pripiše resurse koji mu ne pripadaju. Bitno je da su navedene vrednosti post-implementacione, dakle očitane posle raspoređivanja i povezivanja, a ne posle sinteze. U fazi implementacije alat ukloni deo logike koju sinteza ostavi, pa bi post-sintezni broj precenio površinu. Na jednom primeru iz ovog rada ta razlika iznosi oko 13 procenata.

Jedino vremensko ograničenje koje se zadaje jeste period takta. Kašnjenja na ulaznim i izlaznim portovima se namerno ne zadaju. Merena veličina je time kritična putanja između registara unutar jezgra, a ne vreme koje bi jezgru ostalo u nekom konkretnom okruženju, jer to okruženje u ovoj fazi ne postoji. Navedena radna učestanost je, dakle, svojstvo samog jezgra. Pri ugradnji u sistem, gde ulazi i izlazi imaju stvarna vremena, ostvariva učestanost može biti niža i mora se ponovo proveriti u kontekstu. Sama odluka ipak ne ostavlja interfejs neproverenim. Ulazi jezgara su pretežno registrovani, a tamo gde nisu stoje skid baferi iz odeljka 4.1.2. Uz to je na četiri konfiguracije, po dve SHA-3 i AES, merenje ponovljeno sa interfejsnim budžetom od 0,5 ns sa obe strane. Učestanost se pritom menja od $-1,8$

do +2,1 procenata, u oba smera, dakle ispod šuma alata. Najduža putanja između unutrašnjih registara ostaje mnogo duža od bilo koje putanje do porta, pa dodatno ograničenje nikada ne postane merodavno. Isto pravilo primenjeno je na sva tri jezgra. Radna učestanost se pri tome ne očitava kao ispunjenje zadatog cilja, nego se period zateže do granice, a prijavljuje se period poslednjeg zatvorenog prolaza umanjen za preostalu vremensku rezervu.

Uz to treba reći i šta navedene vrednosti nisu. One nisu apsolutni maksimum jezgra. Period se zateže dok se vremenski uslovi još ispunjavaju, ali uz podrazumevanu strategiju implementacije, a izbor strategije i direktiva ume osetno da promeni radnu učestanost. Na trinaest SHA-3 i AES konfiguracija ponovljenih sa drugačijom početnom učestanošću pretrage ona se menjala od $-7,7$ do $+6,7$ procenata, u oba smera, i taj se opseg u nastavku uzima kao šum alata. Cilj ovde nije najviša moguća vrednost za pojedinačno jezgro, nego jedinstven i ponovljiv postupak koji dopušta poređenje tri jezgra međusobno. Traganje za maksimumom po jezgru narušilo bi upravo tu uporedivost, jer bi svako jezgro bilo mereno pod sopstvenim, drugačije podešenim uslovima.

Propusnost i latencija mere se takt-tačno u simulaciji. Meri se broj taktova od prve prihvaćene do poslednje predate reči, iz čega sledi koliko bita jezgro obradi po taktu. Ta vrednost je svojstvo arhitekture i ne zavisi od tehnologije. Propusnost u apsolutnim jedinicama dobija se množenjem sa najvišom radnom učestanošću:

$$\text{propusnost} = (\text{bita po taktu}) \times F_{max}. \quad (5.1)$$

Isti postupak korišćen je za sva tri jezgra, s tim što se kod ECDH jezgra, koje izvršava jednu dugu operaciju po pozivu, umesto propusnosti meri latencija skalarnog množenja, na samom jezgru, od prihvaćenog zahteva do znaka kraja.

Rad na ploči i zajednički rad lanca podižu istu proveru ispravnosti na stvarni čip, odnosno na povezana jezgra. Nova merila ne uvode, pa su postupci opisani uz same rezultate (odeljci 5.3.4, 5.4.1 i 5.5).

Implementacija i merenja izvedeni su alatom Vivado 2025.1, za čip Zynq Ultra-Scale+ razvojnog sistema Kria KR260, koji je ciljna platforma rada.

Pregled onoga što je za svako jezgro provereno i izmereno dat je u tabeli 5.1.

Tabela 5.1: Nivoi provere po jezgru

Provera	ECDH	SHA-3	AES-GCM
poznati odgovori	sva tri nivoa	sve varijante	ceo akcelerator
granični slučajevi	iscrpan protivpritisak	gust saobraćaj i protivpritisak	obraci zastoja
implementacija van konteksta	15 konfiguracija	14 konfiguracija	28 konfiguracija
latencija/propusnost	16 konfiguracija	8 konfiguracija	pet serija
rad na ploči	—	drajver, PetaLinux	sistem, MTS 5800
zajednički rad lanca	dve strane u simulaciji, razmena ključa pa zaštićeni paketi		

5.2 ECDH akcelerator

5.2.1 Provera po nivoima

Sve vektore generiše referentni model u jeziku Python, a provera ide u tri nivoa. Prvi su pojedinačne operacije u polju, dakle množač, kvadrator, inverter i cela aritmetička jedinica, svaki sa sopstvenim skupom vektora. Drugi su celine iznad njih,

iteracija merdevina, kontroler i završna konverzija. Treći je ceo akcelerator, kroz AXI4-Stream omotač, od ulaznog paketa do afinog rezultata.

Dva polja služe različitoj svrsi. Nad malim poljem $GF(2^4)$ prostor ulaza je dovoljno mali da se pređe u celini, pa se za obe komande i sve širine reči prolaze sve kombinacije protivpritiska na oba izlazna toka. Nad punim poljem B-571, gde iscrpna provera nije moguća, prolaze vektori koji pokrivaju ceo put. Svi nivoi puštaju se na obe konfiguracije jezgra, koje na istim vektorima daju identičan rezultat, čime je potvrđeno da generik `G_LOW_LATENCY` menja raspored operacija u vremenu, a ne sam račun.

5.2.2 Zauzeće resursa i radna učestanost

Za obe konfiguracije jezgra pokrenuta je implementacija za osam vrednosti generika `G_D`, od 1 do 128 udvostručavanjem, ukupno šesnaest tačaka, od kojih je petnaest prošlo. Meri se ceo IP, sa deserijalizacijom i serijalizacijom, pri širini interfejsa od 64 bita i sa stvarnim koeficijentom b krive B-571. Rezultati su dati u tabelama 5.2 i 5.3.

Tabela 5.2: Osnovno ECDH jezgro, implementacija van konteksta

G_D	LUT	FF	F_{max} [MHz]
1	17426	17402	323,17
2	17515	17418	320,79
4	18176	17401	280,26
8	19551	17392	277,84
16	21951	17389	247,05
32	27602	17440	215,29
64	35497	17389	184,93
128	53759	17476	139,38

Tabela 5.3: Paralelno ECDH jezgro, implementacija van konteksta

G_D	LUT	FF	F_{max} [MHz]
1	19495	25380	315,62
2	19404	25495	302,18
4	22109	25518	265,52
8	27847	25364	260,68
16	40409	25522	230,74
32	58112	25511	182,15
64	92710	25807	126,35

Zauzeće raste linearno sa brojem bitova obrađenih u taktu, pa prave kroz izmerene tačke, sa eksponentom I za osnovno i II za paralelno jezgro, glase

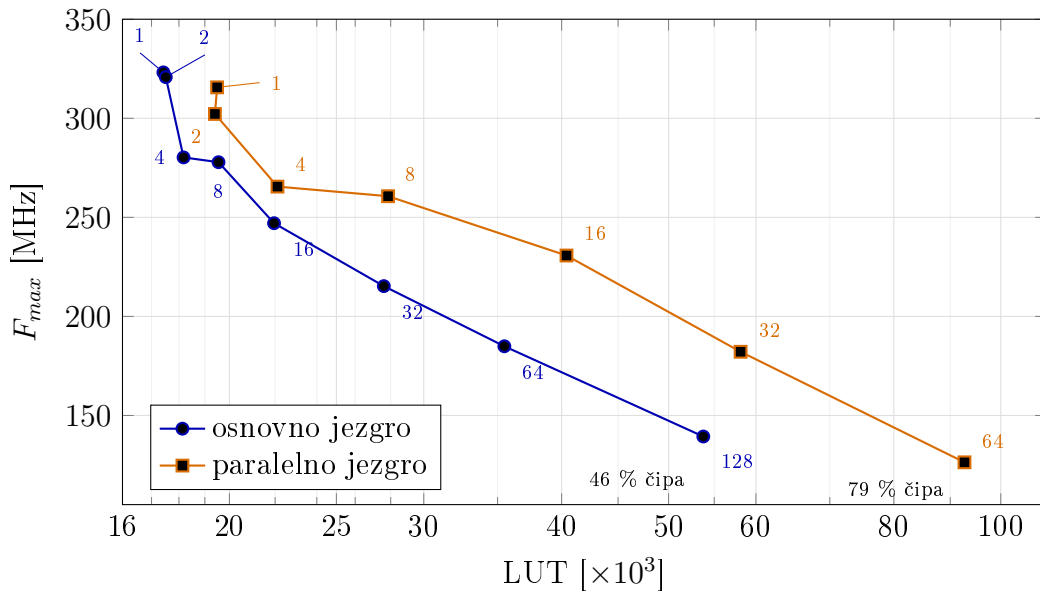
$$\text{LUT}^{\text{I}} = 17\,306 + 286,4 \cdot D, \quad (5.2)$$

$$\text{LUT}^{\text{II}} = 18\,590 + 1181 \cdot D. \quad (5.3)$$

Prava osnovnog jezgra opisuje merenje sa greškom do oko četiri procenta, a prava paralelnog do osam, sa najvećim odstupanjima na dva kraja. Paralelno jezgro pri $D = 1$ i $D = 2$ traži praktično isto, 19 495 naspram 19 404 LUT-a, pa se na donjem kraju ne ponaša linearno, a pri $D = 16$ prava zauzeće precenjuje za sedam procenata.

Nagib osnovnog jezgra iznosi 286,4 LUT-a po bitu obrade, što je polovina veličine polja u granici od pola procenta. Svaki dodatni bit obrade donosi po jedan 571-bitni parcijalni proizvod.

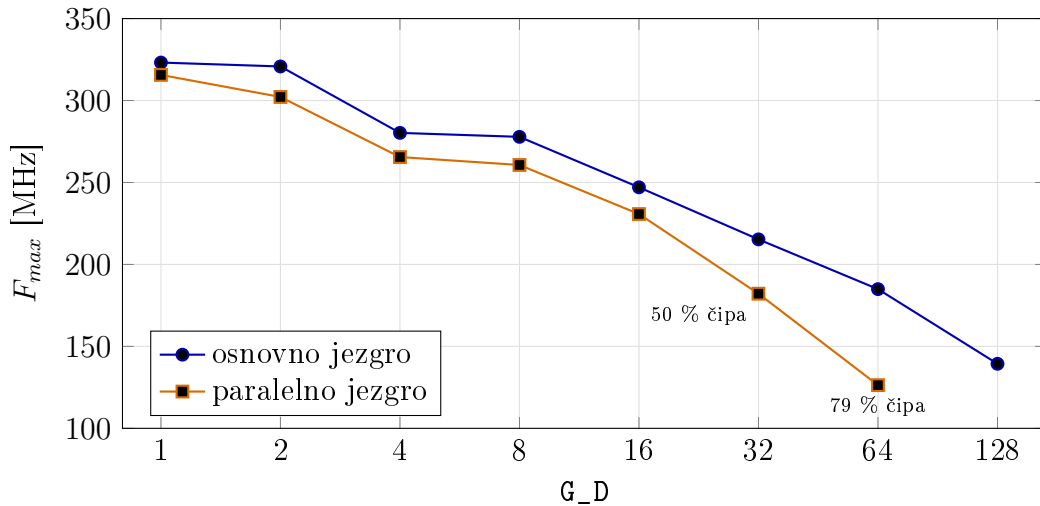
Odnos dva nagiba je 4,12, a to je upravo odnos broja množača. Osnovno jezgro ima jedan, u aritmetičkoj jedinici. Paralelno ih ima četiri, tri u iteraciji merdevina i jedan u sopstvenoj aritmetičkoj jedinici, koju koristi završna konverzija. Sve ostalo što raste sa D staje u preostala tri procenta. Deo koji od D ne zavisi veći je kod paralelnog jezgra oko sedam procenata, a u registrima je razlika stalna, oko četrdeset šest procenata više flipflopova, i od D ne zavisi. Odnos zauzeća i radne učestanosti za sve izmerene tačke prikazan je na slici 5.1.



Slika 5.1: Zauzeće i radna učestanost ECDH jezgra po vrednostima generika G_D . Uz tačku osnovnog jezgra sa $D = 128$ i paralelnog sa $D = 64$ označeno je i zauzeće u procentima čipa

Radna učestanost opada sa širinom obrade, ali znatno sporije nego što pada broj taktova množenja. Od $D = 1$ do $D = 128$ broj taktova po množenju padne 114 puta, sa 571 na 5, a učestanost osnovnog jezgra 2,3 puta, sa 323 na 139 MHz. Šira obrada zato množenje zaista skraćuje, čime je izmeren kompromis najavljen u odeljku 4.2.2. Koja je konfiguracija najbolja zavisi od toga šta se meri. Po zauzeću i radnoj učestanosti to je $D = 1$, koji je istovremeno i najmanji i najbrži po taktu, sa 17 426 LUT-ova i 323,17 MHz. Po vremenu jednog skalarnog množenja on je najslabiji. Već susedna tačka $D = 2$ po ta dva merila granično je lošija, 89 LUT-ova i 2,4 MHz, a trajanje skraćuje 1,94 puta, gotovo dvostruko.

Kritičnu putanju ne čini logika nego povezivanje. Kod osnovnog jezgra sa $D = 1$ logika troši 0,95 ns, a veze 2,42 ns, dakle 72 procenta putanje, i to pri zauzeću ispod petnaest procenata čipa. Radna učestanost jezgra nad poljem B-571 zato se ne može projektovati iz dubine logike, po čemu se ovo jezgro razlikuje od SHA-3 i AES-GCM akceleratora, koji zauzimaju po nekoliko procenata čipa, pa se zagušenje kod njih i ne javlja.



Slika 5.2: Radna učestanost po vrednostima generika G_D , sa zauzećem čipa uz dve tačke paralelnog jezgra

Gornju granicu mreže ne postavljaju vremenski uslovi nego kapacitet ploče. Paralelno jezgro sa $D = 128$ posle optimizacije traži 162 863 LUT-a, a čip ih ima 117 120, pa raspoređivanje odbija da počne i ta tačka u tabeli ne postoji. Uticaj zauzeća vidi se i pre same granice, na slici 5.2, poređenjem dva jezgra pri istoj širini obrade. Množač i njegov kombinacioni lanac su im isti, pa razlika u učestanosti pri istom D potiče od gustine na čipu. Do $D = 16$ paralelno jezgro zaostaje za osnovnim najviše sedam procenata. Pri $D = 32$, gde zauzima polovinu čipa, zaostatak je petnaest procenata, a pri $D = 64$, na četiri petine čipa, trideset dva procenta. Raspoređivanju tada ponestaje prostora i zagušenje veza postaje glavno ograničenje, ispred dužine samog lanca. Jači čip bi te dve pojave razdvojio, jer bi isti dizajn na njemu zauzimao manji udeo površine, pa bi se video čist uticaj širine obrade.

5.2.3 Latencija skalarnog množenja

Zauzeće i radna učestanost mere cenu jezgra, a ovaj odeljak meri šta se za tu cenu dobija. Koliko jedno skalarno množenje traje utvrđeno je takt-tačno u simulaciji, brojanjem ivica takta od one na kojoj jezgro prihvati zahtev do one na kojoj se pojavi znak kraja. Izlaz svakog prolaza poredi se sa afinim koordinatama iz referentnog modela, pa u tabelu ulazi samo trajanje uz koje je rezultat tačan. Merene su sve konfiguracije iz tabela 5.2 i 5.3, kao i broj taktova paralelnog jezgra sa $D = 128$, uz istu vrednost skalara, čija najviša jedinica stoji na 570. mestu, na najdužoj poziciji koju standardni ključ dopušta, pa poravnanje skalara u svakom redu košta isto.

Meri se samo jezgro, bez AXI4-Stream omotača, jer omotač na portove ne izvodi signal kojim jezgro označava kraj računa. Obim se time razlikuje od prethodnog odeljka, gde je merena površina celog IP-a. Razlika je poznata i mala. Omotač po jednom pozivu jezgra prenese komandnu reč i četiri koordinate, dve ulazne i dve

izlazne, po devet reči na magistrali od 64 bita, dakle trideset sedam taktova, što je ispod pola procenta i najkraćeg trajanja u tabeli.

Tabela 5.4: Trajanje jednog skalarnog množenja. Kolona $q = \lceil 571/D \rceil$ je broj digita koje množač obradi po jednom množenju u polju. Vreme t dobija se deljenjem broja taktova radnom učestanošću iz tabela 5.2 i 5.3, a poslednja kolona je odnos broja taktova dva jezgra pri istom D . Paralelno jezgro sa $D = 128$ nema vreme jer ne staje na ploču, pa mu se broj taktova zna iz simulacije, a radna učestanost ne postoji

G_D	q	osnovno jezgro		paralelno jezgro		odnos taktova
		taktova	$t [\mu s]$	taktova	$t [\mu s]$	
1	571	2 005 406	6205,4	670 468	2124,3	2,99
2	286	1 023 866	3191,7	338 728	1120,9	3,02
4	143	531 374	1896,0	172 276	648,8	3,08
8	72	286 850	1032,4	89 632	343,8	3,20
16	36	162 866	659,2	47 728	206,8	3,41
32	18	100 874	468,5	26 776	147,0	3,77
64	9	69 878	377,9	16 300	129,0	4,29
128	5	56 102	402,5	11 644	—	4,82

Broj taktova po iteraciji merdevina linearan je u q , a prave kroz izmerene tačke glase

$$T_{\text{iter}}^{\text{I}} = 6,04 \cdot q + 68, \quad (5.4)$$

$$T_{\text{iter}}^{\text{II}} = 2,04 \cdot q + 10, \quad (5.5)$$

Nagibi su šest i dva, čime je procena iz odeljka 4.2.4 potvrđena na obe realizacije. Slobodni član je ono što ta procena ne obuhvata. Osnovno jezgro troši šezdeset osam taktova po iteraciji na kvadriranja, sabiranja i pozive aritmetičke jedinice po svakoj od četrnaest operacija, a paralelno deset, jer kod njega kvadriranja i sabiranja teku u prolazu između dve runde množenja.

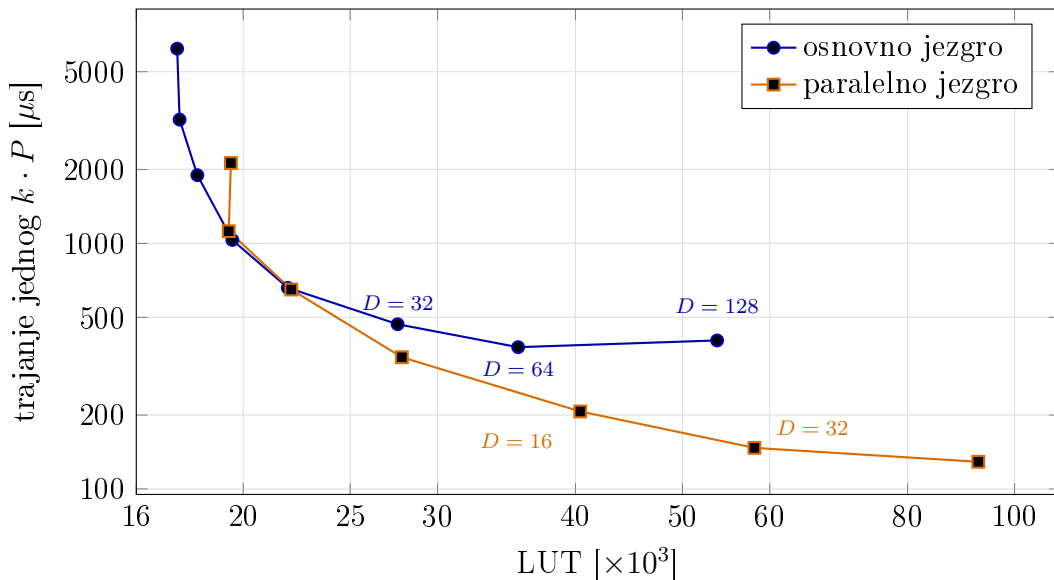
Time je objašnjeno i kretanje odnosa u poslednjoj koloni tabele 5.4. Pri $D = 1$ odnos iznosi 2,99, dakle upravo šest prema dva, jer je q toliko veliko da množenja odrede celu cenu. Sa porastom D broj digita opada, udeo slobodnog člana raste, a kako je on kod osnovnog jezgra skoro sedam puta veći, odnos se penje do 4,82 pri $D = 128$. Prednost paralelnog jezgra utoliko je veća ukoliko je obrada šira.

Izmereno trajanje popunjava i analitičku procenu iz odeljka 3.1.7. Ukupan broj taktova prati $T^{\text{I}} = 3444 \cdot q + 38882$ za osnovno i $T^{\text{II}} = 1164 \cdot q + 5824$ za paralelno jezgro, i oba izraza pogađaju svih šesnaest redova tabele bez ostatka. Nagib osnovnog jezgra tačno je broj množenja u polju, a razlika nagiba, $2280 = 570 \cdot 4$, dolazi otud što šest množenja iteracije na tri množača staju u vreme dva, pa svaka od 570 iteracija uštedi vreme četiri množenja. Slobodni član obuhvata i režiju iteracija i celu završnu konverziju sa jednom inverzijom. Pri $D = 64$ množenja tako nose $3444 \cdot 9 = 30996$ taktova, režija preostalih 38882, a zbir je upravo izmerenih 69878. Na 184,93 MHz to je 377,9 mikrosekundi po skalarnom množenju, dok paralelno jezgro pri istoj širini traži 129,0. Najbrža konfiguracija time opsluži oko 3875 razmena ključa u sekundi, jer 129,0 mikrosekundi daje 7751 množenje u sekundi, a razmena traži dva poziva jezgra.

Uz cenu iz tabela zauzeća sada se može reći koja konfiguracija najbolje troši površinu. Proizvod broja LUT-ova i trajanja najmanji je za paralelno jezgro sa $D = 16$, gde iznosi 8,4 miliona LUT-mikrosekundi, s tim što je susedna tačka $D = 32$

na 8,5 miliona, unutar šuma alata, pa je radna tačka paralelnog jezgra ceo opseg od 16 do 32. Najbolja vrednost osnovnog jezgra je 12,9 miliona pri $D = 32$, pa je paralelno jezgro trećinu povoljnije. Poređenje na bliskoj površini je neposrednije. Paralelno jezgro sa $D = 32$ zauzima 58 112 LUT-ova i završi za 147,0 mikrosekundi, a osnovno sa $D = 128$ zauzima 53 759 LUT-ova i završi za 402,5 mikrosekundi, pa za osam procenata više logike daje 2,7 puta kraće trajanje.

Ceo kompromis prikazan je na slici 5.3, gde svakoj konfiguraciji odgovara jedna tačka sa zauzećem na jednoj i trajanjem na drugoj osi. Povoljne konfiguracije leže bliže donjem levom uglu, jer troše manje i logike i vremena.



Slika 5.3: Zauzeće i trajanje jednog skalarnog množenja za sve merene konfiguracije. Označeno je pet tačaka o kojima govori tekst

Kod osnovnog jezgra širina obrade ima i granicu koja nije kapacitet ploče. Pri $D = 128$ ono traje 402,5 mikrosekundi, dakle duže nego pri $D = 64$ sa 377,9, a traži pedeset jedan procenat više LUT-ova, jer pad učestanosti sa 184,93 na 139,38 MHz nadjača uštedu u taktovima. Poslednji korak udvostručenja širine tu se ne isplati. Kod paralelnog jezgra poslednji korak koji staje na ploču, sa $D = 32$ na 64, još se isplati, jer trajanje opada sa 147,0 na 129,0 mikrosekundi.

5.3 SHA-3 akcelerator

5.3.1 Provera po nivoima

Ispravnost je proverena testovima sa poznatim odgovorima nad celim IP-om, za sve četiri SHA3 varijante, obe SHAKE funkcije i cSHAKE, u kombinacijama širine magistrale od 8 do 64 bita, sa jednom ili dve runde po taktu, a kod varijante SHA3-256 sa svih osam delilaca broja 24. Vektore proizvodi referentni model iz odeljka 5.1, nad porukama uzastopnih dužina, od jednog bajta do višeblokovnih, pa dopuna prođe kroz svaki položaj u reči i preko granice bloka. Softversko uokvirivanje KMAC funkcije provereno je nad KMAC256 i KMACXOF256 vektorima, čime je pokrivena podela posla iz odeljka 4.3.6, po kojoj uokvirivanje pripada strani koja podatke dovodi.

Namenski testovi gustog saobraćaja uz to drže ulaz spremnim u svakom taktu, šalju pakete bez razmaka i koče izlaz nasumičnim protivpritiskom, pa obuhvataju granične slučajeve upravljanja tokom, među njima i upis dopune u već pun blok, dok je `tready` spušten (odjeljak 4.3.2). Jezgro je provereno i na ploči, gde se sažeci porede sa vrednostima OpenSSL biblioteke.

5.3.2 Zauzeće resursa i radna učestanost

Obe arhitekture iz odeljka 4.3.4 implementirane su za sve četiri varijante standarda i obe širine magistrale za koje je merena i propusnost, 32 i 64 bita, ukupno četrnaest konfiguracija. Širine od 8 i 16 bita su izostavljene, jer su ulazno ograničene u svim varijantama, pa za poređenje arhitektura ne donose ništa novo. Kombinacija SHA3-224 sa magistralom od 64 bita ne postoji, jer sažetak od 224 bita nije deljiv širinom magistrale, pa se ta kombinacija odbacuje još pri elaboraciji. Rezultati su dati u tabelama 5.5 i 5.6.

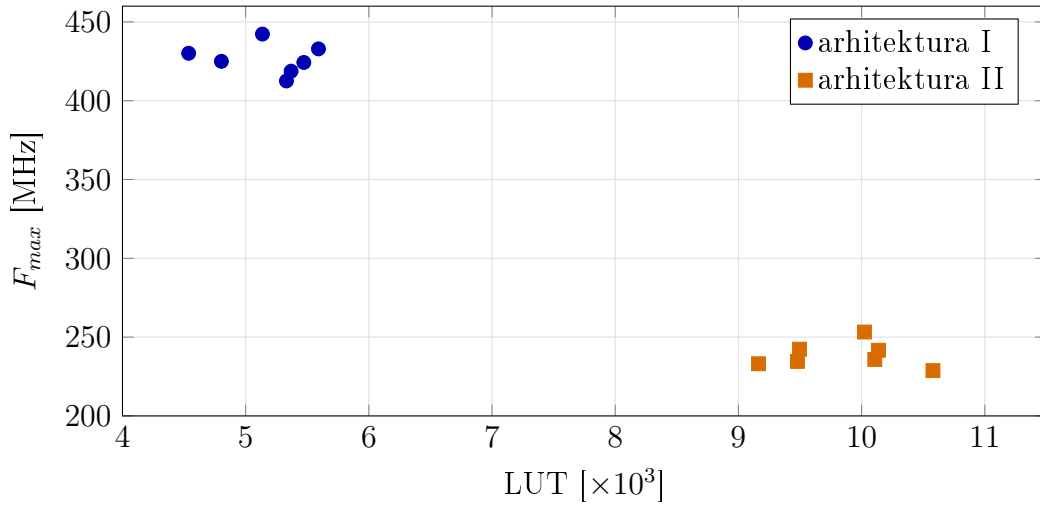
Tabela 5.5: Arhitektura I (jedna runda po taktu), implementacija van konteksta. Kolona usko grlo sledi iz modela u odeljku 5.3.3

Varijanta	DATA_WIDTH	LUT	FF	F_{max} [MHz]	usko grlo
SHA3-224	32	5369	3090	418,76	ulaz
SHA3-256	32	5331	3070	412,54	ulaz
	64	5591	3130	432,90	permutacija
SHA3-384	32	5136	2933	442,28	ulaz
	64	5472	3004	424,27	permutacija
SHA3-512	32	4537	2807	430,11	permutacija
	64	4803	2882	424,99	permutacija

Tabela 5.6: Arhitektura II (dve runde po taktu), implementacija van konteksta. Kolona usko grlo sledi iz modela u odeljku 5.3.3

Varijanta	DATA_WIDTH	LUT	FF	F_{max} [MHz]	usko grlo
SHA3-224	32	10107	3085	235,85	ulaz
SHA3-256	32	10579	3070	228,73	ulaz
	64	10137	3128	241,66	ulaz
SHA3-384	32	9496	2927	242,37	ulaz
	64	10023	3009	253,23	ulaz
SHA3-512	32	9163	2800	233,10	ulaz
	64	9479	2876	234,58	permutacija

Odnos dve arhitekture isti je u svih sedam parova. Arhitektura I postiže preko 400 MHz u svih sedam konfiguracija, a druga plaća od 40 do 46 procenata radne učestanosti, a njena cena u logici iznosi od 1,81 do 2,02 puta više LUT-ova, dakle blizu dvostruke površine. Broj flipflopova se pritom praktično ne menja, jer obe arhitekture drže isto stanje od 1600 bitova, pa je cena u logici, a ne u registrima. Manji *rate* znači uži ulazni bafer, pa SHA3-512 troši najmanje logike u obe arhitekture. Raspored svih četrnaest tačaka prikazan je na slici 5.4.



Slika 5.4: Zauzeće i radna učestanost svih četrnaest SHA-3 konfiguracija

5.3.3 Propusnost i model uskog grla

Propusnost je izmerena takt-tačno u simulaciji, nad porukom od pedeset blokova, uz ulaz spreman u svakom taktu i izlaz prihvaćen bez zastoja. Rezultati su dati u tabeli 5.7.

Tabela 5.7: Izmerena vrednost bita po taktu i propusnost izvedena iz nje pri F_{max} iz tabela 5.5 i 5.6

Konfiguracija	DATA_WIDTH	Arhitektura	Biti po taktu	Propusnost [Gb/s]	Usko grlo
SHA3-256	32	I	30,49	12,58	ulaz
SHA3-256	32	II	30,69	7,02	ulaz
SHA3-256	64	I	42,71	18,49	permutacija
SHA3-256	64	II	59,25	14,32	ulaz
SHA3-512	32	I	22,38	9,63	permutacija
SHA3-512	32	II	29,38	6,85	ulaz
SHA3-512	64	I	22,66	9,63	permutacija
SHA3-512	64	II	43,01	10,09	permutacija

Ponašanje se u celini objašnjava jednostavnim modelom. Po bloku od *rate* bitova troši se

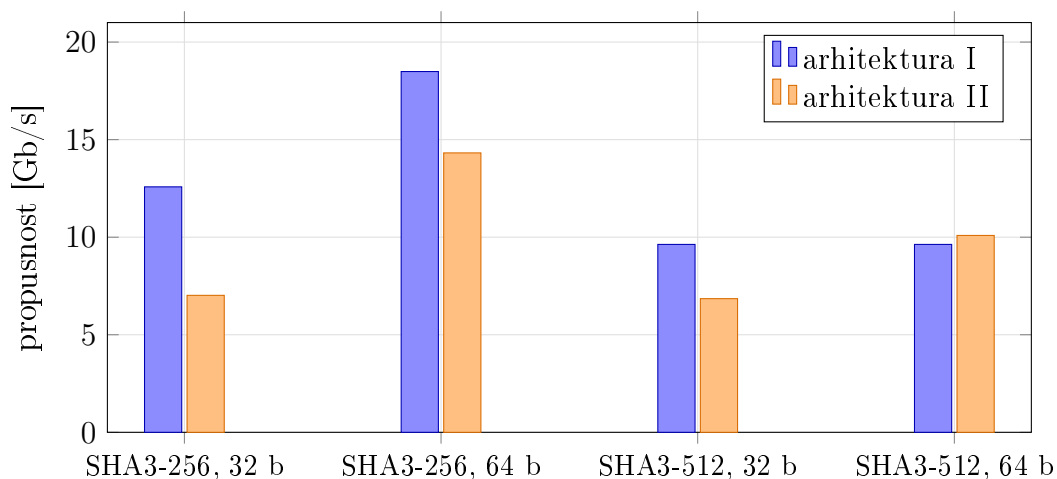
$$T_{blok} = \max\left(\frac{rate}{DATA_WIDTH}, \frac{24}{ROUNDS_PER_CYCLE}\right) + 1 \text{ takt}, \quad (5.6)$$

gde prvi član broji reči potrebne da se blok unese, a drugi trajanje permutacije. Maximum stoji zato što se punjenje narednog bloka i permutacija tekućeg preklapaju, kako je opisano u odeljku 4.3.1, pa brzinu određuje sporiji od njih. Sama pretpostavka o preklapanju proverena je zasebnim testom sa brojačima taktova, koji potvrđuje da se reči primaju tokom permutacije, da se naredni iscedeni deo permutuje dok se prethodni iznosi i da reči naredne poruke ulaze dok izlaz prethodne još teče, a sažeci se pritom i dalje poklapaju sa referentnim.

Dodatni takt je predaja bloka i plaća se na obe strane, zbog čega i stoji izvan maksimuma. Kada je usko grlo unos, ulazni bafer posle poslednje reči spušta **tready** i u tom taktu predaje popunjen blok jezgru umesto da prima novu reč, po odeljku 4.3.2. Kada je usko grlo permutacija, jezgro posle dvadeset četvrte runde ne ulazi pravo u narednu, nego jedan takt čeka blok. Kod SHA3-256 sa magistralom od 32 bita to je $34 + 1 = 35$ taktova, a kod iste varijante sa magistralom od 64 bita, gde permutacija preuzima ulogu uskog grla, $24 + 1 = 25$. Primenjen na svih četrnaest konfiguracija, model ulaz proglašava uskim grlom u devet, a permutaciju u pet, kako je opisano u tabelama 5.5 i 5.6.

Izmerene vrednosti uz cenu po bloku nose i završnu permutaciju sa ispisom sažetka, između sedamnaest i trideset pet taktova, koja se plaća jednom po poruci. Kada se i ona uračuna, model daje izmereni broj taktova tačno, pa je za SHA3-256 sa magistralom od 32 bita u prvoj arhitekturi $50 \cdot 35 + 33 = 1783$ takta, koliko je i izmereno. Kolona bita po taktu potiče iz istog merenja, a od količnika *rate* i cene bloka razlikuje se zato što je merna poruka za jednu reč kraća od pedeset punih blokova, da bi dopuna ostala unutar poslednjeg bloka. Sa dužinom poruke udeo završnog repa opada, pa propusnost teži vrednosti od jedan do tri procenta višoj od navedene u tabeli 5.7.

Iz modela sledi i pravilo za izbor arhitekture, strože nego što se na prvi pogled čini. Druga arhitektura prepolovljuje broj taktova permutacije, ali ne i vreme, jer sa dužom kritičnom putanjom radna učestanost pada od 40 do 46 procenata. Da bi ukupna propusnost porasla, dobitak u bitima po taktu mora da nadmaši taj gubitak, dakle da bude veći od približno 1,8 puta, a njega uopšte nema tamo gde granicu postavlja ulaz. Kod SHA3-256 sa magistralom od 32 bita cena bloka ostaje 35 taktova u obe arhitekture, pa druga gubi celih 44 procenta propusnosti. Kod SHA3-256 sa 64 bita i SHA3-512 sa 32 bita permutacija jeste usko grlo prve arhitekture, ali skraćenje donosi svega 1,4 odnosno 1,3 puta, pa druga ostaje 23 odnosno 29 procenata ispod prve. Prag prelazi jedino SHA3-512 sa 64 bita, gde dobitak u bitima po taktu iznosi 1,9 puta, po modelu 25 naspram 13 taktova po bloku, pa je druga arhitektura ispred, 10,09 naspram 9,63 Gb/s, za nepunih pet procenata, razliku unutar šuma alata, i po ceni dvostruke logike. Za SHA3-224, čija propusnost nije merena, isti zaključak daje sam model. Tamo je *rate* najveći (1152 bita), pa unos bloka traje duže od permutacije, njeno skraćivanje ne menja ništa, a niža učestanost ostaje čist gubitak. Odnosi za sve četiri merene kombinacije prikazani su na slici 5.5.



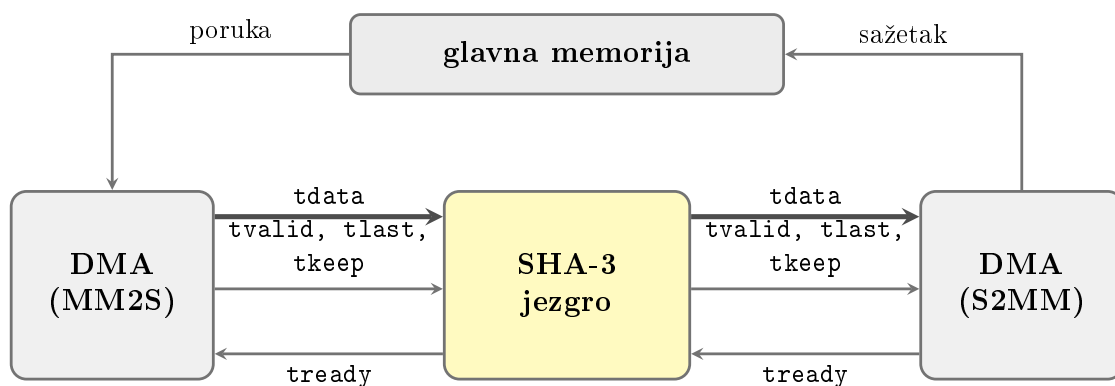
Slika 5.5: Propusnost dve arhitekture za četiri merene kombinacije varijante i širine magistrale, pri izmerenoj radnoj učestanosti

Za kratke poruke merodavna je latencija, a ne propusnost. Za poruku od šesnaest bajtova ona iznosi 39 taktova kod SHA3-256 pri širini od 32 bita u prvoj arhitekturi, odnosno 21 takt pri širini od 64 bita u drugoj. U profilu koji odgovara izvođenju ključeva KMAC okvir iz odeljka 3.2.7 nosi tri bloka, prefiksni, blok ključa i blok sa tajnom Z , a ključ od 256 bitova staje u prvo ceđenje. Izvođenje ključa zato staje u tri cene bloka iz jednačine (5.6) uz rep poruke, $3 \cdot 18 + 17 = 71$ takt pri širini od 64 bita u drugoj arhitekturi, a najviše $3 \cdot 35 + 33 = 138$ taktova u prvoj pri 32 bita, jer treći blok nije pun. Na izmerenim učestanostima to je oko tri desetinke mikrosekunde, zanemarljivo u odnosu na trajanje razmene ključeva, pa izbor konfiguracije za tu ulogu nije kritičan. Određuje ga AES-GCM strana sistema.

5.3.4 Upravljački program i merni put

Merenja na ploči obuhvataju ceo put od datoteke do sažetka, pa ono što se meri nije samo jezgro nego i softver oko njega. Za potrebe ovog rada napisan je upravljački program jezgra operativnog sistema (engl. *kernel module*) za PetaLinux, zajedno sa dva korisnička programa koji ga koriste, jer se bez tog sloja jezgro sa procesorske strane ne može pokrenuti.

Program se sa jezgrom povezuje preko opisa uređaja u stablu uređaja (engl. *device tree*) i korisniku nudi dve datoteke uređaja (engl. *character device*), jednu za slanje poruke i jednu za prijem sažetka. Prenos vodi kroz podsistem DMA Engine jezgra operativnog sistema, a ne programiranjem registara kontrolera. Put podataka u tim merenjima prikazan je na slici 5.6.



Slika 5.6: Put podataka u merenjima na ploči: kanal MM2S čita poruku iz glavne memorije i pretvara je u AXI4-Stream tok, a kanal S2MM sažetak vraća u memoriju. Procesor u samom prenosu ne učestvuje

Tri svojstva tog puta određuju izmerene brojeve.

- Cela datoteka prenosi se u jednom DMA prenosu, jer granicu poruke označava signal **tlast**, na kome jezgro pokreće dopunu. Kako datoteka ne staje u jedan neprekidan blok memorije, koristi se *scatter-gather* DMA preko liste blokova.
- Prijemni kanal se priprema pre slanja, jer bi sažetak inače mogao stići pre nego što ga ima ko primiti.
- Baferi se zauzimaju po pozivu i oslobađaju po završetku. Ta se cena na malim datotekama vidi, a na velikim amortizuje, što je i izmereno u narednom odeljku.

Uz upravljački program napisana su i dva korisnička programa, koja odgovaraju dvama načinima pristupa.

Sa preslikavanjem bafere. Program traži od upravljačkog programa da zauzme bafere, preslikava ih u svoj adresni prostor pozivom `mmap` i sam kopira sadržaj datoteke u njih, pa tek onda pokreće prenos. Prenos i njegovo čekanje razdvojeni su u dva poziva, tako da se prijemni kanal pripremi pre slanja.

Bez kopiranja. Program prosleđuje samo deskriptor otvorene datoteke, a sve ostalo obavlja upravljački program, koji sam određuje veličinu, zauzima bafere, čita datoteku pravo u njih i vodi oba prenosa. Ceo posao staje u jedan poziv `ioctl`, a podatak nijednom ne prolazi kroz korisnički prostor.

Razlika između ta dva načina merljiva je i data u narednom odeljku.

5.3.5 Rad na ploči i poređenje sa softverom

Merenja su izvedena nad datotekama od 1, 10 i 100 megabajta, za varijantu SHA3-512, na jezgru druge arhitekture, na taktu od 230 MHz uz magistralu od 64 bita, odnosno 220 MHz uz 32 bita. Kao softverska osnova korišćene su biblioteka OpenSSL iz PetaLinux distribucije i namenska implementacija u jeziku C, napisana za ovaj rad i prilagođena procesoru Cortex-A53 spajanjem koraka θ , ρ i π u jedan prolaz, čime se smanjuje broj pristupa memoriji. Oba softverska merenja su jednonitna, a namenska realizacija prevedena je uz optimizaciju `-O3` i podešavanje za Cortex-A53. Rezultati za najveću datoteku dati su u tabeli 5.8.

Ono što tabela poredi jeste ceo sistem, a ne jezgra međusobno. Svaki red je merenje pod svojim uslovima, pa se iz njega ne može čitati koja je arhitektura brža, jer je to poređenje dato u odeljcima 5.3.2 i 5.3.3, gde su sve konfiguracije merene istim postupkom.

Tabela 5.8: Propusnost pri heširanju datoteke od 100 MiB, varijanta SHA3-512

Realizacija	Vreme [s]	Propusnost [MiB/s]	Prema OpenSSL-u
akcelerator, arhitektura II, magistrala 64 bita	0,204	490,7	9,74
akcelerator, arhitektura II, magistrala 64 bita, sa kopiranjem u korisnički prostor	0,248	403,6	8,01
akcelerator, arhitektura II, magistrala 32 bita	0,246	406,9	8,07
OpenSSL	1,984	50,4	1,00
namenska implementacija u C-u	2,252	44,4	0,88

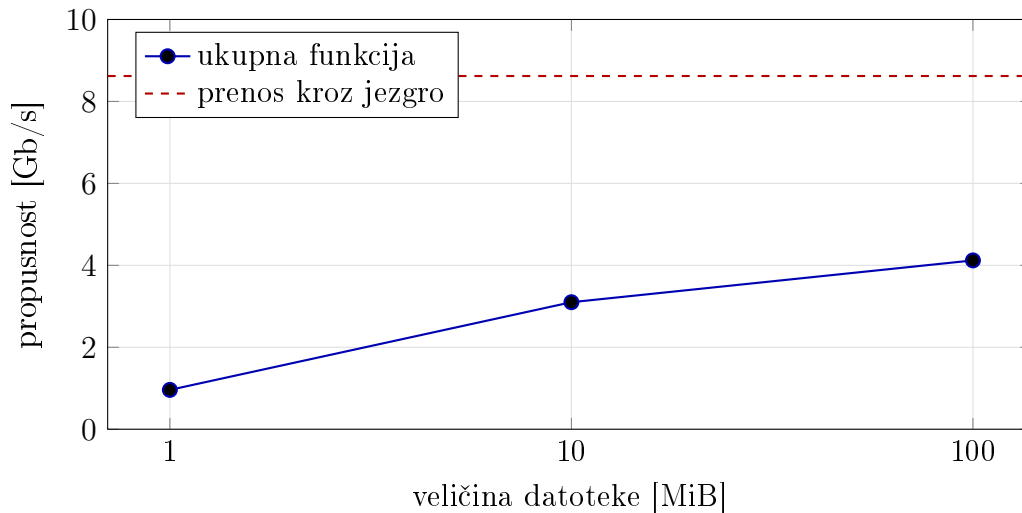
Rezultate treba čitati uz tri napomene, jer bi bez njih navedeno ubrzanje bilo pogrešno shvaćeno.

Odsustvo namenskih instrukcija. Jezgro Cortex-A53 pripada arhitekturi ARMv8.0-A, čije kriptografsko proširenje pokriva AES i funkcije SHA-1 i SHA-256, ali ne i SHA-3. Namenske instrukcije za SHA-3 uvedene su tek sa ARMv8.2-A, pa se ovde SHA-3 izvršava isključivo opštim instrukcijama. Ubrzanje od 9,74 puta zato važi za ovu platformu, a ne za odnos hardvera i softvera uopšte.

Merilo sistema naspram merila akceleratora. Tri vrednosti treba razlikovati. Jezgro na taktu ploče od 230 MHz može najviše 9,89 Gb/s, po 43,01 bita po taktu iz odeljka 5.3.3. Prenos kroz jezgro postiže 8,62 Gb/s, pa DMA drži tok popunjenim oko 87 procenata vremena. Ceo sistem, od datoteke do sažetka, ostvaruje 0,96, 3,10

i 4,12 Gb/s za datoteke od 1, 10 i 100 mebibajta, gde 4,12 Gb/s odgovara vrednosti od 490,7 MiB/s iz tabele, jer mebibajt nosi 2^{20} bajtova. Razlika između prenosa kroz jezgro i celog sistema odlazi na čitanje datoteke, zauzimanje i pripremu ulaznog i izlaznog bafera i komunikaciju sa jezgrom operativnog sistema. Na datoteci od 100 megabajta prenos zauzima 45,6 procenata ukupnog vremena, pa i pri toj veličini sistem daje manje od polovine onoga što prenos može, kako pokazuje slika 5.7.

Uticaј načina pristupa. Jedina dva reda tabele koja se razlikuju u tačno jednoj stvari jesu ona sa kopiranjem i bez njega, jer im je sve ostalo isto. Uklanjanje kopiranja u korisničkom prostoru donosi nepunih 22 procenta, 490,7 naspram 403,6 MiB/s, čime se meri koliko softverski deo puta nosi.



Slika 5.7: Propusnost u zavisnosti od veličine datoteke

5.4 AES-GCM akcelerator

5.4.1 Provera po nivoima

Ispravnost je proverena testovima sa poznatim odgovorima po [9], nad celim akceleratorom. Vektori pokrivaju pakete sa pridruženim podacima, nepotpun poslednji blok i dešifrovanje sa proverom oznake. Ispravnost razmene preko AXI4-Stream interfejsa proverena je i pod zastoјima na oba kraja toka. Regresija broji 36 testova nad jezgrom i omotačima, među njima i promenu ključa usred paketa, a nad celim lancem još 209 slučajeva u četiri skupa, od poznatih odgovora sa prolazom po dužinama do petlje šifrovanje-dešifrovanje. Pun IPsec par uz to prolazi 120 od 120 krugova, u kojima se paket šifruije, dešifruije i vraća bajt-identičan. Serije teku pod GHDL-om, a testovi sa spoljnim imenima signala i merenja celog puta pod Vivado simulatorom.

Akcelerator je proveren i u postojećem mrežnom sistemu, u kome instrument MTS 5800 služi kao generator saobraćaja, a paketi prolaze kroz sistemski omotač na vezi od 10 Gb/s. Veza je sporija od jezgra, pa propusnost ograničava ona, a ne akcelerator. Svrha te provere nije brojka nego verodostojnost, jer jezgro šifruije saobraćaj kakav na mreži zaista postoji, sa paketima promenljivih dužina i razmaka.

5.4.2 Zauzeće resursa i radna učestanost

Merenja idu po tri nivoa. Najpre samo AES jezgro, zatim GHASH jedinica i GCM sloj koji je vezuje za jezgro, a ceo put podataka sa sistemskim omotačem meri se u odeljku 5.4.7. AES jezgro implementirano je u šesnaest konfiguracija, koje pokrivaju obe arhitekture, obe realizacije runde i oba stila upravljanja tokom, a kod višejezgarne i šest brojeva jezgara. Uz njih su implementirane GHASH jedinica i GCM sloj, u obe vremenske organizacije. Rezultati su dati u tabelama 5.9, 5.10 i 5.11. Propusnost je izvedena po jednačini (5.1), iz modelskih bita po taktu samog jezgra, koji za protočnu arhitekturu iznose 128, a za višejezgarnu $128 \cdot \text{NUM_CORES} / (N_r + 1)$ po jednačini (4.1), a merenja iz odeljka 5.4.4 ih potvrđuju unutar procenta.

Tabela 5.9: Protočna arhitektura, implementacija van konteksta

Realizacija runde	Upravljanje tokom	LUT	FF	BRAM	F_{max} [MHz]	Gb/s
BRAM varijanta (T-tabele)	GLOBAL	3339	5321	52	297,97	38,14
	PER_STAGE	3457	5414	52	299,04	38,28
LUT varijanta	GLOBAL	10491	5320	0	401,12	51,34
	PER_STAGE	10579	5413	0	406,50	52,03

Tabela 5.10: Višejezgarne arhitekture, implementacija van konteksta

Realizacija runde	Jezgara	LUT	FF	BRAM	F_{max} [MHz]	Gb/s
BRAM varijanta (T-tabele)	1	1925	1843	4	307,03	2,62
	2	2955	2239	8	314,56	5,37
	4	4994	3024	16	288,85	9,86
	7	8077	4204	28	282,97	16,90
	14	15358	6960	56	278,47	33,27
	15	16365	7355	60	263,92	33,78
LUT varijanta	1	2274	1843	0	388,80	3,32
	2	3618	2239	0	373,12	6,37
	4	5216	3024	0	365,10	12,46
	7	8345	4205	0	339,44	20,28
	14	15899	6960	0	296,12	35,38
	15	16960	7356	0	301,93	38,65

Realizacija runde je kompromis između logike, memorijskih blokova i učestanosti. Protočna arhitektura u BRAM varijanti troši 3339 LUT-ova i 52 memorijska bloka na 298 MHz, a u LUT varijanti 10491 LUT, tri puta više, ali bez ijednog memorijskog bloka i na 401 MHz. Blok memorija dakle štedi logiku, a košta četvrtinu učestanosti, jer je čitanje iz memorije sporije od plitke kombinacione mreže. Kod višejezgarne arhitekture dve realizacije završavaju na skoro istom broju LUT-ova, a razlog je dat u odeljku 5.4.3. Poređenje stilova upravljanja tokom dato je u odeljku 5.4.5.

Tabela 5.11: GHASH jedinica i GCM sloj, implementacija van konteksta

Modul	MULT_CYCLES	LUT	FF	F_{max} [MHz]	granica [Gb/s]
GHASH jedinica	1	4707	390	262,86	33,65
	2	4382	960	348,51	22,30
GCM sloj	1	5197	1023	227,46	29,12
	2	5114	1593	302,18	19,34

GHASH jedinica u dvotaktnoj organizaciji skraćuje kritičnu putanju za četvrtinu, sa 3,80 na 2,87 ns, a isto važi i za ceo GCM sloj, sa 4,40 na 3,31 ns. Cena je registrarska

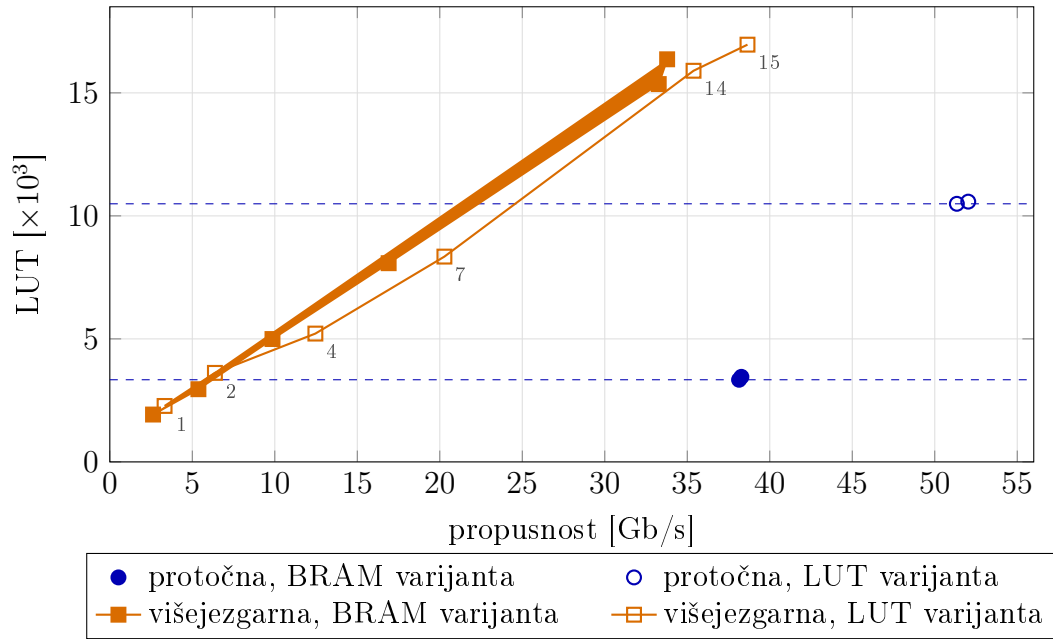
banka koja preseca množačku mrežu, oko 570 flipflopova više. Granica u bitima po taktu se prepolovljava po konstrukciji, ali granica u apsolutnim jedinicama pada samo na dve trećine, sa 33,65 na 22,30 Gb/s, jer dvotaktna organizacija radi na trećinu višoj učestanosti. Šta ta organizacija znači na celom putu podataka, dato je u odeljku 5.4.7.

5.4.3 Poređenje protočne i višejezgarne arhitekture

Obe arhitekture dostižu blok po taktu, pa se razlika svodi na cenu kojom se do njega stiže. Poređenje se i ovde odnosi na samo AES jezgro, bez autentifikacionog puta. Protočna arhitektura u BRAM varijanti taj protok drži sa 3339 LUT-ova. Višejezgarnoj je za isti protok potrebno petnaest jezgara, jer svako jezgro drži blok $N_r + 1 = 15$ taktova, pa u istoj varijanti troši 16365 LUT-ova, 4,9 puta više. Kada se propusnost svede na utrošenu logiku, protočna arhitektura daje pet i po puta više megabita u sekundi po jednom LUT-u u BRAM varijanti, a dvostruko više u LUT varijanti. Zauzeće višejezgarne pritom raste gotovo linearno, od jednog do petnaest jezgara za oko 1030 LUT-ova po jezgru u BRAM i oko 1050 u LUT varijanti. Svako jezgro uz logiku runde nosi i sopstveni raspored ključeva u distribuiranoj memoriji (odeljak 4.4.2.3), koji se u zauzeću vidi kao 74 LUT-a po jezgru, manje od desetine cene jezgra.

Kod višejezgarne arhitekture dve realizacije runde završavaju skoro izjednačene po zauzeću, iako T-tabele u blok memoriji i ovde štede logiku glavnih rundi. Uštedu poništava završna runda, koja u BRAM varijanti u svakom jezgru nosi zaseban sloj od šesnaest S-boksova, dok u LUT varijanti prolazi bez sopstvenog (odeljak 4.4.2.5). Na petnaest jezgara BRAM varijanta zato staje u svega šest stotina LUT-ova manje, a za to plaća 60 memorijskih blokova i za oko 40 MHz nižu učestanost. Za višejezgarnu arhitekturu je LUT varijanta prirodan izbor.

Prednost višejezgarne organizacije je granulacija. Broj jezgara je generik, pa se propusnost bira pri sintezi i plaća srazmerno cilju, dok protočna arhitektura postoji samo kao cela. Na dnu opsega jedno jezgro u BRAM varijanti staje u 1925 LUT-ova i četiri memorijska bloka, a LUT varijanta za 349 LUT-ova više izbegava i ta četiri bloka. Protočna arhitektura i tada traži ceo niz od 10491 LUT ili 52 bloka, uz 5320 flipflopova naspram 1843 koliko nosi jedno jezgro. Svako jezgro donosi petnaestinu punog protoka po ceni od oko 1050 LUT-ova, pa je za ciljeve do 20,28 Gb/s, dokle stiže sedam jezgara, višejezgarna arhitektura jeftinija od cele protočne po oba zauzeća, 8345 naspram 10491 LUT i 4205 naspram 5320 flipflopova. Iznad te granice svaka naredna konfiguracija dostiže zauzeće uporedivo sa protočnim ili veće, a protok joj ostaje manji, pa protočna arhitektura postaje sve efikasnija. Na četrnaest jezgara višejezgarna arhitektura troši 15899 LUT-ova, oko 50 procenata više od protočne, a dostiže 35,38 Gb/s, dok protočna sa 10491 LUT daje 51,34 Gb/s. Sve konfiguracije prikazane su na slici 5.8.



Slika 5.8: Zauzeće i propusnost svih šesnaest AES konfiguracija. Brojevi uz tačke višezvezgarnе LUT varijante su broj jezgara, isprekidane linije su nivoi zauzeća protočne arhitekture

5.4.4 Propusnost u ustaljenom stanju

Propusnost je merena istim postupkom kao kod SHA-3 jezgra, takt-tačno u simulaciji, nad porukom od 512 reči širine 128 bitova, bez pridruženih podataka, dakle nad osam kilobajta korisnog sadržaja, uz ulaz koji uvek ima spremnu reč i izlaz uvek spreman da je primi. Prozor merenja počinje prvom prihvaćenom ulaznom rečju, a zatvara se izlaznom rečju oznake. Ključ i nonce zadaju se pre prve reči, pa priprema potključa H i početak brojačkog niza ostaju ispred prozora. Meri se dakle ustaljeno stanje, a ne latencija prvog bloka. Oznaka tako ulazi u izmereno vreme, ali ne i u brojilac, jer se broj bitova obrađenih u taktu računa nad korisnim sadržajem, pa poruka od 512 reči obrađena kroz 516 taktova daje 127,01. Na putu dešifrovanja je obrnuto, oznaka stiže sa ulazom, pa je izlaz za reč kraći. Izmerene vrednosti i model uporedo daje tabela 5.12.

Tabela 5.12: Izmerena propusnost puta šifrovanja u ustaljenom stanju, uz jednotaktnu GHASH jedinicu. Ulaz je poruka od 512 reči po 128 bitova, jezgro se meri bez sistemskog omotača

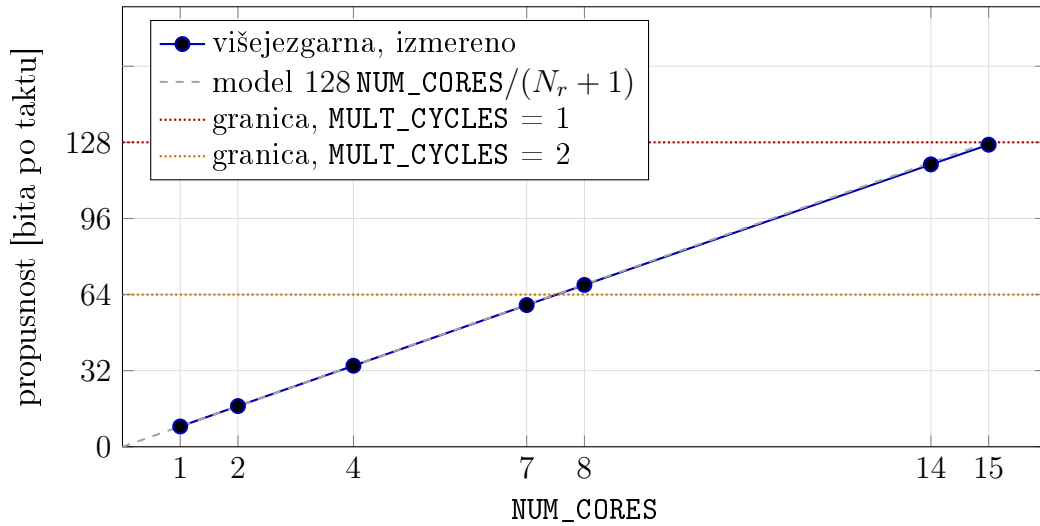
Konfiguracija	Taktova	Biti po taktu	Model
Protočna	516	127,01	128,00
Višejezgarna, 1 jezgro	7670	8,54	8,53
Višejezgarna, 2 jezgra	3831	17,11	17,07
Višejezgarna, 4 jezgra	1923	34,08	34,13
Višejezgarna, 7 jezgara	1100	59,58	59,73
Višejezgarna, 8 jezgara	963	68,05	68,27
Višejezgarna, 14 jezgara	552	118,72	119,47
Višejezgarna, 15 jezgara	516	127,01	128,00

Granice propusnosti akceleratora određuju tri nezavisna ograničenja, od kojih je merodavno najmanje:

$$B \approx 128 \cdot \min\left(B_{\text{sifra}}, \frac{1}{\text{MULT_CYCLES}}, 1\right) \text{ biti po taktu}, \quad (5.7)$$

gde je B_{sifra} jednako jedinici za protočnu arhitekturu, odnosno $\text{NUM_CORES}/(N_r + 1)$ za višejezgarnu, kao prvi član minimuma iz jednačine (4.1). Drugi član je autentifikacioni put, a treći jedan izlazni port. Odstupanje izmerenih vrednosti od modela nigde ne prelazi jedan procenat. Kod jednog i dva jezgra izmerena vrednost je za delić procenta i iznad modela. Model je asimptotski, a brojački blokovi ne zavise od podataka, pa raspodela prvo jezgro pokrene i pre prve prihvaćene reči (odeljak 4.4.2.2). Glava obrade tako ostane ispred prozora merenja, koji za 512 blokova broji 7670 taktova umesto modelskih $512 \cdot 15 = 7680$. Poreklo odstupanja vidi se na protočnom redu tabele. Izmerenih 516 taktova čine 512 taktova prijema i rep od četiri takta, u kome se obave poslednje množenje u GHASH jedinici, množenje nad blokom dužina i predaja reči oznake. Rep je toliko kratak zato što AES u CTR režimu šifruje brojačke blokove, koji ne zavise od podataka, pa niz za šifrovanje teče ispred poruke i poslednja reč korisnog sadržaja ne čeka ništa osim završnih koraka autentifikacije.

Model se najbolje vidi na slici 5.9. Šifarski put raste linearno sa brojem jezgara sve dok ne naiđe na granicu koju postavlja GHASH jedinica, a preko toga dodavanje jezgara ne donosi ništa.



Slika 5.9: Propusnost višejezgarne arhitekture u zavisnosti od broja jezgara

Da je granica zaista na autentifikacionom putu, a ne na šifarskom, pokazuje tabela 5.13, koja isto opterećenje propušta kroz obe vremenske organizacije GHASH jedinice iz tabele 4.9.

Tabela 5.13: Propusnost pri obe vremenske organizacije GHASH jedinice

Konfiguracija	MULT_CYCLES = 1	MULT_CYCLES = 2
Protočna	127,01	63,69
Višejezgarina, 15 jezgara	127,01	63,69
Višejezgarina, 8 jezgara	68,05	63,69

Iz gornje tabele slede tri zaključka. Prvo, dvotaktni množak prepolovljava granicu u bitima po taktu, tačno kako model granice predviđa, i to bez obzira na broj šifarskih jezgara iza njega. To nije samo gubitak. Kraća kombinaciona putanja postiže viši takt, 302,18 naspram 227,46 MHz na GCM sloju po tabeli 5.11, pa je dvotaktna organizacija prirodan izbor kada cilj propusnosti staje ispod prepolovljene granice, jer se isti cilj tada postiže uz blaže vremenske uslove i sa manje jezgara. Drugo, petnaest iterativnih jezgara daje istu vrednost kao protočna arhitektura, u takt, čime se potvrđuje da su dve arhitekture zamenljive po propusnosti u taktu. U bitima u sekundi zamenljive nisu, jer višejezgarina ne dostiže učestanost protočne, kako je ranije izmereno. Treće, osam jezgara je pri dvotaktnoj jedinici dovoljno, jer njihovih 68,05 bita po taktu premašuje granicu od 63,69, pa se razlika ne vidi. Konfiguracije se zato biraju u paru sa vremenskom organizacijom GHASH jedinice, a ne nezavisno od nje.

5.4.5 Poređenje stilova upravljanja tokom

Implementacija je pokazala da upravljanje tokom po stepenima (PER_STAGE) košta više logike od zajedničkog (GLOBAL), a radnu učestanost ne menja merljivo. Ostalo je pitanje da li tu cenu opravdava ponašanje pod zastojem, koje sinteza ne može da izmeri. Zato je isto opterećenje propušteno kroz deset obrazaca zastoja. U prvoj grupi izlazna strana prima reč sa zadatom verovatnoćom po taktu, od 90 do 25 procenata. U drugoj se tako prigušuju oba kraja toka. U trećoj izlazna strana

naizmenično prima zadati broj taktova pa zadati broj taktova stoji, sve do zastoja dužih od celog protočnog niza.

Nijedan obrazac ne razdvaja dva stila upravljanja, i to ne približno nego u takt. Naizmenični obrasci su uvedeni baš zato što nasumično prigušivanje po taktu nikada ne napravi dovoljno dug zastoj da se razlika pokaže, ali je ne pokazuju ni oni, uključujući zastoje od šesnaest uzastopnih taktova, koliko protočni niz ima slobodnih mesta, i dvostruko duže od toga.

Objašnjenje je u samoj protočnoj arhitekturi. Njeno dopuštenje takta uslovljeno je time da je ili izlaz spreman, ili protok još nije pun, pa glava protoka nastavlja da prima podatke i za vreme zastoja na izlazu, sve dok se ne popuni. Zajedničko upravljanje tokom tako bez dodatne logike obavlja glavni deo posla zbog kojeg je stil po stepenima uveden, punjenje niza za vreme zastoja na izlazu. Ostaje mu nedostupno jedino sabijanje mehura unutar niza, a nijedno merenje tu razliku ne pokazuje, pa je zajedničko upravljanje podrazumevani izbor.

Propusnost pritom verno prati raspoloživost izlaza. Kada izlaz prima polovinu taktova, dobija se polovina pune vrednosti, kada prima četvrtinu, četvrtina, a kada prima tri četvrtine taktova, tri četvrtine pune vrednosti. Akcelerator ni u jednom obrascu nije usko grlo, nego propušta onoliko koliko mu okolina dozvoli.

5.4.6 Mrežni paketi

Vrednosti iz ustaljenog stanja važe za duge poruke, a paketi na mreži su kratki. Svaki od njih nosi režiju. Novi nonce menja ceo brojački niz, pa se na granici paketa zatečeni višak starog niza obriše u jednom taktu, ali protočni niz time ostane prazan, i prva reč novog paketa čeka da novi blok J_0 i prvi brojački blok prođu kroz ceo niz, dok izlaz pre toga nosi još reč oznake prethodnog paketa. Potključ H u toj režiji ne učestvuje, jer zavisi samo od ključa, pa ga omotač čuva dok se ključ ne promeni. Zato je ista serija ponovljena nad paketima mrežnih veličina, sa ukupnim količinama od četiri do dvanaest kilobajta. Rezultati su u tabeli 5.14.

Tabela 5.14: Propusnost pri mrežnim veličinama paketa, protočna arhitektura

Paket	Reči	Paketa	Biti po taktu	Udeo granice
64 B	4	64	22,49	17,7 %
512 B	32	16	81,82	64,4 %
1504 B	94	8	108,27	85,2 %
duga poruka	512	1	127,01	100 %

Sve redove sa više paketa objašnjava jedna veličina, režija po paketu od oko osamnaest taktova. Ona se čita neposredno iz merenja. Šezdeset četiri paketa od po četiri reči obrađena su kroz 1457 taktova, višak od 18,8 taktova po paketu, a osam paketa od po 94 reči kroz 889 taktova, višak od 17,1. Za pakete od 1504 bajta taj višak znači 85 procenata pune vrednosti, pa realan saobraćaj radi blizu granice akceleratora. Kod paketa od 64 bajta režija nadjača koristan rad od svega četiri reči i propusnost padne na nepunu petinu. Šta od ove režije ostane na celom putu podataka, pokazuje odeljak 5.4.7.

Praktična posledica je da se puna vrednost akceleratora dobija na dugim paketima. Pri najkraćim paketima merodavna je režija po paketu, a ona potiče od samog režima rada, jer svaki novi nonce traži nov brojački niz.

5.4.7 Ceo put podataka

Dosadašnja merenja obuhvatala su AES jezgro, GHASH jedinicu i GCM sloj svaki za sebe. Ostaje ono što bi se u sistem zaista ugradilo, ceo put podataka zajedno sa sistemskim omotačem iz odeljka 4.4.5, i to u oba smeru. Zaglavlje je dimenzionirano po ESP-u u tunelskom režimu preko Ethernet-a i IPv4, pa se 34 bajta prenose nepromenjena, a 16 bajtova ESP zaglavlja i eksplicitnog inicijalizacionog vektora ulazi u pridružene podatke. Zauzeće, učestanost i propusnost oba smeru daje tabela 5.15.

Tabela 5.15: Ceo put podataka nad paketima ESP tunela. Red *GCM lanac* je GCM sloj sa AES jezgrom, red *sa omotačem* isti lanac uz razdvajanje, sastavljanje i poravnanje oznake. Obe arhitekture su u LUT varijanti, a GHASH jedinica je jednotaktna

Nivo	Smer	višejezgarna, 15 jezgara			protočna		
		LUT	F_{max} [MHz]	Gb/s	LUT	F_{max} [MHz]	Gb/s
GCM lanac	šifrovanje	21908	223,14	28,56	15510	224,27	28,71
	dešifrovanje	22210	221,02	28,29	15613	228,42	29,24
sa omotačem	šifrovanje	23518	228,65	29,27	16765	239,82	30,70
	dešifrovanje	25015	218,18	27,93	18729	230,64	29,52

Protočna arhitektura zadržava prednost i na celom putu podataka. Logike joj treba od 1,34 do 1,42 puta manje. Razlike u učestanosti, od 0,5 do 5,7 procenata u njenu korist, ostaju unutar šuma alata iz odeljka 5.1, pa se sme tvrditi samo da protočna arhitektura manju površinu ne plaća učestanošću.

Sistemi omotač košta od 1255 do 3116 LUT-ova, više na putu dešifrovanja, jer uz razdvajanje i sastavljanje tamo stoji i poravnanje oznake. Učestanost se pritom menja u granicama šuma, a na putu šifrovanja protočne arhitekture omotač je čak podiže, sa 224,27 na 239,82 MHz, jer skid baferi omotača (odeljak 4.1.2) seku putanje koje bi inače prelazile granicu jezgra.

Granicu učestanosti celog puta podataka postavlja GCM sloj. Od delova koji se mere zasebno GHASH jedinica postiže 262,86 MHz, GCM sloj 227,46, a AES blok 301,93 u višejezgarnoj i 401,12 u protočnoj varijanti, prema tabelama 5.11, 5.10 i 5.9. Najniža je vrednost GCM sloja i svih osam merenja staje u njenoj okolini, od 218 do 240 MHz, u granicama šuma alata. AES blok granici ne prilazi ni u jednoj varijanti, pa se izbor arhitekture na učestanost celog puta ne odražava. Put podataka pritom daje blok po taktu, 128 bita po taktu, pa je gornja granica propusnosti sistema oko 30 Gb/s, a najviše izmereno 30,70.

Propusnost ispod te granice bira se brojem jezgara, po jednačini (5.7). Za cilj od 10 Gb/s dovoljno je šest jezgara, jer šest petnaestina granice GCM sloja od 29,12 Gb/s iznosi 11,6, dok pet daje 9,7. Najbliža merena konfiguracija, sedam jezgara sa 8345 LUT-ova u jezgru, cilj pokriva sa rezervom, za manje od polovine logike punih petnaest. Granica isplativosti višejezgarne varijante iz odeljka 5.4.3 važi i sa omotačem, jer GCM sloj i omotač koštaju približno isto u obe varijante, na putu šifrovanja 6558 naspram 6274 LUT-ova preko golog jezgra, a na putu dešifrovanja 8055 naspram 8238, pa se poređenje i ovde svodi na samo jezgro. U propusnosti ta granica na celom putu znači manje nego na golom jezgru. Sedam jezgara ovde nosi sedam petnaestina granice GCM sloja, oko 13,6 Gb/s, jer ceo put radi na učestanosti sloja, a ne jezgra. Dupleksni par sa omotačem, šifrovanje i dešifrovanje zajedno, staje u 48533 LUT-a u višejezgarnoj varijanti, odnosno 41,4 procenta čipa, a u protočnoj u 35494 LUT-a, odnosno 30,3 procenta.

Dvotaktna GHASH jedinica pomera granicu učestanosti celog puta podataka na oko 260 MHz. Sa njom GCM sloj sam postiže 302,18 MHz, a AES blok 301,93, ali deo te razlike ode na cenu spajanja, pa GCM lanac dostiže 266,18, a put sa omotačem 258,76 MHz. Granica u bitima po taktu se pritom prepolovi, pa propusnost padne sa oko 30 na oko 17 Gb/s, 64 bita po taktu na izmerenoj učestanosti, i jednotaktna organizacija ostaje radna tačka za najveću propusnost. Kada dvotaktna ipak ima smisla, pokazano je u odeljku 5.4.4.

Uz zauzeće i učestanost izmerena je, takt-tačno u simulaciji, i propusnost celog puta sa omotačem, nad nizom paketa sa korisnim sadržajem od 1504 bajta u istom rasporedu segmenata. Rezultat je dat u tabeli 5.16. Od 111 taktova po paketu 99 nosi reči paketa. Zastoj od dvanaest taktova čine osam taktova punjenja jezgra novim brojačkim nizom, tri takta čekanja na reč oznake i jedan takt na preklopu segmenata. Samo punjenje traje koliko i protočni niz, ali mu se početak sakrije ispod reči oznake i segmenata koji kroz jezgro ne prolaze, pa se vidi samo osam taktova. Vrednost je ista za obe arhitekture, jer obe drže blok po taktu pa je tok reči kroz put podataka identičan. Poklapanje sa vrednošću izmerenom bez omotača pokazuje da sistemski omotač ne košta propusnost. Istim nizom paketa proveren je i pun krug, u kome se svaki paket šifruje kroz šifarski IP, dešifruje kroz IP za dešifrovanje i vraća bajt-identičan uz potvrđenu oznaku.

Tabela 5.16: Propusnost celog puta podataka u ustaljenom stanju, nad paketima sa korisnim sadržajem od 1504 bajta. Kolona *bez omotača* je isti sadržaj iz tabele 5.14

Reči po paketu	Zastoj [taktova]	Taktova po paketu	Korisni sadržaj [b/takt]	Bez omotača [b/takt]
99	12	111	108,40	108,27

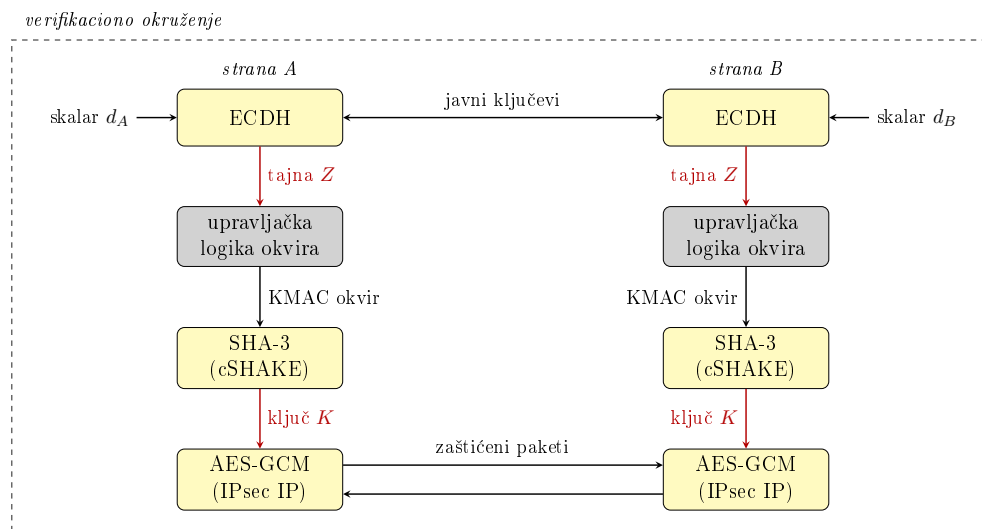
5.5 Sistem u celini

Tri jezgra zajedno pokrivaju ceo hibridni tok iz odeljka 2.4, a merenja iz ovog poglavlja daju svakom od njih radnu tačku. ECDH jezgro je familija od petnaest izmerenih konfiguracija u kojoj se površina i trajanje biraju generikom. Paralelno jezgro na približno istoj površini daje 2,7 puta kraće skalarno množenje, a najbolji proizvod površine i vremena mu je trećinu povoljniji od osnovnog. SHA-3 jezgro u prvoj arhitekturi postiže više od 400 MHz u svih sedam konfiguracija, a dvostruko zauzeće druge arhitekture nigde ne donosi dobitak izvan šuma alata, najbliže mu prilazi SHA3-512 sa magistralom od 64 bita. Kod AES-GCM akceleratora obe arhitekture dostižu blok po taktu. Protočna do njega stiže sa manje logike, pa je izbor za najveću propusnost, sa 30,70 Gb/s na celom putu podataka, dok dupleksni par sa omotačem staje u 30 procenata čipa (odeljak 5.4.7). Višejezgarna se plaća po jezgru, pa je jeftinija za ciljeve do oko 20 Gb/s na nivou jezgra, odnosno do oko 13,6 Gb/s na celom putu podataka. Na paketima realnih veličina akcelerator zadržava 85 procenata pune vrednosti.

Isti izbor konfiguracija daje i budžet čipa. Radna tačka ECDH jezgra, paralelna konfiguracija sa $D = 32$, zauzima 58112 LUT-ova i 25511 flipflopova, SHA-3 jezgro u prvoj arhitekturi kao SHA3-512 sa magistralom od 64 bita 4803 i 2882, a dupleksni AES-GCM par sa omotačem u protočnoj LUT varijanti 35494 i 15984. Zbir implementacija van konteksta iznosi 98409 LUT-ova, oko 84 procenta čipa, i 44377 flipflopova, ispod petine od 234240 raspoloživih. Taj zbir je procena, jer zajedničko rutiranje sva tri jezgra nije izvedeno, ali pokazuje da izabrane radne tačke na ploču staju sa oko 16 procenata rezerve u logici. Rezerva se može i proširiti, jer protočno

jezgro postoji i u BRAM varijanti, koja rundu drži u blok memoriji. Dupleksni par tada po istom računu pada na oko 21 hiljadu LUT-ova uz 104 memorijska bloka od 144 raspoloživa, koje druga dva jezgra ne koriste, pa ukupan zbir silazi na oko 84 hiljade LUT-ova, oko 72 procenta čipa. Pošto je na ovoj ploči upravo logika kritičan resurs, ta zamena je povoljnija, a učestanost celog puta ne menja, jer i BRAM varijanta jezgra ostaje iznad granice GCM sloja. Svako osetnije proširenje ipak traži veću platformu (poglavlje 6).

Zajednički rad sva tri jezgra proveren je simulacijom celog lanca sa slike 4.20, sa obe strane veze. Svaka strana ima svoje ECDH i SHA-3 jezgro i svoj par punih AES-GCM IP-ova sa sistemskim omotačem, u istom rasporedu segmenata ESP tunela kao na celom putu podataka, pa strane uspostavljaju ključ i njime razmenjuju pakete. Postavka je prikazana na slici 5.10. Verifikaciono okruženje igra ulogu upravljačke logike okvira, pa javne ključeve prenosi onako kako su izračunati, sklapa KMAC okvir oko tajne Z po odeljku 3.2.7 i izvedeni ključ predaje AES-GCM lancu. ECDH jezgra rade u paralelnoj konfiguraciji sa $D = 64$, izabranoj da skalarno množenje u simulaciji traje što kraće, a provera teče na Vivado simulatoru.



Slika 5.10: Lanac u simulaciji sa dve strane. Crvene strelice nose tajni materijal

Provera prolazi kroz sve karike. Obe strane izračunaju istu tajnu Z , jednaku referentnom modelu. Obe iz nje izvedu isti ključ sesije od 256 bita, ponovo jednak referentnom. Paket od 64 bajta putuje od jedne strane do druge, a nazad paket od 40 bajtova sa drugom nonce vrednošću. Šifrati i oznake poklapaju se sa referentnim modelima, primalac svaki paket vraća bajt-identičan i potvrđuje oznaku. Ceo tok, od predaje skalara do poslednje potvrđene oznake, staje u 33 290 taktova, od čega dva uzastopna skalarna množenja po strani nose gotovo sve. Time je popunjen i poslednji red tabele 5.1, pa je svaki nivo, od operacije u polju do zajedničkog rada celog lanca, pokriven bar jednom nezavisnom proverom. Pravci u kojima se sistem odavde razvija razmotreni su u poglavlju 6.

Poglavlje 6

Potencijalna unapređenja

Prethodna poglavlja zaokružila su tri jezgra i njihov zajednički lanac izvođenja i upotrebe ključa. Ovo poglavlje izlaže nadogradnje koje iz postignutog neposredno slede. Ne reda ih po jezgrima, nego po tri pravca u kojima bi se sistem dalje razvijao, od puta ka sistemu na ploči, preko pomeranja granice propusnosti puta podataka, do razvoja upravljačke ravni. Iznose se predlozi i njihove očekivane cene, a izmerene vrednosti na koje se predlozi oslanjaju pripadaju poglavlju 5.

6.1 Od simulisanog lanca do sistema na ploči

Lanac iz odeljka 4.5 proveren je u simulaciji, u kojoj ulogu upravljačke logike okvira igra verifikaciono okruženje. Na ploči bi taj posao preuzeo mali namenski automat, i on je prvi korak ka celini. Posao mu je pogodan za hardver, jer su ime funkcije i oznaka namene KMAC okvira fiksirani standardom, a javne delove, so i identitete strana, procesor upiše pri uspostavi sesije, pa je ceo okvir, osim same tajne, poznat pre razmene. Automat bi poznati deo čitao iz male memorije, tajnu Z preuzimao neposredno sa izlaza ECDH jezgra i sastavljeni niz predavao SHA-3 jezgru, čime bi tajni materijal i na ploči ostao unutar programabilne logike, kako predviđa slika 4.20. Ista razmena pritom može da ponese više od jednog ključa. KMAC iz iste tajne, uz različit kontekst ili veću traženu dužinu izlaza, izvodi onoliko ključeva koliko sistemu treba, po jedan za svaki smer ili za kasnije obnavljanje, pa bi uz automat stajala i mala memorija izvedenih ključeva.

Sa procesorske strane, upravljački program SHA-3 jezgra iz odeljka 5.3.4 već daje obrazac za ceo lanac, pa bi se proširio komandama za pokretanje razmene i čitanje statusa sesije. Nove veze između jezgara na ploči dobile bi iste skid bafere koji su se na celom putu podataka pokazali bez merljive cene u učestanosti (odeljak 5.4.7), uz pun regresioni prolaz posle izmene. Ciljna platforma nudi i delimičnu rekonfiguraciju (engl. *dynamic function exchange*, DFX), a šta ona donosi zavisi od uloge sistema. Na strani klijenta ECDH jezgro radi jednom po sesiji, a u paralelnim konfiguracijama zauzima i preko trećine čipa (odeljak 5.2), pa bi se njegova površina posle razmene mogla prepustiti drugom sadržaju, drugom mrežnom kanalu ili nekom drugom algoritmu razmene. Na serverskoj strani, koja razmene vodi za mnogo klijenata istovremeno, jezgro radi neprekidno, pa merodavne postaju latencija skalarnog množenja i broj razmena u sekundi, a izbor se pomera ka najbržim konfiguracijama iz istog odeljka.

Poglavlje 1 predviđa i autentifikaciju učesnika algoritmom ECDSA [38]. Njegova realizacija bi merdevine iz odeljka 4.2.5 upotrebila ponovo za skalarno množenje, ali pored njih traži i aritmetiku po modulu reda tačke, dakle po velikom prostom

broju, koje u sadašnjem jezgru nema. Prirodan oblik nadogradnje je zaseban blok za množenje i inverziju po tom modulu, dok bi kriva, tačke i skalarno množenje ostali zajednički. Pun obim je ipak veći, jer ECDSA traži i kvalitetan generator slučajnih brojeva i sabiranje dve nezavisne tačke pri proveru potpisa, pa aritmetički blok pokriva prvi i najveći deo puta. Time bi se zaokružio i bezbednosni model iz poglavlja 2, u kome razmena ključa bez autentifikacije učesnika ne štiti od aktivnog napadača.

6.2 Pomeranje granice propusnosti puta podataka

Merenja iz odeljka 5.4.7 pokazuju da put podataka drži blok po taktu i da granicu učestanosti postavlja GCM sloj, ispod koga ceo put podataka staje još za cenu spajanja. Granica pritom ne stoji u šifarskom bloku, jer oba AES bloka sama postižu znatno više od celog puta (odeljak 5.4.2). Stoji u povratnoj petlji GHASH akumulatora, koju registar ne može da preseče a da granica u bitima po taktu ne padne, jer svaki blok zavisi od prethodnog. Rad je zato petlju ipak presekao tamo gde je cilj propusnosti niži, jer dvotaktna organizacija skraćuje kritičnu putanju i olakšava sklapanje sistema, po ceni prepolovljene granice (odeljak 5.4.7). Naredni oblik akumulacije istu ideju zadržava bez te cene.

Povratna petlja ipak može da se ubrza, ali tek zajedno sa oblikom akumulacije. Nad parom blokova akumulacija uz potključ H koristi i H^2 , pa množenje sa H izlazi iz povratne petlje, a ona se zatvara kroz množač sa H^2 jednom u dva takta. U nju zato staje množač sa množenjem u dva takta, onaj koji u GCM sloju sam postiže 302,18 MHz (tabela 5.11), a koji je do sada polovio propusnost baš zato što je petlju zatvarao svaki takt. Dva množača sa prepolovljenim putanjama tako zadržavaju blok po taktu, a oba šifarska bloka imaju prostora da prate, protočni sa 401,12 MHz, a višezegarni sa 301,93. Ceo put podataka sa dvotaktnim množačem već dostiže oko 260 MHz, pa bi sa ovakvom akumulacijom na istoj učestanosti nosio pun blok po taktu, oko 33 Gb/s umesto sadašnjih 30. Postupak se uopštava na stepene do H^k , gde se petlja zatvara jednom u k taktova. Sa više taktova po množenju množač ne mora ni da ostane u čisto paralelnom Karatsubinom obliku iz odeljka 4.4.3, jer tada u petlju staju i digit-serijske i hibridne organizacije, spoj digit-serijskog toka i Karatsubine dekompozicije, koje kraćim kombinacionim putanjama potencijalno postižu još višu učestanost. Ulaz od dva bloka po taktu, uz ista dva množača u jednotaktnom obliku, udvostručio bi i granicu u bitima po taktu, ali tada ide u paru sa udvostručenim šifarskim putem, na primer sa dva protočna niza i raspodelom paketa među njima. Brzine reda 100 Gb/s traže još širu obradu. Samom AES jezgru dovoljna bi bila potpuno paralelna organizacija na magistrali od 256 bitova, ali ceo AES-GCM put, sa autentifikacionim putem koji granicu mora da prati, realno ide na magistralu od 512 bitova, četiri bloka po taktu, kakvom se sto gigabita u sekundi obično i prenosi. Za takve ciljeve ovaj sistem je na sadašnjoj ploči već ambiciozan, pa je predviđen prelazak na noviju i snažniju platformu.

Kod SHA-3 jezgra model iz odeljka 5.3.3 pokazuje da pri dve runde po taktu granicu u većini konfiguracija postavlja unos bloka. Šira ulazna magistrala vratila bi granicu na permutaciju, po ceni šireg upisnog puta u ulaznom baferu.

6.3 Upravljačka ravan i budućnost razmene ključa

Prirodna nadogradnja upravljačke ravni je provera primljenog javnog ključa. Jezgro već prima obe koordinate tačke, a provera da tačka leži na krivoj traži da se jednačina krive izračuna i uporedi sa nulom: dva množenja, dva kvadriranja, tri sabiranja i poređenje, ukupno mikrogram od osam operacija po ugledu na završnu konverziju iz odeljka 4.2.5, uz jedan statusni bit ka pozivaocu. Cena od približno $2q$ taktova je ispod desetine procenta trajanja skalarnog množenja (odeljak 5.2.3), pa bi provera mogla biti stalno uključena. Puna provera po SP 800-56A [23], koja potvrđuje i red tačke, košta ceo drugi prolaz merdevina i pripada protokolskom sloju iznad jezgra.

Postojeća aritmetika može i bolje da se iskoristi. Merdevine i završna konverzija paralelnog jezgra nikada ne rade istovremeno, pa se tri množača iz odeljka 4.2.6 mogu podeliti sa konverzijom, umesto da ona drži sopstveni, četvrti množač. Konverzija bi se pritom i ubrzala, jer se njena množenja daju rasporediti na tri množača, a inverzija može da pozajmi drugi za svoje korake kvadriranja i množenja. Isti mehanizam otvara aritmetiku i ka spolja, jer se jedan množač može izložiti preko postojećeg start i done interfejsa, na primer budućem ECDSA bloku.

Prostor postoji i u samim algoritmima množenja u polju, najavljenim u odeljku 4.2.2. Množač GHASH jedinice već pokazuje kako rekursivna Karatsubina dekompozicija smanjuje broj elementarnih množenja po ceni dublje mreže, a ista zamena važi i za digit-serijski množač polja, gde hibrid klasičnog i Karatsubinog množenja skraćuje kombinacionu putanju jednog digita [39]. Kraća putanja diže učestanost, a sa njom i prostor da se pri istom taktu širina digita poveća.

Na duže staze, pomenuto povlačenje binarnih krivih za nove primene (odeljak 3.1.4) znači da nadogradnja u nekom trenutku podrazumeva prelazak na krive nad prostim poljima. Aritmetika se time menja iz korena, jer množenje nad prostim poljem nosi prenos, pa umesto XOR mreža dolaze sabirači i namenski DSP blokovi, koje ciljna platforma nudi u velikom broju, a sadašnji dizajn ne koristi nijedan.

Najdalji korak za ovaj sistem, ali danas već i preporučeni put za razmenu ključa, jeste postkvantna zamena. Standard FIPS 203 [40] propisuje ML-KEM kao mehanizam enkapsulacije ključa, pa hibridni tok iz odeljka 2.4 zadržava isti oblik, u kome mehanizam razmene daje tajnu, iz tajne se izvodi ključ sesije, a ključ štiti saobraćaj. U takvoj zameni ECDH jezgro ustupa mesto ML-KEM jezgru, dok SHA-3 jezgro dobija na značaju, jer ML-KEM sve svoje unutrašnje funkcije heširanja i proširivanja gradi nad Keccak permutacijom, istom onom koju ovaj rad već realizuje u hardveru.

Poglavlje 7

Zaključak

U radu je projektovan, realizovan i izmeren kriptografski akcelerator koji na jednoj FPGA platformi objedinjuje tri algoritma savremene kriptografije: ECDH razmenu ključa nad krivom B-571, SHA-3 sa KMAC konstrukcijom za izvođenje ključeva i AES-GCM za zaštitu saobraćaja. Time je pokriven ceo hibridni tok iz poglavlja 2, od uspostavljanja zajedničke tajne do zaštićenog paketa. Parametri sva tri algoritma birani su usklađeno, pa sistem u celini pruža sigurnosni nivo od 256 bitova, jer razmena ključa nosi približno 285, a izvođenje ključeva i šifrovanje po 256 bitova, dok oznaka od 128 bitova ostaje zasebna kategorija, jer se napad na nju izvodi jedino kroz živu vezu, pokušaj po pokušaj, pa njena dužina ukupni nivo ne spušta (tabela 2.3). Sva tri jezgra su parametrizovana genericima, napisana u VHDL-u i izmerena po jedinstvenoj metodologiji, post-implementaciono i sa taktom kao jedinim ograničenjem (odjeljak 5.1).

Merenja su dala jasnu sliku svakog jezgra. Kod ECDH akceleratora izmerena latencija potvrđuje procenu od šest naspram dva množenja po iteraciji merdevina, pa paralelno jezgro na približno istoj površini daje 2,7 puta kraće skalarno množenje, familija od petnaest izmerenih konfiguracija pokriva raspon od oko petnaest procenata do četiri petine čipa, a najbrža od njih opsluži oko 3875 razmena ključa u sekundi. SHA-3 jezgro sa jednom rundom po taktu postiže više od 400 MHz u svih sedam konfiguracija, a najviše 18,49 Gb/s kod varijante SHA3-256 sa magistralom od 64 bita. Propusnost u svim konfiguracijama određuju unos bloka i trajanje permutacije, uz jedan takt predaje bloka, a izvođenje ključa kroz KMAC okvir staje u tri desetinke mikrosekunde. AES-GCM akcelerator drži blok po taktu sa obe arhitekture AES jezgra koje šifrjuje brojački niz CTR režima, protočnom i višejezgarnom. Protočno jezgro samo dostiže oko 51 Gb/s i na celom putu podataka traži oko 1,4 puta manje logike od višejezgarnog, a višejezgarno je jeftiniji izbor za ciljeve do oko 20 Gb/s na nivou jezgra, odnosno do oko 13,6 Gb/s na celom putu podataka. Na paketima od 1504 bajta akcelerator zadržava 85 procenata pune vrednosti korisnog sadržaja. Gornja granica je 95 procenata, jer zaglavlja i oznaka nose ostatak paketa, a razliku do nje plaća pre svega punjenje jezgra novim brojačkim nizom.

Granicu učestanosti celog puta podataka postavlja GCM sloj, zbog povratne petlje GHASH akumulatora, pa put daje blok po taktu i do oko 30 Gb/s. Paket od 1504 bajta kroz njega prolazi sa 108,40 korisnih bitova po taktu, a sistemski omotač ne košta ni učestanost ni propusnost. Dupleksni par, šifrovanje i dešifrovanje zajedno, staje u 41,4 procenta čipa u višejezgarnoj, odnosno 30,3 procenta u protočnoj varijanti, pa pored akceleratora na čipu ostaje mesta i za ostatak sistema. Sva tri jezgra zajedno, u izabranim radnim tačkama, staju u oko 84 procenta čipa, a sa BRAM varijantom protočnog jezgra u oko 72 (odjeljak 5.5).

Ispravnost je proveravana po nivoima, od operacija u polju do celog sistema. Pored testova sa poznatim odgovorima po standardima, SHA-3 jezgro je provereno i na ploči pod PetaLinux sistemom preko namenskog upravljačkog programa, AES-GCM akcelerator u mrežnom sistemu sa generatorom saobraćaja, a ceo lanac simulacijom u kojoj dve strane uspostave ključ pa razmene zaštićene pakete (odjeljak 5.5).

Pravci daljeg razvoja izloženi su u poglavlju 6, od namenske upravljačke logike okvira koja lanac zaokružuje na ploči, preko pomeranja granice propusnosti puta podataka i prelaska na snažniju platformu za najveće brzine, do postkvantne zamene razmene ključa, danas već i preporučenog puta, u kojoj Keccak permutacija, koju ovaj rad već ubrzava, ostaje središte i budućeg sistema.

Dodatak A

Matematičke osnove

U ovom dodatku sistematizovane su matematičke osnove na koje se oslanjaju sva tri algoritma iz rada, teorija konačnih polja. Cilj dodatka nije ponavljanje kompletnih izvođenja, jer je detaljna razrada, sa dokazima da navedene strukture zaista jesu grupe, prsteni i polja, data u dodatku rada [27]. Ovde je dat pregled definicija i svojstava koja se u glavnom tekstu koriste, sa naglaskom na tri konkretna polja: $\text{GF}(2^8)$ kod AES-a, $\text{GF}(2^{571})$ kod eliptičke krive B-571 i $\text{GF}(2^{128})$ kod GHASH funkcije.

A.1 Grupa, prsten i polje

Abelova grupa $(G, +)$ je skup G sa binarnom operacijom $+$ koja zadovoljava: zatvorenost, asocijativnost, postojanje neutralnog elementa, postojanje inverznog elementa za svaki element skupa i komutativnost.

Prsten $(R, +, \cdot)$ je skup sa dve operacije, takav da je $(R, +)$ Abelova grupa, operacija $\cdot$ asocijativna i distributivna prema $+$. Ako je i $\cdot$ komutativna, prsten je komutativan.

Polje $(F, +, \cdot)$ je komutativan prsten sa jedinicom u kome svaki element različit od nule ima multiplikativni inverz. U polju su, dakle, definisane sve četiri osnovne računske operacije, pri čemu deljenje predstavlja množenje inverzom.

A.2 Konačna (Galoisova) polja

Konačno ili Galoisovo polje je polje sa konačnim brojem elemenata. Osnovna teorema teorije konačnih polja kaže da polje sa q elemenata postoji ako i samo ako je $q = p^n$ stepen prostog broja p , i da je tada jedinstveno do na izomorfizam, a označava se sa $\text{GF}(p^n)$. Broj p naziva se karakteristika polja.

Za $n = 1$ polje $\text{GF}(p)$ čine ostaci pri deljenju sa p , sa sabiranjem i množenjem po modulu p . Za $n > 1$ elementi polja $\text{GF}(p^n)$ predstavljaju se kao polinomi stepena manjeg od n sa koeficijentima iz $\text{GF}(p)$, što je takozvana polinomijalna baza. Sabiranje je sabiranje koeficijenata u $\text{GF}(p)$, a množenje je množenje polinoma po modulu izabranog nesvodljivog polinoma $f(x)$ stepena n . Nesvodljivost polinoma f garantuje da svaki element različit od nule ima inverz.

U kriptografskom hardveru posebno mesto imaju polja karakteristike 2, binarna polja $\text{GF}(2^n)$: element polja je binarna reč od n bitova, sabiranje je bitska XOR operacija bez prenosa, a množenje se gradi od AND i XOR mreža. Sva tri polja korišćena u ovom radu su binarna.

A.3 Polje $\text{GF}(2^8)$ u AES-u

AES bajtove tumači kao elemente polja $\text{GF}(2^8)$ sa nesvodljivim polinomom

$$m(x) = x^8 + x^4 + x^3 + x + 1 = \{11B\}. \quad (\text{A.1})$$

Bajt $b_7b_6 \dots b_0$ predstavlja polinom $b_7x^7 + \dots + b_1x + b_0$. Osnovna operacija množenja sa x , odnosno sa $\{02\}$, je pomeranje ulevo praćeno, ako je najstariji bit bio jedinica, redukcijom XOR-om sa $\{1B\}$. Opšte množenje se dobija razlaganjem množioca na stepene x (primer je dat u glavnom tekstu, u odeljku o MixColumns transformaciji). Multiplikativni inverz, potreban za S-boks, može se izračunati proširenim Euklidovim algoritmom ili, po Fermaovoj maloj teoremi, kao $b^{-1} = b^{254}$. Kompletne tabele S-boksa data je u dodatku B.

A.4 Polje $\text{GF}(2^{571})$ kod eliptičke krive B-571

Aritmetika tačaka krive B-571 svodi se na operacije u polju $\text{GF}(2^{571})$ sa redukcionim pentanomom

$$f(x) = x^{571} + x^{10} + x^5 + x^2 + 1. \quad (\text{A.2})$$

Pentanom je izabran standardom tako da redukcija bude plitka mreža XOR kola. Stepene viška (x^{571} i viši) zamenjuju se sa svega četiri niža stepena. Kvadriranje je u binarnom polju linearna operacija, jer se kvadriranjem polinoma koeficijenti samo razmaknu na parne pozicije ($x^i \mapsto x^{2i}$), pa se kvadriranje sa redukcijom realizuje kao čisto kombinatorna mreža, za razliku od opšteg množenja koje je dominantna operacija po ceni. Inverzija se izvodi Fermaovim pristupom (lanac kvadriranja i množenja, npr. Itoh–Tsuiji metodom) ili proširenim Euklidovim algoritmom, i tipično se svodi na jednu jedinu inverziju na kraju skalarnog množenja. Kompletni parametri krive dati su u dodatku C.

A.5 Polje $\text{GF}(2^{128})$ u GHASH funkciji

GHASH funkcija GCM režima računa u polju $\text{GF}(2^{128})$ sa redukcionim polinomom

$$f(x) = x^{128} + x^7 + x^2 + x + 1. \quad (\text{A.3})$$

Specifičnost standarda [9] je konvencija o redosledu bitova. Krajnji levi bit 128-bitnog bloka je koeficijent uz x^0 , a krajnji desni uz x^{127} , obrnuto od prirodnog binarnog zapisa. Na matematička svojstva polja to nema uticaja, ali hardverski množač mora biti projektovan (ili mu ulazi ogledalski preuređeni) u skladu sa konvencijom. Množenje u ovako velikom polju se u praksi ne realizuje direktno, nego Karatsubinom dekompozicijom. Postupak je opisan u odeljku 3.3.9.3, a izabrana dubina rekurzije i njena cena u odeljku 4.4.3.

Dodatak B

AES tabele

B.1 S-boks

Tabela B.1 sadrži kompletan S-boks AES algoritma [4]. Vrsta se bira gornjim, a kolona donjim polubajtom ulaznog bajta. Za ulaz {53} vrednost se nalazi u vrsti 5 i koloni 3, i iznosi {ED}, što je primer korišćen u odeljku o SubBytes transformaciji. Inverzni S-boks u ovom radu nije potreban, budući da GCM režim koristi AES isključivo u smeru šifrovanja.

Tabela B.1: AES S-boks: vrsta je gornji, kolona donji polubajt ulaza (heksadecimalno)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

B.2 Konstante rundi ekspanzije ključa

Tabela B.2 sadrži konstante $Rcon_i = (x^{i-1}, \{00\}, \{00\}, \{00\})$ koje se koriste u ekspanziji ključa. Prikazan je prvi bajt x^{i-1} , izračunat u polju $GF(2^8)$. AES-256 koristi konstante do $Rcon_7$, dok AES-128 koristi svih deset.

Tabela B.2: Rcon konstante ekspanzije ključa (prvi bajt, heksadecimalno)

i	1	2	3	4	5	6	7	8	9	10
$Rcon_i$	01	02	04	08	10	20	40	80	1B	36

Dodatak C

Parametri krive B-571

Kriva B-571 definisana je preporukom [18] nad poljem $\text{GF}(2^{571})$ sa redukcionim pentanomom $f(x) = x^{571} + x^{10} + x^5 + x^2 + 1$, jednačinom $y^2 + xy = x^3 + ax^2 + b$ i sledećim parametrima ($a = 1$, kofaktor $h = 2$). Vrednosti su date heksadecimalno, u grupama od po 32 bita, sa vodećim nulama do pune dužine od 576 bitova (144 heksadecimalne cifre).

Koeficijent b :

02F40E7E 2221F295 DE297117 B7F3D62F 5C6A97FF CB8CEFF1 CD6BA8CE 4A9A18AD
84FFABBD 8EFA5933 2BE7AD67 56A66E29 4AFD185A 78FF12AA 520E4DE7 39BACAOC
7FFEFF7F 2955727A

Koordinata G_x osnovne tačke:

0303001D 34B85629 6C16C0D4 0D3CD775 0A93D1D2 955FA80A A5F40FC8 DB7B2ABD
BDE53950 F4C0D293 CDD711A3 5B67FB14 99AE6003 8614F139 4ABFA3B4 C850D927
E1E7769C 8EEC2D19

Koordinata G_y osnovne tačke:

037BF273 42DA639B 6DCCFFFE B73D69D7 8C6C27A6 009CBBCA 1980F853 3921E8A6
84423E43 BAB08A57 6291AF8F 461BB2A8 B3531D2F 0485C19B 16E2F151 6E23DD3C
1A4827AF 1B8AC15B

Red n osnovne tačke:

03FFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
FFFFFFF E661CE18 FF559873 08059B18 6823851E C7DD9CA1 161DE93D 5174D66E
8382E9BB 2FE84E47

Red n je prost broj dužine 570 bitova (odjeljak 3.1.6).

Dodatak D

Keccak konstante

D.1 Rotacioni ofseti

Tabela D.1 sadrži rotacione ofsete $r_{x,y}$ koraka ρ permutacije Keccak- $f[1600]$ [16]: traka $A_{x,y}$ ciklično se pomera za $r_{x,y}$ bitova. U hardveru se ove rotacije svode na preusmeravanje veza (odjeljak 3.2.5).

Tabela D.1: Rotacioni ofseti $r_{x,y}$ koraka ρ

	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$
$y = 0$	0	1	62	28	27
$y = 1$	36	44	6	55	20
$y = 2$	3	10	43	25	39
$y = 3$	41	45	15	21	8
$y = 4$	18	2	61	56	14

D.2 Konstante rundi

Tabela D.2 sadrži 24 konstante rundi RC_i koraka ι , koje se sabiraju sa trakom $A_{0,0}$ (odjeljak 3.2.5). Konstante su definisane linearnim povratnim pomeračkim registrom iz standarda [16].

Tabela D.2: Konstante rundi RC_i permutacije Keccak- $f[1600]$ (heksadecimalno)

i	RC_i	i	RC_i
0	0000000000000001	12	000000008000808B
1	0000000000008082	13	800000000000008B
2	800000000000808A	14	8000000000008089
3	8000000080008000	15	8000000000008003
4	000000000000808B	16	8000000000008002
5	0000000080000001	17	8000000000000080
6	8000000080008081	18	000000000000800A
7	8000000000008009	19	800000008000000A
8	000000000000008A	20	8000000080008081
9	0000000000000088	21	8000000000008080
10	0000000080008009	22	0000000080000001
11	000000008000000A	23	8000000080008008

Reference

- [1] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010.
- [2] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*. CRC Press, 3rd ed., 2020.
- [3] A. Kerckhoffs, “La cryptographie militaire,” *Journal des sciences militaires*, vol. IX, pp. 5–38, 1883.
- [4] National Institute of Standards and Technology, “Advanced encryption standard (AES),” Tech. Rep. FIPS PUB 197, U.S. Department of Commerce, 2001.
- [5] E. Barker and A. Roginsky, “Transitioning the use of cryptographic algorithms and key lengths,” Tech. Rep. NIST Special Publication 800-131A, Rev. 2, National Institute of Standards and Technology, 2019.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF protocols.” RFC 8439, IETF, 2018.
- [7] M. S. Turan, K. McKay, D. Chang, J. Kang, and J. Kelsey, “ASCON-based lightweight cryptography standards for constrained devices,” Tech. Rep. NIST Special Publication 800-232, National Institute of Standards and Technology, 2025.
- [8] M. Dworkin, “Recommendation for block cipher modes of operation: Methods and techniques,” Tech. Rep. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.
- [9] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” Tech. Rep. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
- [10] T. Krovetz and P. Rogaway, “The OCB authenticated-encryption algorithm.” RFC 7253, IETF, 2014.
- [11] M. Dworkin, “Recommendation for block cipher modes of operation: The CCM mode for authentication and confidentiality,” Tech. Rep. NIST Special Publication 800-38C, National Institute of Standards and Technology, 2004.
- [12] A. Inoue, T. Iwata, K. Minematsu, and B. Poettering, “Cryptanalysis of OCB2: Attacks on authenticity and confidentiality,” in *Advances in Cryptology — CRYPTO 2019*, pp. 3–31, Springer, 2019.
- [13] W. Diffie and M. E. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.

- [14] N. Koblitz, “Elliptic curve cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987.
- [15] V. S. Miller, “Use of elliptic curves in cryptography,” in *Advances in Cryptology — CRYPTO ’85*, vol. 218 of *LNCS*, pp. 417–426, Springer, 1986.
- [16] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” Tech. Rep. FIPS PUB 202, U.S. Department of Commerce, 2015.
- [17] D. Hankerson, A. Menezes, and S. Vanstone, *Guide to Elliptic Curve Cryptography*. Springer, 2004.
- [18] L. Chen, D. Moody, K. Randall, A. Regenscheid, and A. Robinson, “Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters,” Tech. Rep. NIST SP 800-186, National Institute of Standards and Technology, 2023.
- [19] T. Itoh and S. Tsujii, “A fast algorithm for computing multiplicative inverses in $GF(2^m)$ using normal bases,” *Information and Computation*, vol. 78, no. 3, pp. 171–177, 1988.
- [20] J. Guajardo and C. Paar, “Itoh-tsujii inversion in standard basis and its application in cryptography and codes,” *Designs, Codes and Cryptography*, vol. 25, no. 2, pp. 207–216, 2002.
- [21] P. L. Montgomery, “Speeding the Pollard and elliptic curve methods of factorization,” *Mathematics of Computation*, vol. 48, no. 177, pp. 243–264, 1987.
- [22] J. López and R. Dahab, “Fast multiplication on elliptic curves over $GF(2^m)$ without precomputation,” in *Cryptographic Hardware and Embedded Systems — CHES ’99*, vol. 1717 of *LNCS*, pp. 316–327, Springer, 1999.
- [23] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, “Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography,” Tech. Rep. NIST Special Publication 800-56A, Rev. 3, National Institute of Standards and Technology, 2018.
- [24] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, “The Keccak reference.” <https://keccak.team/>, 2011.
- [25] J. Kelsey, S. jen Chang, and R. Perlner, “SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash,” Tech. Rep. NIST Special Publication 800-185, National Institute of Standards and Technology, 2016.
- [26] E. Barker, L. Chen, and R. Davis, “Recommendation for key-derivation methods in key-establishment schemes,” Tech. Rep. NIST Special Publication 800-56C, Rev. 2, National Institute of Standards and Technology, 2020.
- [27] M. Gavrilović, “Efikasna hardverska realizacija AES algoritma za enkripciju,” završni rad, Univerzitet u Beogradu, Elektrotehnički fakultet, 2024.
- [28] J. Daemen and V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer, 2002.

- [29] J. Savard, “Aes encryption round visualization,” 2024. Accessed: 23-09-2024, Licensed under CC0. Available at <https://commons.wikimedia.org/w/index.php?curid=103718081>.
- [30] D. A. McGrew and J. Viega, “The Galois/Counter Mode of operation (GCM).” Submission to NIST Modes of Operation Process, 2004.
- [31] A. Karatsuba and Y. Ofman, “Multiplication of many-digital numbers by automatic computers,” *Proceedings of the USSR Academy of Sciences*, vol. 145, pp. 293–294, 1962.
- [32] A. Weimerskirch and C. Paar, “Generalizations of the Karatsuba algorithm for efficient implementations,” *Cryptology ePrint Archive, Report 2006/224*, 2006. <https://eprint.iacr.org/2006/224>.
- [33] A. Joux, “Authentication failures in NIST version of GCM,” in *Comments submitted to NIST Modes of Operation Process*, 2006.
- [34] ARM Ltd., “AMBA® 4 AXI4-Stream protocol specification.” <https://developer.arm.com/documentation/ih10051/a/>, 2010.
- [35] K. Okeya and K. Sakurai, “Efficient elliptic curve cryptosystems from a scalar multiplication algorithm with recovery of the y-coordinate on a Montgomery-form elliptic curve,” in *Cryptographic Hardware and Embedded Systems — CHES 2001*, vol. 2162 of *LNCS*, pp. 126–141, Springer, 2001.
- [36] M. Sundal and R. Chaves, “Efficient FPGA implementation of the SHA-3 hash function,” in *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 86–91, 2017.
- [37] J. Viega and D. McGrew, “The use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP).” RFC 4106, IETF, 2005.
- [38] National Institute of Standards and Technology, “Digital signature standard (DSS),” Tech. Rep. FIPS PUB 186-4, U.S. Department of Commerce, 2013.
- [39] K. Kumari, G. K. Purohit, and A. Karmakar, “Efficient hardware implementation of modular multiplier over $GF(2^m)$ on FPGA,” 2025. arXiv:2506.09464.
- [40] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” Tech. Rep. FIPS PUB 203, U.S. Department of Commerce, 2024.